

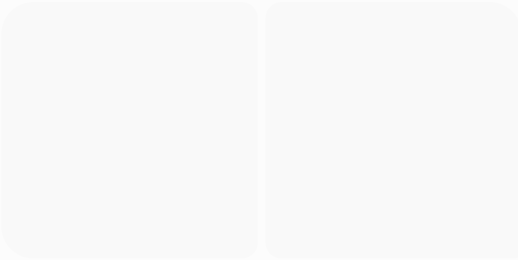
첨부 화면은 현재 behavior야. 저것을 먼저 right-left slide in으로 바꿔

Receipt scanned를 menus scanned 아래로 옮겨라. 이것은
현재 비어있는 category다

put the current three menus under the menus scanned, and put receipts scanned category title under the menus scanned category. the receipts scanned category is currently empty.



a receipt was scanned, but the history shows previous menu scanning records.



a receipt is scanned with 7 items, but history shows a receipt record with a wrong restaurant name. the receipt record has no restaurant name.

it still put a wrong restaurant name.

in history, receipt records should be expandable.

receipts scanned item shows two different colors. make them unnotieable (make them the same color or one transparent).

```
//  
// HistoryDayOverlay.swift  
// MenuCal  
//  
// Created by Dae-Kyoo Kim on 12/28/25.  
//  
import SwiftUI  
  
// MARK: - History Category Enum  
private enum HistoryCategory {  
    case menus  
    case receipts  
}  
  
struct HistoryDayOverlay: View {  
    let wrapper: MenuScanListWrapper  
    let onBack: () -> Void  
  
    @Namespace private var categoryNamespace  
  
    @StateObject private var historyStore =  
HistoryRecordStore.shared  
    @StateObject private var favRestaurantStore =  
FavoriteRestaurantStore.shared
```

```

@StateObject private var favMenuStore =
FavoriteMenuStore.shared
private let receiptStore = ReceiptScanStore.shared
@State private var expandedReceiptIDs: Set<UUID> =
[]

@State private var isVisible = false

@State private var receiptRecords:
[ReceiptScanRecord] = []

private func reloadReceiptRecords() {
    receiptRecords = receiptStore.loadAll().filter {
        Calendar.current.isDate($0.scannedAt,
inSameDayAs: wrapper.date)
    }
}

private func closePanel() {
    withAnimation(.easeInOut(duration: 0.3)) {
        isVisible = false
    }
    DispatchQueue.main.asyncAfter(deadline: .now() +
0.3) {
        onBack()
    }
}

var body: some View {
    GeometryReader { geo in
        ZStack {
            Color.black
                .opacity(isVisible ? 0.5 : 0)
                .ignoresSafeArea()
                .animation(.easeInOut(duration: 0.3), value:
isVisible)
                .onTapGesture {
                    closePanel()
                }

            HStack {
                Spacer()

                VStack(spacing: 16) {
                    // =====
                    // 🏠 Header

```

```

// =====
HStack {
  Button(action: {
    closePanel()
  }) {
    HStack(spacing: 4) {
      Image(systemName: "chevron.left")
      Text("Back")
    }
    .foregroundColor(.white)
    .font(.system(size: 17, weight:
.semibold))
  }
  Spacer()
}
.padding(.horizontal)
.padding(.top, 12)

Text(wrapper.date.toYMD())
  .font(.system(size: 28, weight: .bold))
  .foregroundColor(.white)

// =====
// 📁 Menus Scanned Header
// =====
Text("Menus Scanned")
  .fontWeight(.bold)
  .foregroundColor(.white)
  .frame(maxWidth: .infinity, alignment:
.leading)
  .padding(.horizontal)
  .padding(.top, 4)

// =====
// 📅 Menus + Receipts (Single Scroll Flow)
// =====
ScrollView {
  VStack(alignment: .leading, spacing: 20)
{

  // ===== Menus Scanned List =====
  ForEach(
    historyStore.visibleRecords.filter {
      Calendar.isSameDay($0.scannedAt, wrapper.date)
    }
    ) { scan in

```



```

        ScanCard(
            scan: scan,
            favRestaurantStore:
favRestaurantStore,
            favMenuStore: favMenuStore
        )
    }

    // ===== Divider =====
    Divider()

    .background(Color.white.opacity(0.3))
        .padding(.vertical, 8)

    // ===== Receipts Scanned Header
=====
        Text("Receipts Scanned")
            .fontWeight(.bold)
            .foregroundColor(.white)
            .frame(maxWidth: .infinity,
alignment: .leading)
            .padding(.top, 4)

    // ===== Receipts Placeholder =====
    Group {
        if receiptRecords.isEmpty {
            VStack {
                Spacer(minLength: 40)
                Text("No receipts scanned
yet")

                .foregroundColor(.white.opacity(0.7))
                    .font(.system(size: 16,
weight: .medium))

                Spacer()
            }
        } else {
            VStack(alignment: .leading,
spacing: 12) {
                ForEach(receiptRecords) {
record in

                    let isExpanded =
expandedReceiptIDs.contains(record.id)

                    HStack(alignment: .top,
spacing: 0) {

```

```

// 1. Content Card (Now
forced to be transparent)
ReceiptHistoryCard(
    record: record,
    isExpanded: isExpanded
)
.frame(maxWidth: .infinity,
alignment: .leading)

// Set background to clear
here to override any internal defaults
.background(Color.clear)

// 2. Trash Button
Button {
    receiptStore.delete(id:
record.id)

    reloadReceiptRecords()
} label: {
    Image(systemName:
"trash")

.foregroundColor(.red.opacity(0.85))
.font(.system(size:
16))

.padding(.trailing, 16)
.padding(.top, 18)
}
.buttonStyle(.plain)
}
// ✅ Apply the SINGLE
background color to the entire row
// Use the exact color from
your Menus Scanned section
.background(Color(white:
0.15))

.cornerRadius(12)
.contentShape(Rectangle())
.onTapGesture {

withAnimation(.easeInOut(duration: 0.25)) {
    if isExpanded {

expandedReceiptIDs.remove(record.id)
    } else {

expandedReceiptIDs.insert(record.id)

```

```

        }
    }
}
}
}
    .padding(.top, 8)
}
}
    .frame(maxWidth: .infinity)
}
    .padding(.horizontal)
    .padding(.vertical)
}
    Spacer(minLength: 0)
}
    .frame(width: geo.size.width * 0.88, height:
geo.size.height)
    .background(Color.black)
    .offset(x: isVisible ? 0 : geo.size.width)
    .animation(.easeInOut(duration: 0.3), value:
isVisible)
    .contentShape(Rectangle())
    .background(
        Color.clear
        .contentShape(Rectangle())
        .onTapGesture { closePanel() }
    )
}
}
    .onAppear {
        withAnimation(.easeInOut(duration: 0.3)) {
            isVisible = true
        }
        reloadReceiptRecords()
    }
}
}
}
}

```

Show less ^

recipes reviewed section should list recipe names reviewed.

the recipes viewed section shows scanned menu items,
not recipes.

in the attached image, the trash next menu item is not working. it does not remove the item on the view immediately. when the view is reopened it is gone, but when the app restarts, it is back. it seems the trash removes it only from the memory but fails to reflect on the view immediately. and when the app starts again, it reads the record from the DB and make it re-appear. check the following code.

```
//  
// HistoryDayOverlay.swift  
// MenuCal  
//  
// Created by Dae-Kyoo Kim on 12/28/25.
```

```
//
import SwiftUI

// MARK: - History Category Enum
private enum HistoryCategory {
    case menus
    case receipts
}

struct HistoryDayOverlay: View {
    let wrapper: MenuScanListWrapper
    let onBack: () -> Void

    @Namespace private var categoryNamespace

    @StateObject private var historyStore =
HistoryRecordStore.shared
    @StateObject private var favRestaurantStore =
FavoriteRestaurantStore.shared
    @StateObject private var favMenuStore =
FavoriteMenuStore.shared
    private let recipeHistoryStore =
RecipeViewHistoryStore.shared
    private let receiptStore = ReceiptScanStore.shared
    @State private var expandedReceiptIDs: Set<UUID> =
[]
    @State private var showRecipeHistory = false

    @State private var isVisible = false

    @State private var receiptRecords:
[ReceiptScanRecord] = []

    @EnvironmentObject var bg: BackgroundManager

    private func reloadReceiptRecords() {
        receiptRecords = receiptStore.loadAll().filter {
            Calendar.current.isDate($0.scannedAt,
inSameDayAs: wrapper.date)
        }
    }

    private func closePanel() {
        withAnimation(.easeInOut(duration: 0.3)) {
            isVisible = false
        }
    }
}
```



```

DispatchQueue.main.asyncAfter(deadline: .now() +
0.3) {
    onBackPressed()
}

// MARK: - Derived Data

@State private var menusScannedToday:
[MenuScanRecord] = []
@State private var recipesViewedToday: [String] = []

var body: some View {
    GeometryReader { geo in
        ZStack {
            Color.black
                .opacity(isVisible ? 0.5 : 0)
                .ignoresSafeArea()
                .animation(.easeInOut(duration: 0.3), value:
isVisible)
                .onTapGesture {
                    closePanel()
                }

            HStack {
                Spacer()

                VStack(spacing: 16) {
                    // =====
                    // 🏠 Header
                    // =====
                    HStack {
                        Button(action: {
                            closePanel()
                        }) {
                            HStack(spacing: 4) {
                                Image(systemName: "chevron.left")
                                Text("Back")
                            }
                            .foregroundColor(.white)
                            .font(.system(size: 17, weight:
.semibold))
                        }
                        Spacer()
                    }
                }
                .padding(.horizontal)
            }
        }
    }
}

```

```

        .padding(.top, 12)

        Text(wrapper.date.toYMD())
            .font(.system(size: 28, weight: .bold))
            .foregroundColor(.white)

        // =====
        // 📁 Menus Scanned Header
        // =====
        Text("Menus Scanned")
            .font(.system(size: 17, weight:
.semibold))
            .foregroundColor(.blue.opacity(0.8))
            .frame(maxWidth: .infinity, alignment:
.leading)
            .padding(.horizontal)
            .padding(.top, 4)
        // =====
        // 📋 Menus + Receipts (Single Scroll Flow)
        // =====
        ScrollView {
            historyContent
                .padding(.horizontal)
                .padding(.vertical)
        }
        Spacer(minLength: 0)
    }
    .frame(width: geo.size.width * 0.88, height:
geo.size.height)
    .background(Color.black)
    .offset(x: isVisible ? 0 : geo.size.width)
    .animation(.easeInOut(duration: 0.3), value:
isVisible)
    .contentShape(Rectangle())
    .background(
        Color.clear
        .contentShape(Rectangle())
        .onTapGesture { closePanel() }
    )
}
}
.onAppear {
    withAnimation(.easeInOut(duration: 0.3)) {
        isVisible = true
    }
    reloadReceiptRecords()

```

```

        recomputeDerivedData()
    }
}
.sheet(isPresented: $showRecipeHistory) {
    RecipeHistoryView()
        .environmentObject(bg)
}
}

private func recomputeDerivedData() {
    // Menus scanned today
    let scans = historyStore.visibleRecords.filter {
        Calendar.isSameDay($0.scannedAt, wrapper.date)
    }
    menusScannedToday = scans

    // ✅ Recipes viewed today (REAL recipe views)
    recipesViewedToday = recipeHistoryStore
        .recipesViewed(on: wrapper.date)
        .map { $0.recipeName }
    }
}

// MARK: - History Scroll Content
private extension HistoryDayOverlay {

    @ViewBuilder
    var historyContent: some View {
        VStack(alignment: .leading, spacing: 20) {

            // ===== Menus Scanned =====
            ForEach(menusScannedToday) { scan in
                ScanCard(
                    scan: scan,
                    favRestaurantStore: favRestaurantStore,
                    favMenuStore: favMenuStore
                )
            }

            Divider()
                .background(Color.white.opacity(0.3))
                .padding(.vertical, 8)

            // ===== Receipts Scanned =====
            Text("Receipts Scanned")
                .font(.system(size: 17, weight: .semibold))

```

```

        .foregroundColor(.blue.opacity(0.8))

receiptsSection

// ===== Recipes Viewed =====
Text("Recipes Viewed")
    .font(.system(size: 17, weight: .semibold))
    .foregroundColor(.blue.opacity(0.8))

recipesViewedSection
    }
}

@ViewBuilder
var receiptsSection: some View {
    if receiptRecords.isEmpty {
        Text("No receipts scanned yet")
            .foregroundColor(.white.opacity(0.7))
    } else {
        VStack(spacing: 12) {
            ForEach(receiptRecords) { record in
                receiptRow(record)
            }
        }
    }
}

func receiptRow(_ record: ReceiptScanRecord) ->
some View {
    let isExpanded =
expandedReceiptIDs.contains(record.id)

    return HStack(alignment: .top, spacing: 0) {
        ReceiptHistoryCard(
            record: record,
            isExpanded: isExpanded
        )
        .frame(maxWidth: .infinity, alignment: .leading)

        Button {
            receiptStore.delete(id: record.id)
            reloadReceiptRecords()
        } label: {
            Image(systemName: "trash")
                .foregroundColor(.red.opacity(0.85))
                .padding(.trailing, 16)
        }
    }
}

```

```

        .padding(.top, 18)
    }
    .buttonStyle(.plain)
}
.contentShape(Rectangle())
.onTapGesture {
    withAnimation(.easeInOut(duration: 0.25)) {
        expandedReceiptIDs.toggle(record.id)
    }
}
}

@ViewBuilder
var recipesViewedSection: some View {
    if recipesViewedToday.isEmpty {
        Text("No recipes viewed")
        .foregroundColor(.white.opacity(0.6))
    } else {
        VStack(spacing: 8) {
            ForEach(recipesViewedToday.prefix(5), id:
\self) { name in
                Button {
                    showRecipeHistory = true
                } label: {
                    HStack {
                        Text(name)
                            .foregroundColor(.white)
                        Spacer()
                        Image(systemName: "chevron.right")
                            .foregroundColor(.white.opacity(0.4))
                    }
                    .padding()
                    .background(Color.white.opacity(0.08))
                    .cornerRadius(12)
                }
                .buttonStyle(.plain)
            }
        }
    }
}

// MARK: - Set<UUID> helper
private extension Set where Element == UUID {
    mutating func toggle(_ id: UUID) {
        if contains(id) {
            remove(id)
        } else {
            insert(id)
        }
    }
}

```

```

    } else {
        insert(id)
    }
}
}
import SwiftUI

struct MenuScanListSheet: View {
    let wrapper: MenuScanListWrapper

    @StateObject private var historyStore =
HistoryRecordStore.shared
    @StateObject private var favRestaurantStore =
FavoriteRestaurantStore.shared
    @StateObject private var favMenuStore =
FavoriteMenuStore.shared

    var body: some View {
        ScrollView {
            VStack(spacing: 24) {

                Text(wrapper.date.toYMD())
                    .font(.title.bold())
                    .foregroundColor(.white)
                    .padding(.top, 4)

                ForEach(
                    historyStore.visibleRecords.filter {
                        Calendar.isSameDay($0.scannedAt,
wrapper.date)
                    }
                    ) { scan in
                    ScanCard(
                        scan: scan,
                        favRestaurantStore: favRestaurantStore,
                        favMenuStore: favMenuStore
                    )
                }

                Spacer(minLength: 0)
            }
            .padding()
        }
        .scrollContentBackground(.hidden)
        .presentationDetents([.fraction(0.45)])
        .presentationDragIndicator(.hidden)
    }
}

```

```

        .presentationBackground(.clear)
    }
}

struct ScanCard: View {
    let scan: MenuScanRecord
    @ObservedObject var favRestaurantStore:
FavoriteRestaurantStore
    @ObservedObject var favMenuStore:
FavoriteMenuStore

    @State private var isExpanded = false

    private var restaurantName: String {
        scan.restaurantName ?? "Unknown Restaurant"
    }

    private var isFavorite: Bool {
        favRestaurantStore.isFavorite(restaurantName)
    }

    var body: some View {
        VStack(alignment: .leading, spacing: 12) {

            // =====
            // Header (탭 → expand)
            // =====
            HStack(alignment: .center, spacing: 8) {

                // 🍽️ Restaurant Name
                Text(restaurantName)
                    .font(.headline)
                    .foregroundColor(isFavorite ? .yellow : .white)

                // ⭐ Favorite (이름 바로 옆)
                Button {
                    favRestaurantStore.toggle(restaurantName)
                } label: {
                    Image(systemName: isFavorite ? "star.fill" :
"star")
                        .foregroundColor(.yellow)
                        .font(.system(size: 16))
                }
                .buttonStyle(.plain)

                Spacer()
            }
        }
    }
}

```

```

// 🗑 Delete (오른쪽 끝)
Button {
    HistoryRecordStore.shared.hide(scan)
} label: {
    Image(systemName: "trash")
      .foregroundColor(.red.opacity(0.85))
      .font(.system(size: 16))
    }
  .buttonStyle(.plain)
}
.contentShape(Rectangle()) // 🔥 전체 터치 영역
.onTapGesture {
    withAnimation(.easeInOut) {
        isExpanded.toggle()
    }
}

// =====
// Expanded Content
// =====
if isExpanded {
    VStack(alignment: .leading, spacing: 16) {

        if let addr = scan.address, !addr.isEmpty {
            HStack(spacing: 6) {
                Image(systemName:
"mappin.and.ellipse")
                  .foregroundColor(.yellow)

                Text(addr)
                  .foregroundColor(.white.opacity(0.9))
                  .font(.caption)
            }
        }

        Divider()
          .background(Color.white.opacity(0.4))

        ForEach(scan.items) { item in
            ScanMenuRow(
                item: item,
                restaurantName: restaurantName,
                favMenuStore: favMenuStore
            )
        }
    }
}

```



```

        }
        .padding(.top, 8)
        .transition(.opacity.combined(with: .move(edge:
.top)))
    }
}
.padding()
.frame(maxWidth: .infinity, alignment: .leading)
.background(Color.white.opacity(0.1))
.cornerRadius(14)
}
}
//
// HistoryVisibilityStore.swift
// MenuCal
//
// Created by Dae-Kyoo Kim on 12/24/25.
//

```

```

import Foundation
import Combine

```

```

@MainActor
final class HistoryRecordStore: ObservableObject {

    static let shared = HistoryRecordStore()

    @Published private(set) var visibleRecords:
[MenuScanRecord] = []

    private var hiddenIDs: Set<UUID> = []
    private let menuStore = MenuScanStore.shared
    private var cancellables: Set<AnyCancellable> = []

    private init() {
        rebuild()

        menuStore.$records
            .sink { [weak self] _ in
                self?.rebuild()
            }
            .store(in: &cancellables)
    }

    private func rebuild() {
        visibleRecords =

```

```
        menuStore.records.filter {
            !hiddenIDs.contains($0.id)
        }
    }

    // History 전용 "삭제"
    func hide(_ id: UUID) {
        hiddenIDs.insert(id)
        rebuild()
    }

    func hide(_ record: MenuScanRecord) {
        hide(record.id)
    }

    // (선택) 복구
    func unhide(_ id: UUID) {
        hiddenIDs.remove(id)
        rebuild()
    }
}
suggest clear code patch.
```

Show less ^

it's been saved properly.

reduce the space between receipt scanned items.

```
//  
// HistoryDayOverlay.swift  
// MenuCal  
//  
// Created by Dae-Kyoo Kim on 12/28/25.  
//  
import SwiftUI  
  
// MARK: - History Category Enum  
private enum HistoryCategory {
```

```

        case menus
        case receipts
    }

    struct HistoryDayOverlay: View {
        let wrapper: MenuScanListWrapper
        let onBack: () -> Void

        @Namespace private var categoryNamespace

        @StateObject private var historyStore =
        MenuVisibilityStore.shared
        @StateObject private var favRestaurantStore =
        FavoriteRestaurantStore.shared
        @StateObject private var favMenuStore =
        FavoriteMenuStore.shared
        @State private var expandedReceiptIDs: Set<UUID> =
        []
        @State private var showRecipeHistory = false

        @State private var isVisible = false

        @StateObject private var receiptStore =
        ReceiptScanStore.shared
        @StateObject private var receiptVisibility =
        ReceiptVisibilityStore.shared

        @EnvironmentObject var bg: BackgroundManager

        private var filteredReceiptRecords:
        [ReceiptScanRecord] {
            receiptStore.records
                .filter { Calendar.current.isDate($0.scannedAt,
inSameDayAs: wrapper.date) }
                .filter { !receiptVisibility.isHidden($0.id) }
        }

        private func closePanel() {
            withAnimation(.easeInOut(duration: 0.3)) {
                isVisible = false
            }
            DispatchQueue.main.asyncAfter(deadline: .now() +
0.3) {
                onBack()
            }
        }
    }

```

```
// MARK: - Derived Data
```

```
@State private var recipesViewedToday: [String] = []
```

```
var body: some View {
    GeometryReader { geo in
        ZStack {
            Color.black
                .opacity(isVisible ? 0.5 : 0)
                .ignoresSafeArea()
                .animation(.easeInOut(duration: 0.3), value:
isVisible)
                .onTapGesture {
                    closePanel()
                }

            HStack {
                Spacer()

                VStack(spacing: 16) {
                    // =====
                    // 🏠 Header
                    // =====
                    HStack {
                        Button(action: {
                            closePanel()
                        }) {
                            HStack(spacing: 4) {
                                Image(systemName: "chevron.left")
                                Text("Back")
                            }
                            .foregroundColor(.white)
                            .font(.system(size: 17, weight:
.semibold))
                        }
                        Spacer()
                    }
                    .padding(.horizontal)
                    .padding(.top, 12)

                    Text(wrapper.date.toYMD())
                        .font(.system(size: 28, weight: .bold))
                        .foregroundColor(.white)

                    // =====
                }
            }
        }
    }
}
```

```

// 📄 Menus Scanned Header
// =====
Text("Menus Scanned")
    .font(.system(size: 17, weight:
.semibold))
    .foregroundColor(.blue.opacity(0.8))
    .frame(maxWidth: .infinity, alignment:
.leading)
    .padding(.horizontal)
    .padding(.top, 4)
// =====
// 📖 Menus + Receipts (Single Scroll Flow)
// =====
ScrollView {
    historyContent
        .padding(.horizontal)
        .padding(.vertical)
    }
    Spacer(minLength: 0)
}
.frame(width: geo.size.width * 0.88, height:
geo.size.height)
    .background(Color.black)
    .offset(x: isVisible ? 0 : geo.size.width)
    .animation(.easeInOut(duration: 0.3), value:
isVisible)
    .contentShape(Rectangle())
    .background(
        Color.clear
        .contentShape(Rectangle())
        .onTapGesture { closePanel() }
    )
}
}
.onAppear {
    withAnimation(.easeInOut(duration: 0.3)) {
        isVisible = true
    }
    receiptStore.refresh() // 🔥 이 한 줄이 핵심
    recomputeDerivedData()
}
}
.sheet(isPresented: $showRecipeHistory) {
    RecipeHistoryView()
        .environmentObject(bg)
}

```

```

    }

    private func recomputeDerivedData() {
        recipesViewedToday =
            RecipeViewStore.shared
                .records(on: wrapper.date)
                .filter {
!RecipeVisibilityStore.shared.isHidden($0.recipeName) }
                .map { $0.recipeName }
        }
    }
}

```

// MARK: - History Scroll Content

```

private extension HistoryDayOverlay {

    @ViewBuilder
    var historyContent: some View {
        VStack(alignment: .leading, spacing: 20) {

            // ===== Menus Scanned =====
            ForEach(
                historyStore.visibleRecords.filter {
                    Calendar.isSameDay($0.scannedAt,
wrapper.date)
                }
            ) { scan in
                ScanCard(
                    scan: scan,
                    favRestaurantStore: favRestaurantStore,
                    favMenuStore: favMenuStore
                )
            }

            Divider()
                .background(Color.white.opacity(0.3))
                .padding(.vertical, 8)

            // ===== Receipts Scanned =====
            Text("Receipts Scanned")
                .font(.system(size: 17, weight: .semibold))
                .foregroundColor(.blue.opacity(0.8))

            receiptsSection

            // ===== Recipes Viewed =====
            Text("Recipes Viewed")

```

```

        .font(.system(size: 17, weight: .semibold))
        .foregroundColor(.blue.opacity(0.8))

        recipesViewedSection
    }
}

@ViewBuilder
var receiptsSection: some View {
    if filteredReceiptRecords.isEmpty {
        Text("No receipts scanned yet")
        .foregroundColor(.white.opacity(0.7))
    } else {
        VStack(spacing: 12) {
            ForEach(filteredReceiptRecords) { record in
                receiptRow(record)
            }
        }
    }
}

func receiptRow(_ record: ReceiptScanRecord) ->
some View {
    let isExpanded =
expandedReceiptIDs.contains(record.id)

    return HStack(alignment: .top, spacing: 0) {
        ReceiptHistoryCard(
            record: record,
            isExpanded: isExpanded
        )
        .frame(maxWidth: .infinity, alignment: .leading)

        Button {
            receiptVisibility.hide(record.id)
        } label: {
            Image(systemName: "trash")
                .foregroundColor(.red.opacity(0.85))
                .padding(.trailing, 16)
                .padding(.top, 18)
        }
        .buttonStyle(.plain)
    }
    .contentShape(Rectangle())
    .onTapGesture {
        withAnimation(.easeInOut(duration: 0.25)) {

```



```

        expandedReceiptIDs.toggle(record.id)
    }
}
}

@ViewBuilder
var recipesViewedSection: some View {
    if recipesViewedToday.isEmpty {
        Text("No recipes viewed")
            .foregroundColor(.white.opacity(0.6))
    } else {
        VStack(spacing: 8) {
            ForEach(recipesViewedToday.prefix(5), id:
\self) { name in
                HStack {
                    Text(name).foregroundColor(.white)
                    Spacer()
                    Button {
                        RecipeVisibilityStore.shared.hide(name)
                        recomputeDerivedData()
                    } label: {
                        Image(systemName: "trash")
                            .foregroundColor(.red.opacity(0.8))
                    }
                    .buttonStyle(.plain)
                }
                .padding()
                .background(Color.white.opacity(0.08))
                .cornerRadius(12)
            }
        }
    }
}

// MARK: - Set<UUID> helper
private extension Set where Element == UUID {
    mutating func toggle(_ id: UUID) {
        if contains(id) {
            remove(id)
        } else {
            insert(id)
        }
    }
}

```

Show less ^

remove the trash for individual menu card in history.

recipe viewed item을 누르면 detail recipe가 보이게 해라.

```
//  
// HistoryDayOverlay.swift  
// MenuCal  
//  
// Created by Dae-Kyoo Kim on 12/28/25.  
//  
import SwiftUI  
  
// MARK: - History Category Enum  
private enum HistoryCategory {  
    case menus  
    case receipts
```

```
}
```

```
struct HistoryDayOverlay: View {  
    let wrapper: MenuScanListWrapper  
    let onBack: () -> Void  
  
    @Namespace private var categoryNamespace  
  
    @StateObject private var historyStore =  
MenuVisibilityStore.shared  
    @StateObject private var favRestaurantStore =  
FavoriteRestaurantStore.shared  
    @StateObject private var favMenuStore =  
FavoriteMenuStore.shared  
    @State private var expandedReceiptIDs: Set<UUID> =  
[]  
    @State private var expandedMenuIDs: Set<UUID> = []  
    // 🔥 Menus  
    @State private var expandedRecipeNames:  
Set<String> = [] // 🔥 Recipes  
  
    @State private var showRecipeHistory = false  
  
    @State private var isVisible = false  
  
    @StateObject private var receiptStore =  
ReceiptScanStore.shared  
    @StateObject private var receiptVisibility =  
ReceiptVisibilityStore.shared  
  
    @EnvironmentObject var bg: BackgroundManager  
  
    private var filteredReceiptRecords:  
[ReceiptScanRecord] {  
        receiptStore.records  
            .filter { Calendar.current.isDate($0.scannedAt,  
inSameDayAs: wrapper.date) }  
            .filter { !receiptVisibility.isHidden($0.id) }  
    }  
  
    private func closePanel() {  
        withAnimation(.easeInOut(duration: 0.3)) {  
            isVisible = false  
        }  
        DispatchQueue.main.asyncAfter(deadline: .now() +  
0.3) {
```



```

        onBackPressed()
    }
}

// MARK: - Derived Data

@State private var recipesViewedToday: [String] = []

var body: some View {
    GeometryReader { geo in
        ZStack {
            Color.black
                .opacity(isVisible ? 0.5 : 0)
                .ignoresSafeArea()
                .animation(.easeInOut(duration: 0.3), value:
isVisible)
                .onTapGesture {
                    closePanel()
                }

            HStack {
                Spacer()

                VStack(spacing: 16) {
                    // =====
                    // 🏠 Header
                    // =====
                    HStack {
                        Button(action: {
                            closePanel()
                        }) {
                            HStack(spacing: 4) {
                                Image(systemName: "chevron.left")
                                Text("Back")
                            }
                            .foregroundColor(.white)
                            .font(.system(size: 17, weight:
.semibold))
                        }
                        Spacer()
                    }
                    .padding(.horizontal)
                    .padding(.top, 12)

                    Text(wrapper.date.toYMD())
                        .font(.system(size: 28, weight: .bold))
                }
            }
        }
    }
}

```

```

        .foregroundColor(.white)

        // =====
        // 📄 Menus Scanned Header
        // =====
        Text("Menus Scanned")
            .font(.system(size: 17, weight:
.semibold))
            .foregroundColor(.blue.opacity(0.8))
            .frame(maxWidth: .infinity, alignment:
.leading)
            .padding(.horizontal)
            .padding(.top, 4)
        // =====
        // 📖 Menus + Receipts (Single Scroll Flow)
        // =====
        ScrollView {
            historyContent
                .padding(.horizontal)
                .padding(.vertical)
        }
        Spacer(minLength: 0)
    }
    .frame(width: geo.size.width * 0.88, height:
geo.size.height)
    .background(Color.black)
    .offset(x: isVisible ? 0 : geo.size.width)
    .animation(.easeInOut(duration: 0.3), value:
isVisible)
    .contentShape(Rectangle())
    .background(
        Color.clear
        .contentShape(Rectangle())
        .onTapGesture { closePanel() }
    )
}
}
.onAppear {
    withAnimation(.easeInOut(duration: 0.3)) {
        isVisible = true
    }
    receiptStore.refresh() // 🔥 이 한 줄이 핵심
    recomputeDerivedData()
}
}
.sheet(isPresented: $showRecipeHistory) {

```

```

RecipeHistoryView()
    .environmentObject(bg)
}
}

private func recomputeDerivedData() {
    recipesViewedToday =
        RecipeViewStore.shared
            .records(on: wrapper.date)
            .filter {
!RecipeVisibilityStore.shared.isHidden($0.recipeName) }
            .map { $0.recipeName }
    }
}

// MARK: - History Scroll Content
private extension HistoryDayOverlay {

    @ViewBuilder
    var historyContent: some View {
        VStack(alignment: .leading, spacing: 20) {

            // ===== Menus Scanned =====
            ForEach(
                historyStore.visibleRecords.filter {
                    Calendar.isSameDay($0.scannedAt,
wrapper.date)
                }
            ) { scan in
                ScanCard(
                    scan: scan,
                    favRestaurantStore: favRestaurantStore,
                    favMenuStore: favMenuStore,
                    expandedMenuIDs: $expandedMenuIDs
                )
            }

            Divider()
                .background(Color.white.opacity(0.3))
                .padding(.vertical, 8)

            // ===== Receipts Scanned =====
            Text("Receipts Scanned")
                .font(.system(size: 17, weight: .semibold))
                .foregroundColor(.blue.opacity(0.8))
        }
    }
}

```

```

receiptsSection

// ===== Recipes Viewed =====
Text("Recipes Viewed")
    .font(.system(size: 17, weight: .semibold))
    .foregroundColor(.blue.opacity(0.8))

recipesViewedSection
}
}

@ViewBuilder
var receiptsSection: some View {
    if filteredReceiptRecords.isEmpty {
        Text("No receipts scanned yet")
            .foregroundColor(.white.opacity(0.7))
    } else {
        VStack(spacing: 12) {
            ForEach(filteredReceiptRecords) { record in
                receiptRow(record)
            }
        }
    }
}

func receiptRow(_ record: ReceiptScanRecord) ->
some View {
    let isExpanded =
expandedReceiptIDs.contains(record.id)
    let isExpandable = record.items.count > 1

    return HStack(alignment: .top, spacing: 0) {

        ReceiptHistoryCard(
            record: record,
            isExpanded: isExpanded
        )
        .frame(maxWidth: .infinity, alignment: .leading)

        if isExpandable {
            ExpandChevron(isExpanded: isExpanded)
        }

        Button {
            receiptVisibility.hide(record.id)
        } label: {

```

```

        Image(systemName: "trash")
        .foregroundColor(.white.opacity(0.3))
        .padding(.top, 18)
        .padding(.trailing, 16)
    }
    .buttonStyle(.plain)
}
.contentShape(Rectangle())
.onTapGesture {
    guard isExpandable else { return }
    withAnimation(.easeInOut(duration: 0.25)) {
        expandedReceiptIDs.toggle(record.id)
    }
}
}
}

```

```

@ViewBuilder
var recipesViewedSection: some View {
    if recipesViewedToday.isEmpty {
        Text("No recipes viewed")
        .foregroundColor(.white.opacity(0.6))
    } else {
        VStack(spacing: 8) {
            ForEach(recipesViewedToday.prefix(5), id:
\self) { name in

```

let isExpandable = true // 🔥 Recipe는 detail
항상 존재

let isExpanded =
expandedRecipeNames.contains(name)

```

HStack {
    Text(name).foregroundColor(.white)
    Spacer()
    // ✅ 🔥 chevron은 trash 왼쪽
    if isExpandable {
        ExpandChevron(isExpanded:
isExpanded)
    }
}

```

```

Button {
    RecipeVisibilityStore.shared.hide(name)
    recomputeDerivedData()
} label: {
    Image(systemName: "trash")
    .foregroundColor(.white.opacity(0.3))
}

```


}

}

Show less ^

문제가 있다. history에서 메뉴를 보면 방금 본 메뉴가 계속 history에 잡힌다.

```
import Foundation
import SwiftUI
import UIKit
```

```
// =====
// MARK: - Similar Recipe Model (✅ name-only)
// =====
```

```
struct SimilarRecipe: Identifiable {
    let id = UUID()
    let name: String
    let score: Float
```

```

}

// =====
// MARK: - Recipe Detail View
// =====

struct RecipeDetailView: View {

    let detail: RecipeDetail
    let queryName: String      // ✅ name 기반
    let caloriesByIngredient: [String: (kcal: Int, unit: String)]
    let totalCalories: Int

    @State private var hasRecordedView = false

    private var dishName: String { detail.name }

    @State private var similarRecipes: [SimilarRecipe] = []
    @StateObject private var favoriteStore =
FavoriteRecipeStore.shared
    private let haptic = UIImpactFeedbackGenerator(style:
.medium)
    @State private var bestDetail: RecipeDetail?

    @EnvironmentObject var bg: BackgroundManager

    // -----
    -----
    // IngredientItem (유지)
    // -----
    -----

    struct IngredientItem: Identifiable {
        let text: String
        var id: String { text }
    }

    private func capitalizeFirstLetter(of string: String) ->
String {
        let trimmed = string.trimmingCharacters(in:
.whitespacesAndNewlines)
        guard let first = trimmed.first else { return "" }
        return String(first).uppercased() +
String(trimmed.dropFirst())
    }

    // =====

```

```

// MARK: - View Body
// =====

var body: some View {
    NavigationStack {
        ScrollView {
            content
        }
        .background(
            Color.black.ignoresSafeArea()
        )
        .navigationBarTitleDisplayMode(.inline)
        .toolbar {
            ToolbarItem(placement: .topBarTrailing) {
                let canonicalName =
NameNormalizer.canonical(dishName, kind: .recipe)

                Button {
                    haptic.prepare()
                    haptic.impactOccurred()
                    favoriteStore.toggle(canonicalName)
                } label: {
                    Image(systemName:
                        favoriteStore.isFavorite(canonicalName)
                        ? "star.fill"
                        : "star"
                    )
                    .foregroundColor(.yellow)
                }
            }
        }
        .onAppear {
            Task {
                // ✅ 레시피 viewed 기록 (단 1회)
                if !hasRecordedView {
                    hasRecordedView = true
                    RecipeViewStore.shared.recordView(
                        recipeName: detail.name,
                        recipeID: nil // ✅ name-based recipe →
nil이 정상
                    )
                }

                self.bestDetail = detail
                await loadSimilarRecipes(fromName:
queryName)
            }
        }
    }
}

```



```

    }
  }
}

private var content: some View {
  VStack(alignment: .leading, spacing: 30) {

    // HEADER
    HStack(alignment: .firstTextBaseline, spacing: 12)
    {

      Text(autoEmoji(for: dishName))
        .font(.system(size: 40))

      Text(bestDetail?.name ?? dishName)
        .font(.system(size: 26, weight: .bold, design:
.rounded))
        .foregroundColor(.white)
        .lineLimit(3)
        .minimumScaleFactor(0.8)

      Spacer()

      if !caloriesByIngredient.isEmpty {
        HStack(spacing: 4) {

          Text("\(totalCalories)")
            .foregroundColor(.yellow)
            .font(.headline)
            .fontWeight(.bold)

          Text("kcal")
            .foregroundColor(.yellow.opacity(0.8))
            .font(.caption)

          Text("/ dish")
            .foregroundColor(.white.opacity(0.5))
            .font(.caption)
        }
      }
    }
    .padding(.top, 20)

    // ✅ 여기 추가 (정확히 여기)
    if !caloriesByIngredient.isEmpty {

```

```

        CalorieProgressBar(calories:
Double(totalCalories))
    }

    // ✅ INGREDIENTS (이게 지금 빠져 있음)
    if !detail.ingredients.isEmpty {
        VStack(alignment: .leading, spacing: 8) {
            Text("Ingredients")
                .font(.title2)
                .fontWeight(.semibold)

            ForEach(detail.ingredients.indices, id: \.self) {
i in
                let ing = detail.ingredients[i]

                HStack {
                    Text("• \((ing)")
                        .foregroundColor(.white)

                    Spacer()

                    // 🔥 칼로리 / 단위
                    if let cal = caloriesByIngredient[ing] {
                        HStack(spacing: 4) {
                            Text("\((cal.kcal)")
                                .foregroundColor(.yellow)
                                .font(.subheadline)
                                .fontWeight(.semibold)

                            Text("kcal")

                            .foregroundColor(.yellow.opacity(0.8))
                                .font(.caption)

                            Text("/ \((cal.unit)")

                            .foregroundColor(.white.opacity(0.5))
                                .font(.caption)
                        }
                    }
                }
            }
        }
    }
}

```

```

// ✅ STEPS
if !detail.steps.isEmpty {
    VStack(alignment: .leading, spacing: 8) {
        Text("Steps")
            .font(.title2)
            .fontWeight(.semibold)

        ForEach(detail.steps.indices, id: \.self) { i in
            Text("\((i + 1). \((detail.steps[i])")
        }
    }
}

// =====
// SIMILAR RECIPES (✅ name 기반)
// =====
if !similarRecipes.isEmpty {
    VStack(alignment: .leading, spacing: 8) {
        Text("Similar Recipes")
            .font(.title2)
            .fontWeight(.semibold)
            .foregroundColor(.primary) // ✅
    }
}

Ingredients와 동일

ForEach(similarRecipes) { item in
    NavigationLink {
        // ✅ name-only 이동
        RecipeDetailLoadingView(name:
item.name)
    } label: {
        HStack {
            Text("• \((item.name)")
                .foregroundColor(.white)
                .lineLimit(1)
            Spacer()
            Image(systemName: "chevron.right")
        }
    }
    .buttonStyle(.plain)
}

Spacer()
}
.padding(.horizontal, 35)

```

```

}

struct CalorieProgressBar: View {

    let calories: Double

    private var progress: Double {
        min(calories / 1000.0, 1.0)
    }

    private var barColor: Color {
        switch calories {
        case ..<600:
            return .green
        case 600..

```

```

    }
    .font(.caption2)
    .foregroundColor(.white.opacity(0.6))
  }
  .padding(.top, 4)
}
}

// =====
// MARK: - Similar Recipes (✅ name 기반)
// =====

@MainActor
private func loadSimilarRecipes(fromName name:
String) async {
    let results = await
RecipeSearchService.shared.searchSimilarRecipes(
    to: name,
    topK: 10
    )

    var seen = Set<String>()
    var uniques: [SimilarRecipe] = []

    for r in results {
        let key = NameNormalizer.canonical(r.name, kind:
.recipe)

        if seen.contains(key) { continue }
        seen.insert(key)

        uniques.append(
            SimilarRecipe(name: r.name, score: r.score)
        )

        if uniques.count == 5 { break }
    }

    self.similarRecipes = uniques
}
}

```

view record를 detail view에서 생긴현상이다. view record를
아래 코드에서 해야 하는거 아닌가?

```

//
// RecipeDetailLoadingView.swift
// MenuCal
//
// Created by Dae-Kyoo Kim on 12/8/25.
//

import Foundation
import SwiftUI

struct RecipeDetailLoadingView: View {

    let name: String
    @State private var loadedDetail: RecipeDetail? = nil
    @State private var calorieMap: [String: (kcal: Int, unit: String)] = [:]
    @State private var totalCalories: Int = 0

    var body: some View {

        if let detail = loadedDetail {
            RecipeDetailView(
                detail: detail,
                queryName: name,
                caloriesByIngredient: calorieMap,
                totalCalories: totalCalories // ✅ 전달
            )
        } else {
            VStack(spacing: 12) {
                Text("Loading...")
                    .font(.headline)
                    .foregroundColor(.gray)

                ProgressView()
                    .controlSize(.large)
            }
            .navigationTitle(name)
            .onAppear {
                Task {
                    guard let detail = await
RecipeDetailDB.shared.detail(named: name) else {
                        await MainActor.run {
                            self.loadedDetail = nil
                        }
                    }
                    return
                }
            }
        }
    }
}

```

```
let parser = RecipeParser()
let estimate = parser.buildEstimate(from:
detail)

// ingredient별 kcal map 생성 (UI용)
// ingredient별 kcal map 생성 (UI용)
// ✅ key는 detail.ingredients(원문) 그대로 → UI
lookup과 1:1 일치
var calorieMap: [String: (kcal: Int, unit:
String)] = [:]

for recipeIng in detail.ingredients {
    let recipeLower = recipeIng.lowercased()

    if let matched =
estimate.ingredients.first(where: { line in
recipeLower.contains(line.key.lowercased()) ||
recipeLower.contains(line.display.lowercased())
}) {
        calorieMap[recipeIng] = (
            Int(matched.calPerUnit),
            matched.unit
        )
    }
}

await MainActor.run {
    self.loadedDetail = detail
    self.calorieMap = calorieMap
    self.totalCalories =
Int(estimate.totalCalories)

    // ✅ 캐시는 "결과"만 저장
    RecipeCalorieCache.shared.setCalories(
        Int(estimate.totalCalories),
        for: detail.name
    )
}
}
}
}
}
```

Show less ^

chevron의 방향을 오른쪽으로 해라. 그리고 메뉴를 본 시간을 보여
줘라.

trash랄 chevron이라 곁친가. chevron을 trash왼쪽에 놓아라.
시스템 chevron이라 옮길수 없으면 trash를 chevron왼쪽으로
옮겨라

```
//  
// HistoryDayOverlay.swift  
// MenuCal  
//  
// Created by Dae-Kyoo Kim on 12/28/25.  
//  
import SwiftUI  
  
// MARK: - History Category Enum  
private enum HistoryCategory {  
    case menus  
    case receipts  
}  
  
struct HistoryDayOverlay: View {  
    let wrapper: MenuScanListWrapper  
    let onBack: () -> Void  
  
    @Namespace private var categoryNamespace  
  
    @StateObject private var historyStore =  
MenuVisibilityStore.shared  
    @StateObject private var favRestaurantStore =  
FavoriteRestaurantStore.shared  
    @StateObject private var favMenuStore =  
FavoriteMenuStore.shared  
    @State private var expandedReceiptIDs: Set<UUID> =  
[]  
    @State private var expandedMenuIDs: Set<UUID> = []  
    // 🔥 Menus
```

```
@State private var showRecipeHistory = false
```

```
@State private var isVisible = false
```

```
@StateObject private var receiptStore =  
ReceiptScanStore.shared
```

```
@StateObject private var receiptVisibility =  
ReceiptVisibilityStore.shared
```

```
@EnvironmentObject var bg: BackgroundManager
```

```
private var filteredReceiptRecords:  
[ReceiptScanRecord] {  
    receiptStore.records  
        .filter { Calendar.current.isDate($0.scannedAt,  
inSameDayAs: wrapper.date) }  
        .filter { !receiptVisibility.isHidden($0.id) }  
}
```

```
private func closePanel() {  
    withAnimation(.easeInOut(duration: 0.3)) {  
        isVisible = false  
    }  
    DispatchQueue.main.asyncAfter(deadline: .now() +  
0.3) {  
        onBack()  
    }  
}
```

```
// MARK: - Derived Data
```

```
@State private var recipesViewedToday:  
[RecipeViewRecord] = []
```

```
var body: some View {  
    GeometryReader { geo in  
        ZStack {  
            Color.black  
                .opacity(isVisible ? 0.5 : 0)  
                .ignoresSafeArea()  
                .animation(.easeInOut(duration: 0.3), value:  
isVisible)  
                .onTapGesture {  
                    closePanel()  
                }  
        }  
    }  
}
```

```

HStack {
  Spacer()

  VStack(spacing: 16) {
    // =====
    // 🏠 Header
    // =====
    HStack {
      Button(action: {
        closePanel()
      }) {
        HStack(spacing: 4) {
          Image(systemName: "chevron.left")
          Text("Back")
        }
        .foregroundColor(.white)
        .font(.system(size: 17, weight:
.semibold))
      }
      Spacer()
    }
    .padding(.horizontal)
    .padding(.top, 12)

    Text(wrapper.date.toYMD())
      .font(.system(size: 28, weight: .bold))
      .foregroundColor(.white)

    // =====
    // 📁 Menus Scanned Header
    // =====
    Text("Menus Scanned")
      .font(.system(size: 17, weight:
.semibold))
      .foregroundColor(.blue.opacity(0.8))
      .frame(maxWidth: .infinity, alignment:
.leading)
      .padding(.horizontal)
      .padding(.top, 4)
    // =====
    // 📋 Menus + Receipts (Single Scroll Flow)
    // =====
    ScrollView {
      historyContent
        .padding(.horizontal)

```

```

        .padding(.vertical)
    }
    Spacer(minLength: 0)
}
.frame(width: geo.size.width * 0.88, height:
geo.size.height)
    .background(Color.black)
    .offset(x: isVisible ? 0 : geo.size.width)
    .animation(.easeInOut(duration: 0.3), value:
isVisible)
    .contentShape(Rectangle())
    .background(
        Color.clear
        .contentShape(Rectangle())
        .onTapGesture { closePanel() }
    )
}
}
.onAppear {
    withAnimation(.easeInOut(duration: 0.3)) {
        isVisible = true
    }
    receiptStore.refresh() // 🔥 이 한 줄이 핵심
    recomputeDerivedData()
}
}
.sheet(isPresented: $showRecipeHistory) {
    RecipeHistoryView()
    .environmentObject(bg)
}
}

private func recomputeDerivedData() {
    recipesViewedToday =
        RecipeViewStore.shared
            .records(on: wrapper.date)
            .filter {
!RecipeVisibilityStore.shared.isHidden($0.recipeName) }
            .sorted { $0.viewedAt > $1.viewedAt } // 🔥 최신
순
}

private static let timeFormatter: DateFormatter = {
    let f = DateFormatter()
    f.locale = Locale.current
    f.timeStyle = .short // 3:42 PM / 15:42
}

```



```

        f.dateStyle = .none
        return f
    }()
}

// MARK: - History Scroll Content
private extension HistoryDayOverlay {

    @ViewBuilder
    var historyContent: some View {
        VStack(alignment: .leading, spacing: 20) {

            // ===== Menus Scanned =====
            ForEach(
                historyStore.visibleRecords.filter {
                    Calendar.isSameDay($0.scannedAt,
wrapper.date)
                }
            ) { scan in
                ScanCard(
                    scan: scan,
                    favRestaurantStore: favRestaurantStore,
                    favMenuStore: favMenuStore,
                    expandedMenuIDs: $expandedMenuIDs
                )
            }

            Divider()
                .background(Color.white.opacity(0.3))
                .padding(.vertical, 8)

            // ===== Receipts Scanned =====
            Text("Receipts Scanned")
                .font(.system(size: 17, weight: .semibold))
                .foregroundColor(.blue.opacity(0.8))

            receiptsSection

            // ===== Recipes Viewed =====
            Text("Recipes Viewed")
                .font(.system(size: 17, weight: .semibold))
                .foregroundColor(.blue.opacity(0.8))

            recipesViewedSection
        }
    }
}

```

```
}
```

```
@ViewBuilder
```

```
var receiptsSection: some View {  
    if filteredReceiptRecords.isEmpty {  
        Text("No receipts scanned yet")  
        .foregroundColor(.white.opacity(0.7))  
    } else {  
        VStack(spacing: 12) {  
            ForEach(filteredReceiptRecords) { record in  
                receiptRow(record)  
            }  
        }  
    }  
}
```

```
func receiptRow(_ record: ReceiptScanRecord) ->  
some View {  
    let isExpanded =  
expandedReceiptIDs.contains(record.id)  
    let isExpandable = record.items.count > 1  
  
    return HStack(alignment: .top, spacing: 0) {  
  
        ReceiptHistoryCard(  
            record: record,  
            isExpanded: isExpanded  
        )  
        .frame(maxWidth: .infinity, alignment: .leading)  
  
        if isExpandable {  
            ExpandChevron(isExpanded: isExpanded)  
        }  
  
        Button {  
            receiptVisibility.hide(record.id)  
        } label: {  
            Image(systemName: "trash")  
                .foregroundColor(.white.opacity(0.3))  
                .padding(.top, 18)  
                .padding(.trailing, 16)  
        }  
        .buttonStyle(.plain)  
    }  
    .contentShape(Rectangle())  
    .onTapGesture {
```

```

guard isExpandable else { return }
withAnimation(.easeInOut(duration: 0.25)) {
    expandedReceiptIDs.toggle(record.id)
}
}
}
}

```

@ViewBuilder

```

var recipesViewedSection: some View {
    if recipesViewedToday.isEmpty {
        Text("No recipes viewed")
        .foregroundColor(.white.opacity(0.6))
    } else {
        VStack(spacing: 8) {
            ForEach(recipesViewedToday.prefix(5)) { record

```

in

```

                let normalizedName =
                    record.recipeName
                        .trimmingCharacters(in:
                            .whitespacesAndNewlines)

```

```

                .precomposedStringWithCanonicalMapping

```

```

                ZStack {
                    // ✅ Row 전체를 NavigationLink로
                    NavigationLink {
                        RecipeDetailLoadingView(name:
normalizedName)
                    } label: {
                        HStack(spacing: 12) {

                            VStack(alignment: .leading, spacing:
4) {

                                Text(record.recipeName)
                                    .foregroundColor(.white)
                                    .multilineTextAlignment(.leading)

                                Text(
                                    Self.timeFormatter.string(
                                        from: record.viewedAt
                                    )
                                )
                                    .font(.caption)

```

```

                .foregroundColor(.white.opacity(0.55))

```

```

    }

    Spacer()

    // 📺 Chevron RIGHT
    Image(systemName: "chevron.right")

    .foregroundColor(.white.opacity(0.35))
    }
    .padding()
    .background(Color.white.opacity(0.08))
    .cornerRadius(12)
    .contentShape(Rectangle()) // 🔥 터치 안
정화 핵심
    }
    .buttonStyle(.plain)

    // 🗑️ Trash 버튼은 위에 덮어쓰기
    HStack {
        Spacer()
        Button {

RecipeVisibilityStore.shared.hide(record.recipeName)
            recomputeDerivedData()
        } label: {
            Image(systemName: "trash")

    .foregroundColor(.white.opacity(0.3))
        .padding()
    }
    .buttonStyle(.plain)
    }
    }
    }
    }
    }
    }
    }

struct ExpandChevron: View {
    let isExpanded: Bool

    var body: some View {
        Image(systemName: "chevron.down")
        .foregroundColor(.white.opacity(0.35))
        .rotationEffect(.degrees(isExpanded ? 180 : 0))

```

```
        .animation(.easeInOut(duration: 0.2), value:
isExpanded)
        .padding(.top, 22)
        .padding(.trailing, 8)
    }
}
```

```
// MARK: - Set<UUID> helper
extension Set where Element == UUID {
    mutating func toggle(_ id: UUID) {
        if contains(id) {
            remove(id)
        } else {
            insert(id)
        }
    }
}
```

Show less ^

첨부화면에서 each category를 expand/collaps하게 만들어라.
아래로

```
//  
// HistoryDayOverlay.swift  
// MenuCal  
//  
// Created by Dae-Kyoo Kim on 12/28/25.  
//  
import SwiftUI  
  
// MARK: - History Category Enum  
private enum HistoryCategory {  
    case menus  
    case receipts  
}  
  
struct HistoryDayOverlay: View {  
    let wrapper: MenuScanListWrapper  
    let onBack: () -> Void  
  
    @Namespace private var categoryNamespace  
  
    @StateObject private var historyStore =  
MenuVisibilityStore.shared  
    @StateObject private var favRestaurantStore =  
FavoriteRestaurantStore.shared  
    @StateObject private var favMenuStore =  
FavoriteMenuStore.shared  
    @State private var expandedReceiptIDs: Set<UUID> =  
[]  
    @State private var expandedMenuIDs: Set<UUID> = []  
    // 🔥 Menus  
  
    @State private var showRecipeHistory = false  
  
    @State private var isVisible = false  
  
    @StateObject private var receiptStore =  
ReceiptScanStore.shared  
    @StateObject private var receiptVisibility =  
ReceiptVisibilityStore.shared  
  
    @EnvironmentObject var bg: BackgroundManager  
  
    private var filteredReceiptRecords:
```

```

[ReceiptScanRecord] {
    receiptStore.records
        .filter { Calendar.current.isDate($0.scannedAt,
inSameDayAs: wrapper.date) }
        .filter { !receiptVisibility.isHidden($0.id) }
    }

    private func closePanel() {
        withAnimation(.easeInOut(duration: 0.3)) {
            isVisible = false
        }
        DispatchQueue.main.asyncAfter(deadline: .now() +
0.3) {
            onBack()
        }
    }
}

```

// MARK: - Derived Data

```

@State private var recipesViewedToday:
[RecipeViewRecord] = []

var body: some View {
    GeometryReader { geo in
        ZStack {
            Color.black
                .opacity(isVisible ? 0.5 : 0)
                .ignoresSafeArea()
                .animation(.easeInOut(duration: 0.3), value:
isVisible)
                .onTapGesture {
                    closePanel()
                }

            HStack {
                Spacer()

                VStack(spacing: 16) {
                    // =====
                    // 🏠 Header
                    // =====
                    HStack {
                        Button(action: {
                            closePanel()
                        }) {
                            HStack(spacing: 4) {

```

```

        Image(systemName: "chevron.left")
        Text("Back")
    }
    .foregroundColor(.white)
    .font(.system(size: 17, weight:
.semibold))
    }
    Spacer()
}
.padding(.horizontal)
.padding(.top, 12)

Text(wrapper.date.toYMD())
    .font(.system(size: 28, weight: .bold))
    .foregroundColor(.white)

// =====
// 📁 Menus Scanned Header
// =====
Text("Menus Scanned")
    .font(.system(size: 17, weight:
.semibold))
    .foregroundColor(.blue.opacity(0.8))
    .frame(maxWidth: .infinity, alignment:
.leading)
    .padding(.horizontal)
    .padding(.top, 4)
// =====
// 📅 Menus + Receipts (Single Scroll Flow)
// =====
ScrollView {
    historyContent
        .padding(.horizontal)
        .padding(.vertical)
    }
    Spacer(minLength: 0)
}
.frame(width: geo.size.width * 0.88, height:
geo.size.height)
    .background(Color.black)
    .offset(x: isVisible ? 0 : geo.size.width)
    .animation(.easeInOut(duration: 0.3), value:
isVisible)
    .contentShape(Rectangle())
    .background(
        Color.clear

```

```

        .contentShape(Rectangle())
        .onTapGesture { closePanel() }
    )
}
}
.onAppear {
    withAnimation(.easeInOut(duration: 0.3)) {
        isVisible = true
    }
    receiptStore.refresh() // 🔥 이 한 줄이 핵심
    recomputeDerivedData()
}
}
.sheet(isPresented: $showRecipeHistory) {
    RecipeHistoryView()
        .environmentObject(bg)
}
}

private fun recomputeDerivedData() {
    recipesViewedToday =
        RecipeViewStore.shared
            .records(on: wrapper.date)
            .filter { !RecipeVisibilityStore.shared.isHidden(id:
$0.id) }
            .sorted { $0.viewedAt > $1.viewedAt } // 🔥 최신
순
}

private static let timeFormatter: DateFormatter = {
    let f = DateFormatter()
    f.locale = Locale.current
    f.timeStyle = .short // 3:42 PM / 15:42
    f.dateStyle = .none
    return f
}()

}

// MARK: - History Scroll Content
private extension HistoryDayOverlay {

    @ViewBuilder
    var historyContent: some View {
        VStack(alignment: .leading, spacing: 20) {

```

```

// ===== Menus Scanned =====
ForEach(
    historyStore.visibleRecords.filter {
        Calendar.isSameDay($0.scannedAt,
wrapper.date)
    }
) { scan in
    ScanCard(
        scan: scan,
        favRestaurantStore: favRestaurantStore,
        favMenuStore: favMenuStore,
        expandedMenuIDs: $expandedMenuIDs
    )
}

Divider()
    .background(Color.white.opacity(0.3))
    .padding(.vertical, 8)

// ===== Receipts Scanned =====
Text("Receipts Scanned")
    .font(.system(size: 17, weight: .semibold))
    .foregroundColor(.blue.opacity(0.8))

receiptsSection

// ===== Recipes Viewed =====
Text("Recipes Viewed")
    .font(.system(size: 17, weight: .semibold))
    .foregroundColor(.blue.opacity(0.8))

recipesViewedSection
}
}

@ViewBuilder
var receiptsSection: some View {
    if filteredReceiptRecords.isEmpty {
        Text("No receipts scanned yet")
            .foregroundColor(.white.opacity(0.7))
    } else {
        VStack(spacing: 12) {
            ForEach(filteredReceiptRecords) { record in
                receiptRow(record)
            }
        }
    }
}

```

```

    }
}

func receiptRow(_ record: ReceiptScanRecord) ->
some View {
    let isExpanded =
expandedReceiptIDs.contains(record.id)
    let isExpandable = record.items.count > 1

    return HStack(alignment: .top, spacing: 0) {

        ReceiptHistoryCard(
            record: record,
            isExpanded: isExpanded
        )
        .frame(maxWidth: .infinity, alignment: .leading)

        if isExpandable {
            ExpandedChevron(isExpanded: isExpanded)
        }

        Button {
            receiptVisibility.hide(record.id)
        } label: {
            Image(systemName: "trash")
                .foregroundColor(.white.opacity(0.3))
                .padding(.top, 18)
                .padding(.trailing, 16)
        }
        .buttonStyle(.plain)
    }
    .contentShape(Rectangle())
    .onTapGesture {
        guard isExpandable else { return }
        withAnimation(.easeInOut(duration: 0.25)) {
            expandedReceiptIDs.toggle(record.id)
        }
    }
}

@ViewBuilder
var recipesViewedSection: some View {
    if recipesViewedToday.isEmpty {
        Text("No recipes viewed")
            .foregroundColor(.white.opacity(0.6))
    } else {

```

```

VStack(spacing: 8) {
    ForEach(recipesViewedToday.prefix(5)) { record
in
        let normalizedName =
            record.recipeName
                .trimmingCharacters(in:
.whitespacesAndNewlines)

.precomposedStringWithCanonicalMapping

        HStack(spacing: 12) {

            // ✅ Navigation 영역
            NavigationLink {
                RecipeDetailView(name:
normalizedName)
            } label: {
                VStack(alignment: .leading, spacing: 4) {
                    Text(record.recipeName)
                        .foregroundColor(.white)
                        .multilineTextAlignment(.leading)

                    Text(
                        Self.timeFormatter.string(
                            from: record.viewedAt
                        )
                    )
                        .font(.caption)
                        .foregroundColor(.white.opacity(0.55))
                }
            }
            .buttonStyle(.plain)

            Spacer()

            // 🗑️ trash → chevron 왼쪽
            Button {
                RecipeVisibilityStore.shared.hide(id:
record.id)

                recomputeDerivedData()
            } label: {
                Image(systemName: "trash")
                    .foregroundColor(.white.opacity(0.3))
            }
            .buttonStyle(.plain)

```

```

        // 📺 Chevron RIGHT (항상 마지막)
        Image(systemName: "chevron.right")
            .foregroundColor(.white.opacity(0.35))
    }
    .padding()
    .background(Color.white.opacity(0.08))
    .cornerRadius(12)
    .contentShape(Rectangle()) // 🔥 터치 안정화
}
}
}
}
}
}
}
}

```

```

struct ExpandChevron: View {
    let isExpanded: Bool

    var body: some View {
        Image(systemName: "chevron.down")
            .foregroundColor(.white.opacity(0.35))
            .rotationEffect(.degrees(isExpanded ? 180 : 0))
            .animation(.easeInOut(duration: 0.2), value:
isExpanded)
            .padding(.top, 22)
            .padding(.trailing, 8)
    }
}

```

```

// MARK: - Set<UUID> helper
extension Set where Element == UUID {
    mutating func toggle(_ id: UUID) {
        if contains(id) {
            remove(id)
        } else {
            insert(id)
        }
    }
}
}

```

Show less ^

아래 코드에서 menus scanned섹션의 item들이 직접그린 chevron과 trash를 삭제해. 첨부화면에서 녹색 박스에 있는 녀석들 삭제해.

```
import SwiftUI

struct HistoryDayOverlay: View {
    // MARK: - Properties & State
    let wrapper: MenuScanListWrapper
    let onBack: () -> Void

    @StateObject private var historyStore =
MenuVisibilityStore.shared
    @StateObject private var favRestaurantStore =
FavoriteRestaurantStore.shared
    @StateObject private var favMenuStore =
FavoriteMenuStore.shared
    @StateObject private var receiptStore =
ReceiptScanStore.shared
    @StateObject private var receiptVisibility =
ReceiptVisibilityStore.shared

    @EnvironmentObject var bg: BackgroundManager

    @State private var expandedReceiptIDs: Set<UUID> =
```



```

[]

@State private var expandedMenuIDs: Set<UUID> = []
@State private var recipesViewedToday:
[RecipeViewRecord] = []

@State private var isVisible = false
@State private var showRecipeHistory = false

@State private var isMenusExpanded: Bool = true
@State private var isReceiptsExpanded: Bool = true
@State private var isRecipesExpanded: Bool = true

// MARK: - Computed Properties
private var filteredReceiptRecords:
[ReceiptScanRecord] {
    receiptStore.records
        .filter { Calendar.current.isDate($0.scannedAt,
inSameDayAs: wrapper.date) }
        .filter { !receiptVisibility.isHidden($0.id) }
    }

private var hasMenus: Bool {
    historyStore.visibleRecords.contains {
Calendar.isSameDay($0.scannedAt, wrapper.date) }
    }

private var hasReceipts: Bool {
!filteredReceiptRecords.isEmpty }
private var hasRecipes: Bool {
!recipesViewedToday.isEmpty }

// MARK: - Body
var body: some View {
    GeometryReader { geo in
        ZStack {
            // Background Overlay
            Color.black
                .opacity(isVisible ? 0.5 : 0)
                .ignoresSafeArea()
                .onTapGesture { closePanel() }

            HStack {
                Spacer()
                VStack(spacing: 0) {
                    headerView

```

```

        Text(wrapper.date.toYMD())
            .font(.system(size: 28, weight: .bold))
            .foregroundColor(.white)
            .padding(.vertical, 16)

        // 스와이프 가능한 리스트
        historyList
    }
    .frame(width: geo.size.width * 0.88, height:
geo.size.height)
    .background(Color.black)
    .offset(x: isVisible ? 0 : geo.size.width)
    .animation(.easeInOut(duration: 0.3), value:
isVisible)
    }
}
.onAppear {
    withAnimation { isVisible = true }
    receiptStore.refresh()
    recomputeDerivedData()
}
}
.sheet(isPresented: $showRecipeHistory) {
    RecipeHistoryView().environmentObject(bg)
}
}
}
}

```

// MARK: - Subviews & Logic

```

private extension HistoryDayOverlay {

    var headerView: some View {
        HStack {
            Button(action: closePanel) {
                HStack(spacing: 4) {
                    Image(systemName: "chevron.left")
                    Text("Back")
                }
                .foregroundColor(.white)
                .font(.system(size: 17, weight: .semibold))
            }
            Spacer()
        }
        .padding(.horizontal)
        .padding(.top, 12)
    }
}

```

```

var historyList: some View {
    List {
        // 1. Menus Scanned
        // 1. Menus Scanned 섹션 부분
        Section(header: HistorySectionHeader(title:
"Menus Scanned", hasContent: hasMenus, isExpanded:
$isMenusExpanded)) {
            if hasMenus && isMenusExpanded {
                ForEach(historyStore.visibleRecords.filter {
Calendar.isSameDay($0.scannedAt, wrapper.date) }) {
scan in
                                // HStack으로 감싸서 ScanCard 우측에 웨브론
배치
                                HStack(alignment: .top, spacing: 0) {
                                    ScanCard(
                                        scan: scan,
                                        favRestaurantStore:
favRestaurantStore,
                                        favMenuStore: favMenuStore,
                                        expandedMenuIDs:
$expandedMenuIDs
                                    )
                                    .frame(maxWidth: .infinity, alignment:
.leading)

                                    // 기존 쓰레기통 버튼 대신 웨브론 배치
                                    ExpandChevron(isExpanded:
expandedMenuIDs.contains(scan.id))
                                }
                                .contentShape(Rectangle())
                                .onTapGesture {
                                    withAnimation(.easeInOut(duration:
0.25)) {
                                        expandedMenuIDs.toggle(scan.id)
                                    }
                                }
                                .swipeActions(edge: .trailing) {
                                    Button(role: .destructive) {
                                        historyStore.hide(scan.id)
                                    } label: {
                                        Label("Delete", systemImage: "trash")
                                    }
                                }
                            }
                        }
                    }
                .listRowBackground(Color.clear)

```

```

        .listRowSeparator(.hidden)
    }
}
.textCase(nil)

// 2. Receipts Scanned
Section(header: HistorySectionHeader(title:
"Receipts Scanned", hasContent: hasReceipts,
isExpanded: $isReceiptsExpanded)) {
    if hasReceipts && isReceiptsExpanded {
        ForEach(filteredReceiptRecords) { record in
            receiptRow(record)
                .swipeActions(edge: .trailing) {
                    Button(role: .destructive) {
receiptVisibility.hide(record.id) } label: { Label("Delete",
systemImage: "trash") }
                }
        }
        .listRowBackground(Color.clear)
        .listRowSeparator(.hidden)
    }
}
.textCase(nil)

// 3. Recipes Viewed
Section(header: HistorySectionHeader(title:
"Recipes Viewed", hasContent: hasRecipes, isExpanded:
$isRecipesExpanded)) {
    if hasRecipes && isRecipesExpanded {
        ForEach(recipesViewedToday.prefix(5)) {
record in
            recipeRow(record)
                .swipeActions(edge: .trailing) {
                    Button(role: .destructive) {
                        RecipeVisibilityStore.shared.hide(id:
record.id)
                            recomputeDerivedData()
                        } label: { Label("Delete", systemImage:
"trash") }
                }
        }
        .listRowBackground(Color.clear)
        .listRowSeparator(.hidden)
    }
}
.textCase(nil)

```

```

    }
    .listStyle(.plain)
    .scrollContentBackground(.hidden)
    .background(Color.black)
}

func receiptRow(_ record: ReceiptScanRecord) ->
some View {
    let isExpanded =
expandedReceiptIDs.contains(record.id)
    return HStack(alignment: .top, spacing: 0) {
        ReceiptHistoryCard(record: record, isExpanded:
isExpanded)
        .frame(maxWidth: .infinity, alignment: .leading)
        if record.items.count > 1 {
            ExpandChevron(isExpanded: isExpanded)
        }
    }
    .contentShape(Rectangle())
    .onTapGesture {
        if record.items.count > 1 {
            withAnimation {
expandedReceiptIDs.toggle(record.id) }
        }
    }
}

func recipeRow(_ record: RecipeViewRecord) -> some
View {
    let normalizedName =
record.recipeName.trimmingCharacters(in:
.whitespacesAndNewlines).precomposedStringWithCano
nicalMapping

    return NavigationLink {
        RecipeDetailLoadingView(name:
normalizedName)
    } label: {
        HStack {
            VStack(alignment: .leading, spacing: 4) {
                Text(record.recipeName)
                .foregroundColor(.white)
                Text(Self.timeFormatter.string(from:
record.viewedAt))
                .font(.caption)
                .foregroundColor(.white.opacity(0.55))
            }
        }
    }
}

```

```

    }
    Spacer()
    // 직접 그린 chevron.right 이미지를 삭제했습니다.
    // List 내부의 NavLink는 시스템이 자동으로 우
측 쉼표를 생성합니다.
    }
    .padding()
    .background(Color.white.opacity(0.08))
    .cornerRadius(12)
    }
    .buttonStyle(.plain)
}

// MARK: - Helper Methods
func closePanel() {
    withAnimation(.easeInOut(duration: 0.3)) { isVisible =
false }
    DispatchQueue.main.asyncAfter(deadline: .now() +
0.3) { onBack() }
}

func recomputeDerivedData() {
    recipesViewedToday = RecipeViewStore.shared
        .records(on: wrapper.date)
        .filter { !RecipeVisibilityStore.shared.isHidden(id:
$0.id) }
        .sorted { $0.viewedAt > $1.viewedAt }
    }

    static let timeFormatter: DateFormatter = {
        let f = DateFormatter()
        f.timeStyle = .short
        return f
    }()
}

// MARK: - Supporting Components (Internal)
struct HistorySectionHeader: View {
    let title: String
    let hasContent: Bool
    @Binding var isExpanded: Bool

    var body: some View {
        HStack {
            Text(title).font(.system(size: 17, weight:
.semibold)).foregroundColor(.blue.opacity(0.8))

```

```

        Spacer()
        if hasContent {
            Image(systemName: "chevron.down")
                .foregroundColor(.blue.opacity(0.8))
                .rotationEffect(.degrees(isExpanded ? 0 :
-90))
        }
    }
    .padding(.vertical, 8)
    .contentShape(Rectangle())
    .onTapGesture {
        if hasContent { withAnimation {
isExpanded.toggle() } }
    }
}
}

struct ExpandChevron: View {
    let isExpanded: Bool
    var body: some View {
        Image(systemName: "chevron.down")
            .foregroundColor(.white.opacity(0.35))
            .rotationEffect(.degrees(isExpanded ? 180 : 0))
            .padding(.top, 22).padding(.trailing, 8)
    }
}

extension Set where Element == UUID {
    mutating func toggle(_ id: UUID) {
        if contains(id) { remove(id) } else { insert(id) }
    }
}

```

Show less ^

menus scanned item들도 receipts scanned items 처럼
item 갯수와 총 칼로리 시간을 나타내라.

```
import SwiftUI
```

```
struct HistoryDayOverlay: View {
```

```
    // MARK: - Properties & State
```

```
    let wrapper: MenuScanListWrapper
```

```
    let onBack: () -> Void
```

```
    @StateObject private var historyStore =  
MenuVisibilityStore.shared
```

```
    @StateObject private var favRestaurantStore =  
FavoriteRestaurantStore.shared
```

```
    @StateObject private var favMenuStore =  
FavoriteMenuStore.shared
```

```
    @StateObject private var receiptStore =  
ReceiptScanStore.shared
```

```
    @StateObject private var receiptVisibility =  
ReceiptVisibilityStore.shared
```

```
    @EnvironmentObject var bg: BackgroundManager
```

```
    @State private var expandedReceiptIDs: Set<UUID> =  
[]
```

```
    @State private var expandedMenuIDs: Set<UUID> = []
```

```
    @State private var recipesViewedToday:  
[RecipeViewRecord] = []
```

```
    @State private var isVisible = false
```

```
    @State private var showRecipeHistory = false
```

```

@State private var isMenusExpanded: Bool = true
@State private var isReceiptsExpanded: Bool = true
@State private var isRecipesExpanded: Bool = true

// MARK: - Computed Properties
private var filteredReceiptRecords:
[ReceiptScanRecord] {
    receiptStore.records
        .filter { Calendar.current.isDate($0.scannedAt,
inSameDayAs: wrapper.date) }
        .filter { !receiptVisibility.isHidden($0.id) }
    }

private var hasMenus: Bool {
    historyStore.visibleRecords.contains {
        Calendar.isSameDay($0.scannedAt, wrapper.date) }
    }

private var hasReceipts: Bool {
!filteredReceiptRecords.isEmpty }
private var hasRecipes: Bool {
!recipesViewedToday.isEmpty }

// MARK: - Body
var body: some View {
    GeometryReader { geo in
        ZStack {
            // Background Overlay
            Color.black
                .opacity(isVisible ? 0.5 : 0)
                .ignoresSafeArea()
                .onTapGesture { closePanel() }

            HStack {
                Spacer()
                VStack(spacing: 0) {
                    headerView

                    Text(wrapper.date.toYMD())
                        .font(.system(size: 28, weight: .bold))
                        .foregroundColor(.white)
                        .padding(.vertical, 16)

                    // 스와이프 가능한 리스트
                    historyList
                }
            }
        }
    }
}

```

```

        .frame(width: geo.size.width * 0.88, height:
geo.size.height)
        .background(Color.black)
        .offset(x: isVisible ? 0 : geo.size.width)
        .animation(.easeInOut(duration: 0.3), value:
isVisible)
    }
}
.onAppear {
    withAnimation { isVisible = true }
    receiptStore.refresh()
    recomputeDerivedData()
}
}
.sheet(isPresented: $showRecipeHistory) {
    RecipeHistoryView().environmentObject(bg)
}
}
}
}

```

// MARK: - Subviews & Logic

```

private extension HistoryDayOverlay {

    var headerView: some View {
        HStack {
            Button(action: closePanel) {
                HStack(spacing: 4) {
                    Image(systemName: "chevron.left")
                    Text("Back")
                }
                .foregroundColor(.white)
                .font(.system(size: 17, weight: .semibold))
            }
            Spacer()
        }
        .padding(.horizontal)
        .padding(.top, 12)
    }

    var historyList: some View {
        List {
            // 1. Menu Scanned 섹션 부분
            Section(header: HistorySectionHeader(title:
"Menu Scanned", hasContent: hasMenus, isExpanded:
$isMenusExpanded)) {
                if hasMenus && isMenusExpanded {

```



```

        ForEach(historyStore.visibleRecords.filter {
            Calendar.isSameDay($0.scannedAt, wrapper.date) }) {
            scan in
                // HStack으로 감싸서 ScanCard 우측에 웨브론
                배치
                HStack(alignment: .top, spacing: 0) {
                    ScanCard(
                        scan: scan,
                        favRestaurantStore:
favRestaurantStore,
                        favMenuStore: favMenuStore,
                        expandedMenuIDs:
$expandedMenuIDs
                    )
                    .frame(maxWidth: .infinity, alignment:
.leading)

                        // 기존 쓰레기통 버튼 대신 웨브론 배치
                        ExpandChevron(isExpanded:
expandedMenuIDs.contains(scan.id))
                }
                .contentShape(Rectangle())
                .onTapGesture {
                    withAnimation(.easeInOut(duration:
0.25)) {
                        expandedMenuIDs.toggle(scan.id)
                    }
                }
                .swipeActions(edge: .trailing) {
                    Button(role: .destructive) {
                        historyStore.hide(scan.id)
                    } label: {
                        Label("Delete", systemImage: "trash")
                    }
                }
            }
            .listRowBackground(Color.clear)
            .listRowSeparator(.hidden)
        }
    }
    .textCase(nil)

    // 2. Receipts Scanned
    Section(header: HistorySectionHeader(title:
"Receipts Scanned", hasContent: hasReceipts,
isExpanded: $isReceiptsExpanded)) {

```

```

        if hasReceipts && isReceiptsExpanded {
            ForEach(filteredReceiptRecords) { record in
                receiptRow(record)
                    .swipeActions(edge: .trailing) {
                        Button(role: .destructive) {
                            receiptVisibility.hide(record.id) } label: { Label("Delete",
                                systemImage: "trash") }
                    }
            }
            .listRowBackground(Color.clear)
            .listRowSeparator(.hidden)
        }
    }
    .textCase(nil)

// 3. Recipes Viewed
Section(header: HistorySectionHeader(title:
"Recipes Viewed", hasContent: hasRecipes, isExpanded:
$isRecipesExpanded)) {
    if hasRecipes && isRecipesExpanded {
        ForEach(recipesViewedToday.prefix(5)) {
            record in
                recipeRow(record)
                    .swipeActions(edge: .trailing) {
                        Button(role: .destructive) {
                            RecipeVisibilityStore.shared.hide(id:
                                record.id)
                                recomputeDerivedData()
                        } label: { Label("Delete", systemImage:
                            "trash") }
                    }
        }
        .listRowBackground(Color.clear)
        .listRowSeparator(.hidden)
    }
}
.textCase(nil)
}
.listStyle(.plain)
.scrollContentBackground(.hidden)
.background(Color.black)
}

```

```

func receiptRow(_ record: ReceiptScanRecord) ->
some View {
    let isExpanded =

```

```

expandedReceiptIDs.contains(record.id)
    return HStack(alignment: .top, spacing: 0) {
        ReceiptHistoryCard(record: record, isExpanded:
isExpanded)
            .frame(maxWidth: .infinity, alignment: .leading)
        if record.items.count > 1 {
            ExpandChevron(isExpanded: isExpanded)
        }
    }
    .contentShape(Rectangle())
    .onTapGesture {
        if record.items.count > 1 {
            withAnimation {
expandedReceiptIDs.toggle(record.id) }
        }
    }
}

func recipeRow(_ record: RecipeViewRecord) -> some
View {
    let normalizedName =
record.recipeName.trimmingCharacters(in:
.whitespacesAndNewlines).precomposedStringWithCano
nicalMapping

    return NavigationLink {
        RecipeDetailLoadingView(name:
normalizedName)
    } label: {
        HStack {
            VStack(alignment: .leading, spacing: 4) {
                Text(record.recipeName)
                    .foregroundColor(.white.opacity(0.7))
                Text(Self.timeFormatter.string(from:
record.viewedAt))
                    .font(.caption)
                    .foregroundColor(.white.opacity(0.55))
            }
            Spacer()
            // 직접 그린 chevron.right 이미지를 삭제했습니다.
            // List 내부의 NavigationLink는 시스템이 자동으로 우
측 웨브론을 생성합니다.
        }
        .padding()
        .background(Color.white.opacity(0.08))
        .cornerRadius(12)
    }
}

```

```

    }
    .buttonStyle(.plain)
}

// MARK: - Helper Methods
func closePanel() {
    withAnimation(.easeInOut(duration: 0.3)) { isVisible =
false }
    DispatchQueue.main.asyncAfter(deadline: .now() +
0.3) { onBack() }
}

func recomputeDerivedData() {
    recipesViewedToday = RecipeViewStore.shared
        .records(on: wrapper.date)
        .filter { !RecipeVisibilityStore.shared.isHidden(id:
$0.id) }
        .sorted { $0.viewedAt > $1.viewedAt }
}

static let timeFormatter: DateFormatter = {
    let f = DateFormatter()
    f.timeStyle = .short
    return f
}()
}

// MARK: - Supporting Components (Internal)
struct HistorySectionHeader: View {
    let title: String
    let hasContent: Bool
    @Binding var isExpanded: Bool

    var body: some View {
        HStack {
            Text(title).font(.system(size: 18, weight:
.semibold)).foregroundColor(.white.opacity(1.0))
            Spacer()
            if hasContent {
                Image(systemName: "chevron.down")
                    .foregroundColor(.white.opacity(0.35))
                    .rotationEffect(.degrees(isExpanded ? 0 :
-90))
            }
        }
        .padding(.vertical, 8)
    }
}

```

```

        .contentShape(Rectangle())
        .onTapGesture {
            if hasContent { withAnimation {
isExpanded.toggle() } }
        }
    }
}

struct ExpandChevron: View {
    let isExpanded: Bool
    var body: some View {
        Image(systemName: "chevron.down")
            .foregroundColor(.white.opacity(0.35))
            .rotationEffect(.degrees(isExpanded ? 180 : 0))
            .padding(.top, 22).padding(.trailing, 8)
    }
}

extension Set where Element == UUID {
    mutating func toggle(_ id: UUID) {
        if contains(id) { remove(id) } else { insert(id) }
    }
}

```

Show less ^

recipe viewed item에 총칼로리가 나오게 해라

안나온다. daily record의 총칼로리가 receipt의것인지 recipe의
것인지 어떻게 아나? recipeviewrecord에 있는것을 직접써야 하
는것 아닌가?

칼로리가 나온다 칼로리를 시간 앞에 두고 시간과 색깔을 같게 해라.
앞의 dot은 지운다.

```
//  
// RecipeHistoryView.swift  
// MenuCal  
//  
  
import Foundation  
import SwiftUI  
  
struct RecipeHistoryView: View {  
    @Environment(\.dismiss) var dismiss  
    @EnvironmentObject var bg: BackgroundManager  
  
    @State private var grouped: [(date: Date, records:  
[DailyRecord])] = []  
  
    // 🌟 name 기반 추천 레시피  
    @State private var recommendedRecipes: [String] = []  
  
    var body: some View {  
        NavigationView {  
            ScrollView {  
                VStack(spacing: 25) {  
  
                    recommendationSection()  
                        .padding(.vertical, 10)  
                }  
            }  
        }  
    }  
}
```



```

        ForEach(grouped, id: \.date) { group in
            sectionView(group.date, group.records)
        }
    }
    .padding(.vertical)
}
.background(
    Image(bg.imageName(for: 4))
        .resizable()
        .aspectRatio(contentMode: .fill)
        .ignoresSafeArea()
)
.navigationTitle("Recipes")
.toolbar {
    ToolbarItem(placement: .navigationBarLeading)
{
    Button { dismiss() } label: {
        HStack(spacing: 3) {
            Image(systemName: "chevron.left")
            Text("Back")
        }
        .foregroundColor(.white)
    }
}
}
.onAppear {
    recomputeGrouped()
    loadRecommendations()
}
}
}

```

// MARK: - 추천 로딩 (✅ name 기반)

```

private func loadRecommendations() {
    Task {
        let names = await
RecipeSearchService.shared.getRandomRecipes(count:
3)
        await MainActor.run {
            self.recommendedRecipes = names
        }
    }
}
}

```

// MARK: - 추천 UI

```

private func recommendationSection() -> some View {
    VStack(alignment: .leading, spacing: 15) {
        Text("Today's Picks")
            .font(.system(size: 24, weight: .bold))
            .foregroundColor(.white)
            .padding(.leading, 20)

        VStack(spacing: 0) {
            ForEach(recommendedRecipes, id: \.self) {
name in
                NavigationLink {
                    // ✅ name-only 이동
                    RecipeDetailLoadingView(name: name)
                } label: {
                    recommendationRow(dish: name)
                }
            }
        }
        .padding(.horizontal)
    }
}

private func recomputeGrouped() {
    grouped = []
}

// MARK: - 추천 항목 UI
private func recommendationRow(dish: String) ->
some View {
    HStack(spacing: 14) {
        Text(autoEmoji(for: dish))
            .font(.system(size: 26))

        Text(dish)
            .font(.system(size: 19, weight: .semibold,
design: .rounded))
            .foregroundColor(.white)
            .multilineTextAlignment(.leading)

        Spacer()

        Image(systemName: "sparkles")
            .foregroundColor(.yellow.opacity(0.8))
    }
    .frame(maxWidth: .infinity, alignment: .leading)
    .padding(.vertical, 8)
}

```

```

        .padding(.horizontal, 14)
        .background(Color.clear)
        .cornerRadius(12)
    }

    // MARK: - 날짜 섹션
    private func sectionView(_ date: Date, _ records:
[DailyRecord]) -> some View {
        VStack(alignment: .leading, spacing: 10) {

            Text(date.toString())
                .foregroundColor(.white)

            VStack(spacing: 0) {
                ForEach(records) { rec in
                    let dish = rec.items.first ?? "Unknown"

                    NavigationLink {
                        // ✅ name-only
                        RecipeDetailLoadingView(name: dish)
                    } label: {
                        recipeRow(rec)
                    }
                    .buttonStyle(.plain)
                }
            }
        }
        .padding(.horizontal)
    }

    // MARK: - 레시피 항목 UI
    private func recipeRow(_ rec: DailyRecord) -> some
View {
        let dish = rec.items.first ?? "Unknown"

        return HStack(spacing: 12) {
            Text(autoEmoji(for: dish))
                .font(.system(size: 22))

            VStack(alignment: .leading, spacing: 2) {
                Text(dish)
                    .foregroundColor(.white)

                // ✅ Optional 아님 → 바로 사용
                Text("\(Int(rec.totalCalories)) kcal")
                    .font(.caption)
            }
        }
    }

```

```

        .foregroundColor(.yellow.opacity(0.85))
    }

    Spacer()
}
.padding()
}
}

// 기존 확장 함수들은 그대로 유지
extension Date {
    func toString(_ format: String = "yyyy-MM-dd") ->
String {
        let df = DateFormatter()
        df.dateFormat = format
        return df.string(from: self)
    }
}

extension DailyRecord {
    var dayOnly: Date {
        Calendar.current.startOfDay(for: self.date)
    }
}

```

Show less ^

왜 안바뀌나?

```
//  
// RecipeHistoryView.swift  
// MenuCal  
//  
  
import Foundation  
import SwiftUI  
  
struct RecipeHistoryView: View {  
    @Environment(\.dismiss) var dismiss  
    @EnvironmentObject var bg: BackgroundManager  
  
    @State private var grouped: [(date: Date, records:  
[DailyRecord])] = []  
  
    // 🌟 name 기반 추천 레시피  
    @State private var recommendedRecipes: [String] = []  
  
    var body: some View {  
        NavigationView {  
            ScrollView {  
                VStack(spacing: 25) {  
  
                    recommendationSection()  
                        .padding(.vertical, 10)  
  
                    ForEach(grouped, id: \.date) { group in  
                        sectionView(group.date, group.records)  
                    }  
                }  
                .padding(.vertical)  
            }  
            .background(  
                Image(bg.imageName(for: 4))  
                    .resizable()  
                    .aspectRatio(contentMode: .fill)  
                    .ignoresSafeArea()  
            )  
        }  
    }  
}
```

```

    )
    .navigationTitle("Recipes")
    .toolbar {
        ToolbarItem(placement: .navigationBarLeading)
    {
        Button { dismiss() } label: {
            HStack(spacing: 3) {
                Image(systemName: "chevron.left")
                Text("Back")
            }
            .foregroundColor(.white)
        }
    }
}
.onAppear {
    recomputeGrouped()
    loadRecommendations()
}
}

```

// MARK: - 추천 로딩 (✅ name 기반)

```

private func loadRecommendations() {
    Task {
        let names = await
RecipeSearchService.shared.getRandomRecipes(count:
3)

        await MainActor.run {
            self.recommendedRecipes = names
        }
    }
}

```

// MARK: - 추천 UI

```

private func recommendationSection() -> some View {
    VStack(alignment: .leading, spacing: 15) {
        Text("Today's Picks")
            .font(.system(size: 24, weight: .bold))
            .foregroundColor(.white)
            .padding(.leading, 20)

        VStack(spacing: 0) {
            ForEach(recommendedRecipes, id: \.self) {
name in
                NavigationLink {
                    // ✅ name-only 이동

```

```

        RecipeDetailLoadingView(name: name)
    } label: {
        recommendationRow(dish: name)
    }
    }
}
}
.padding(.horizontal)
}
}

private func recomputeGrouped() {
    grouped = []
}

// MARK: - 추천 항목 UI
private func recommendationRow(dish: String) ->
some View {
    HStack(spacing: 14) {
        Text(autoEmoji(for: dish))
            .font(.system(size: 26))

        Text(dish)
            .font(.system(size: 19, weight: .semibold,
design: .rounded))
            .foregroundColor(.white)
            .multilineTextAlignment(.leading)

        Spacer()

        Image(systemName: "sparkles")
            .foregroundColor(.yellow.opacity(0.8))
    }
    .frame(maxWidth: .infinity, alignment: .leading)
    .padding(.vertical, 8)
    .padding(.horizontal, 14)
    .background(Color.clear)
    .cornerRadius(12)
}

// MARK: - 날짜 섹션
private func sectionView(_ date: Date, _ records:
[DailyRecord]) -> some View {
    VStack(alignment: .leading, spacing: 10) {

        Text(date.toString())
            .foregroundColor(.white)

```

```

VStack(spacing: 0) {
    ForEach(records) { rec in
        let dish = rec.items.first ?? "Unknown"

        NavigationLink {
            // ✅ name-only
            RecipeDetailLoadingView(name: dish)
        } label: {
            recipeRow(rec)
        }
        .buttonStyle(.plain)
    }
}
.padding(.horizontal)
}

// MARK: - 레시피 항목 UI
private func recipeRow(_ rec: DailyRecord) -> some
View {
    let dish = rec.items.first ?? "Unknown"

    return HStack(spacing: 12) {
        Text(autoEmoji(for: dish))
        .font(.system(size: 22))

        VStack(alignment: .leading, spacing: 2) {
            Text(dish)
            .foregroundColor(.white)

            HStack(spacing: 6) {
                // 🔥 칼로리 먼저
                Text("\(Int(rec.totalCalories)) kcal")

                // 🔥 시간
                Text(rec.date.formatted(date: .omitted, time:
.shortened))
            }
            .font(.caption)
            .foregroundColor(.white.opacity(0.55)) // ✅ 시
간과 동일한 색
        }

        Spacer()
    }
}

```



```
        .padding()
    }
}

// 기존 확장 함수들은 그대로 유지
extension Date {
    fun toString(_ format: String = "yyyy-MM-dd") ->
    String {
        let df = DateFormatter()
        df.dateFormat = format
        return df.string(from: self)
    }
}

extension DailyRecord {
    var dayOnly: Date {
        Calendar.current.startOfDay(for: self.date)
    }
}
}
```

Show less ^

아래에서 각 category title "Menus Scanned", "Receipts Scanned" "Recipes Viewed" 앞에 첨부된 홈화면의 이모티콘과 같은 이모티콘을 타이틀 앞에 붙여라.

```
VStack(alignment: .leading, spacing: 2) {  
  Text("\(record.items.count) items · \  
(record.totalCalories) kcal")  
  Text(timeString)  
}  
.font(.caption) // ✅ 시간과 동일  
.foregroundColor(.white.opacity(0.5)) // ✅ 시간과 동일
```


receipt scanned item의 카드 색을 menu scanned item의 카드 색과 같게 해라.

첨부화면에서 default로 캘렌터 밑에 today's record를 나타낼수
있나?

칼로리와 단위가 없는것은 왜 삭제 마크가 붙지 않나? 붙여라.

in the following code, make ingredient categories expandable (expand/collapse) under "Select Ingredient (single)" window. see the attached image. by default, they are all collapsed.

```
//  
// MyRecipeEditorView.swift  
// MenuCal  
//  
// Created by Dae-Kyoo Kim on 12/19/25.  
//
```

```
import SwiftUI  
import Foundation //  이 줄 추가
```

```
struct MyRecipeEditorView: View {
```

```
    enum Mode: Identifiable {  
        case create  
        case edit(UserRecipe)
```

```

var id: String {
    switch self {
        case .create: return "create"
        case .edit(let r): return r.id.uuidString
    }
}
}
}

```

```

let mode: Mode
@Environment(\.dismiss) private var dismiss
@State private var isTyping = false

```

```

@StateObject private var store =
UserRecipeStore.shared
@State private var isShowingAddCustomIngredient =
false

```

```

// ---- name + suggestions
@State private var recipeName: String = ""
@State private var suggestions: [String] = []
@State private var isSuggesting = false
@FocusState private var isNameFocused: Bool

```

```

private let units = ["g","ml","cup","tbsp","tsp","piece"]

```

```

@State private var selectedIngredient: String? = nil
@State private var selectedUnit: String = "g"
@State private var selectedAmount: Int = 1

```

```

// ---- current recipe ingredients
@State private var pickedIngredients: [UserIngredient]
= []
@EnvironmentObject var ingredientStore:
IngredientStore

```

```

// ---- loading
@State private var isLoadingDB = true

```

```

var body: some View {
    ScrollView {
        VStack(alignment: .leading, spacing: 18) {

            if isLoadingDB {
                ProgressView()
                    .padding(.top, 40)
                    .frame(maxWidth: .infinity)
            }

```

```

    } else {
        nameSection
        suggestionDropdown
        ingredientGridSection
        unitAmountSection
        addIngredientButton

        Text("Can't find it?")
            .font(.footnote)
            .foregroundColor(.white.opacity(0.6))
            .frame(maxWidth: .infinity, alignment:
.center)

        addCustomIngredientButton

        pickedListSection
        saveButton
    }
}
.padding()
}
// ✅🔥 핵심: ScrollView 바깥 배경
.navigationTitle(modeTitle)
.navigationBarTitleDisplayMode(.inline)
.toolbar {

    // Cancel (항상 표시)
    ToolbarItem(placement: .topBarLeading) {
        Button("Cancel") {
            dismiss()
        }
    }

    // 🔥 Delete (Edit 모드에서만)
    ToolbarItem(placement: .topBarTrailing) {
        if case .edit(let recipe) = mode {
            Button(role: .destructive) {
                store.delete(recipe)
                dismiss()
            } label: {
                Image(systemName: "trash")
            }
        }
    }
}
}
.sheet(isPresented:

```

```

$isShowingAddCustomIngredient) {
    AddIngredientBar(store: ingredientStore) {
newIngredient in
        selectedIngredient = newIngredient
    }
}
.task {
    await RecipeDetailStore.shared.loadIfNeeded()
    isLoadingDB = false

    switch mode {
    case .create:
        break
    case .edit(let r):
        recipeName = r.name
        pickedIngredients = r.ingredients
    }
}
.onChange(of: recipeName) { text in
    Task { await updateSuggestions(for: text) }
}
.onTapGesture {
    endTyping()
}
}

private func endTyping() {
    isTyping = false
    suggestions = []
    isSuggesting = false
    isNameFocused = false // 🔥 키보드 내려감
}

private var modeTitle: String {
    switch mode {
    case .create: return "Create"
    case .edit: return "Edit"
    }
}

// MARK: - Sections

private var nameSection: some View {
    VStack(alignment: .leading, spacing: 8) {
        Text("Recipe Name")
        .foregroundColor(.white)
    }
}

```

```
.font(.system(size: 16, weight: .bold))
```

```
        TextField("Type recipe name...", text:
$recipeName)
            .textInputAutocapitalization(.words)
            .padding(12)
            .background(Color.black.opacity(0.7))
            .foregroundColor(.white)
            .cornerRadius(10)
            .focused($isNameFocused)
            .onChange(of: recipeName) { _ in
                isTyping = true
            }
            .onSubmit {
                // ✅ 타이핑 종료
                isTyping = false
                suggestions = []
                isSuggesting = false
            }
        }
    }
}
```

```
private var suggestionDropdown: some View {
    Group {
        if isTyping && !suggestions.isEmpty {
            VStack(alignment: .leading, spacing: 0) {
                ForEach(suggestions.indices, id: \.self) { i in
                    let s = suggestions[i]
                    Button {
                        recipeName = s
                        endTyping()
                        hideKeyboard()
                    } label: {
                        HStack {
                            Text(s)
                                .foregroundColor(.white)
                                .lineLimit(1)
                            Spacer()
                        }
                    }
                    .padding(.vertical, 10)
                    .padding(.horizontal, 12)
                    .background(Color.black.opacity(0.22))
                }
                .buttonStyle(.plain)

                if i != suggestions.count - 1 {
```



```

        Rectangle().fill(Color.white.opacity(0.12)).frame(height: 1)
    }
}
}
    .cornerRadius(10)
}
}
}

```

```

private var ingredientGridSection: some View {
    VStack(alignment: .leading, spacing: 10) {

```

```

        Text("Select Ingredient (single)")
        .foregroundColor(.white)
        .font(.system(size: 16, weight: .bold))

```

```

        ScrollView {
            VStack(alignment: .leading, spacing: 20) {

```

```

                ForEach(ingredientStore.categorizedIngredients, id:
                \.category) { section in

```

```

                    Text(section.category.rawValue)
                    .font(.system(size: 17, weight:
                    .semibold))

```

```

                    .foregroundColor(Color.mint.opacity(0.95)) // ★ 핵심 변경

```

```

                        .padding(.top, 8)
                        .padding(.leading, 2)

```

```

                    Rectangle()
                    .fill(Color.white.opacity(0.15))
                    .frame(height: 1)
                    .padding(.bottom, 6)

```

```

                    LazyVGrid(
                        columns: Array(
                            repeating: GridItem(.flexible()),
                            count: 3
                        ),
                        spacing: 10, alignment: .leading,
                    ),
                    spacing: 10
                ) {

```

```

        ForEach(section.items.indices, id: \.self)
    { index in
        let ing = section.items[index]
        ingredientCell(ing)
    }
    .frame(maxWidth: .infinity, alignment:
.leading)
    .padding(.top, 4)
    }
    }
    }
    .padding(.vertical, 4)
    }
    .frame(maxHeight: 320)
    .background(Color.black.opacity(0.18))
    .cornerRadius(12)
    }
    }

private var unitAmountSection: some View {
    Group {
        if selectedIngredient != nil {
            VStack(alignment: .leading, spacing: 8) {
                Text("Unit & Amount")
                    .foregroundColor(.white)
                    .font(.system(size: 16, weight: .bold))

                HStack(spacing: 12) {
                    Picker("Unit", selection: $selectedUnit) {
                        ForEach(units, id: \.self) { Text($0) }
                    }
                    .pickerStyle(.wheel)
                    .frame(maxWidth: .infinity, minHeight: 120)

                    Picker("Amount", selection:
$selectedAmount) {
                        ForEach(1...30, id: \.self) { Text("\($0)") }
                    }
                    .pickerStyle(.wheel)
                    .frame(maxWidth: .infinity, minHeight: 120)
                }
                .background(Color.black.opacity(0.18))
                .cornerRadius(12)
            }
        }
    }
}

```

```

}

private var addCustomIngredientButton: some View {
    Button {
        // 기존 AddIngredientBar가 하던 동작으로 연결
        isShowingAddCustomIngredient = true
    } label: {
        HStack {
            Image(systemName: "plus.circle.fill")
            Text("Add Custom Ingredient")
                .fontWeight(.semibold)
        }
        .foregroundColor(.black)
        .frame(maxWidth: .infinity)
        .padding(.vertical, 14)
        .background(
            LinearGradient(
                colors: [Color.green.opacity(0.85),
Color.green],
                startPoint: .leading,
                endPoint: .trailing
            )
        )
        .cornerRadius(14)
    }
}

```

```

private var addIngredientButton: some View {
    Button {
        guard let ing = selectedIngredient else { return }

        // 같은 재료를 여러 번 허용할지 정책 필요:
        // 지금은 "허용" (원하면 중복 방지로 바꿔줄게)
        pickedIngredients.append(
            UserIngredient(name: ing, unit: selectedUnit,
amount: selectedAmount)
        )

        // ✅ single selection reset
        selectedIngredient = nil
        selectedAmount = 1
        selectedUnit = "g"
    } label: {
        HStack {
            Image(systemName: "plus")
            Text("Add Ingredient")

```

```

        .fontWeight(.bold)
    }
    .foregroundColor(.black)
    .frame(maxWidth: .infinity)
    .padding(.vertical, 12)
    .background(
        (selectedIngredient == nil) ?
Color.gray.opacity(0.35) : Color.yellow.opacity(0.95)
    )
    .cornerRadius(14)
}
.disabled(selectedIngredient == nil)
}

private var pickedListSection: some View {
    VStack(alignment: .leading, spacing: 10) {
        Text("Ingredients Added")
            .foregroundColor(.white)
            .font(.system(size: 16, weight: .bold))

        if pickedIngredients.isEmpty {
            Text("No ingredients yet.")
                .foregroundColor(.white.opacity(0.7))
                .font(.system(size: 14))
        } else {
            VStack(spacing: 8) {
                ForEach(pickedIngredients.indices, id: \.self) {
idx in
                    let item = pickedIngredients[idx]
                    HStack {
                        Text("• \(item.name)")
                            .foregroundColor(.white)
                        Spacer()
                        Text("\(item.amount) \(item.unit)")
                            .foregroundColor(.white.opacity(0.8))
                        Button {
                            pickedIngredients.remove(at: idx)
                        } label: {
                            Image(systemName: "trash")
                                .foregroundColor(.red.opacity(0.9))
                        }
                        .buttonStyle(.plain)
                    }
                    .padding(12)
                    .background(Color.black.opacity(0.25))
                    .cornerRadius(10)
                }
            }
        }
    }
}

```

```

    }
    }
    }
    }
}

```

```

private var saveButton: some View {
    Button {
        let finalName = resolvedRecipeName()
        guard !pickedIngredients.isEmpty else { return }

        switch mode {
        case .create:
            let new = UserRecipe(
                name: finalName,
                ingredients: pickedIngredients
            )
            store.add(new)

        case .edit(let old):
            var edited = old
            edited.name = finalName
            edited.ingredients = pickedIngredients
            edited.updatedAt = Date()
            store.update(edited)
        }

        dismiss()

    } label: {
        HStack {
            Image(systemName: "checkmark.circle.fill")
            Text("Save")
                .fontWeight(.bold)
        }
        .foregroundColor(.black)
        .frame(maxWidth: .infinity)
        .padding(.vertical, 14)
        .background(Color.green.opacity(0.9))
        .cornerRadius(16)
    }
    .disabled(pickedIngredients.isEmpty)
    .padding(.top, 6)
}

```

```

private func resolvedRecipeName() -> String {

```

```
let trimmed = recipeName.trimmingCharacters(in:
.whitespacesAndNewlines)
```

```
// 1 정상 입력된 이름이 있으면 그대로 사용
if trimmed.count >= 2 {
    return trimmed
}
```

```
// 2 default 이름 생성 (중복 방지)
let base = "My Recipe"
let existingNames = store.recipes.map { $0.name }
```

```
if !existingNames.contains(base) {
    return base
}
```

```
var index = 2
while existingNames.contains("\(base) \(index)") {
    index += 1
}
```

```
return "\(base) \(index)"
}
```

```
// MARK: - Suggestions
```

```
@MainActor
private func updateSuggestions(for text: String) async
{
    let q = text.trimmingCharacters(in:
.whitespacesAndNewlines)
```

```
// 1글자도 제안 나오게: 빈 문자열만 차단
guard !q.isEmpty else {
    suggestions = []
    return
}
```

```
let qLower = q.lowercased()
```

```
// ⚠ allRecipeNames() 없으면, 네가 이미 쓰던 hits 기반
(임베딩)으로 가져와도 됨.
```

```
// 여기서는 "이름 풀"이 있다고 가정한 버전.
```

```
let allNames =
RecipeDetailStore.shared.allRecipeNames()
```

```

// ✅ 핵심: 문자열 "처음부터" 정확 prefix 매칭
let matches = allNames.filter { name in
    let candidate = name
        .lowercased()
        .trimmingCharacters(in:
.whitespacesAndNewlines)

    return candidate.hasPrefix(qLower)
}

// ✅ 중복 제거 + 최대 5개
let uniq = Array(
    Dictionary(grouping: matches, by: {
$0.lowercased() })
        .values
        .compactMap { $0.first }
    )
    .prefix(5)

suggestions = Array(uniq)
}

private func ingredientCell(_ name: String) -> some
View {
    let isSelected = selectedIngredient == name
    let isCustom =
ingredientStore.isCustomIngredient(name) // ✅ 핵심

    return Button {
        selectedIngredient = isSelected ? nil : name
    } label: {
        HStack(spacing: 6) {

            // ✅ 사용자 생성 재료 표시
            if isCustom {
                Image(systemName: "person.fill")
                    .font(.caption2)
                    .foregroundColor(.orange)
            }

            Text(name)
                .font(.system(size: 14, weight: .semibold))
                .foregroundColor(
                    isSelected
                    ? .black
                    : isCustom

```

```

        ? .orange // ✅ 사용자 재료 텍스트 색
        : .white
    )

    Spacer()
}
.frame(maxWidth: .infinity, alignment: .leading)
.padding(.horizontal, 8)
.padding(.vertical, 12)
.background(
    isSelected
    ? Color.yellow.opacity(0.9)
    : isCustom
    ? Color.orange.opacity(0.18) // ✅ 사용자 재료
배경
    : Color.black.opacity(0.25)
)
.cornerRadius(10)
}
.buttonStyle(.plain)
}
}

// MARK: - Hide keyboard helper
private extension View {
    func hideKeyboard() {
        #if canImport(Uikit)

        UIApplication.shared.sendAction(#selector(UIResponder.
resignFirstResponder),
                                        to: nil, from: nil, for: nil)

        #endif
    }
}

```

Show less ^

첨부된 화면에서 Unit & Amount를 Amount & Unit으로 순서를 바꾸고 밑에 selection도 amount, unit 순으로 바꿔라.

첨부화면에서 existing ingredient에서 선택해서 추가된 (예를 들어 Prawn)은 "+ Add Ingredient" 버튼 아래에 리스팅되고 custom ingredient은 "+ Add Custom Ingredient"밑에 리스팅되게 해라.



첨부 화면에서 "+ Add Custom Ingredient"를 누르면 두번째 첨부 화면이 나오는데 여기서 이미 과거에 사용자가 추가한 fish를 선택하면 바로 추가되게 해라. 이기능은 existing recipe editor에서는 되는데 여기서 안된다.

"3 piece" 가 "My fish"앞에 있어야 한다 "3 piece My fish"

재료 설명글도 길면 "...하지말고 아랫줄로 내려서 다 보이게 해라.

```
//
// RecipeEditView.swift
// MenuCal
//
// Created by Dae-Kyoo Kim on 12/30/25.
//

import Foundation
import SwiftUI
struct ExistingRecipeEditView: View {

    let detail: RecipeDetail
    let caloriesByIngredient: [String: (kcal: Int, unit: String)]
    let totalCalories: Int

    @Environment(\.dismiss) var dismiss

    // 🖋 편집용 상태
    @State private var ingredients: [EditableIngredient]
    @State private var pendingCustomIngredient: String?
    = nil
    @State private var pendingAmount: Int = 1
    @State private var pendingUnit: String = "g"
    @State private var steps: [String]

    // ➕ Add bars
    @State private var showAddIngredient = false
    @State private var showAddStep = false

    // 현재 편집 중인 ingredient index
    @FocusState private var focusedIngredientIndex: Int?

    init(
        detail: RecipeDetail,
        caloriesByIngredient: [String: (kcal: Int, unit: String)],
        totalCalories: Int
    ) {
        self.detail = detail
        self.caloriesByIngredient = caloriesByIngredient
        self.totalCalories = totalCalories
    }
}
```

```
self.totalCalories = totalCalories
```

```
_ingredients = State(  
  initialValue: detail.ingredients.map {  
    EditableIngredient(  
      name: $0,  
      amount: nil,  
      unit: nil,  
      isCustom: false  
    )  
  }  
)  
_steps = State(initialValue: detail.steps)  
}
```

```
var body: some View {  
  ZStack {  
    // 🔥 EDIT MODE BACKGROUND (확실히 구분됨)  
    LinearGradient(  
      colors: [  
        Color.black,  
        Color.blue.opacity(0.25)  
      ],  
      startPoint: .top,  
      endPoint: .bottom  
    )  
    .ignoresSafeArea()  
  
    NavigationStack {  
      ScrollView {  
        VStack(alignment: .leading, spacing: 30) {  
  
          // HEADER  
          HStack(spacing: 12) {  
            Text(autoEmoji(for: detail.name))  
              .font(.system(size: 40))  
  
            Text(detail.name)  
              .font(.system(size: 26, weight: .bold,  
design: .rounded))  
              .foregroundColor(.white)  
  
            Spacer()  
  
            HStack(spacing: 4) {  
              Text("\(totalCalories)")
```

```

        .foregroundColor(.yellow)
        .font(.headline)
        .fontWeight(.bold)

        Text("kcal")

        .foregroundColor(.yellow.opacity(0.8))
        .font(.caption)

        Text("/ dish")

        .foregroundColor(.white.opacity(0.5))
        .font(.caption)
    }
}

CalorieProgressBar(calories: totalCalories)

ingredientsSection

if let pending = pendingCustomIngredient {
    VStack(alignment: .leading, spacing: 8) {
        Text("Amount & Unit · Custom
Ingredient")

        .foregroundColor(.orange)
        .font(.headline)

        HStack(spacing: 12) {
            Picker("Amount", selection:
$pendingAmount) {
                ForEach(1...30, id: \.self) { Text("\
($0)") }
            }
            .pickerStyle(.wheel)

            Picker("Unit", selection:
$pendingUnit) {
                ForEach(["g","ml","tbsp","tsp","cup","piece"], id: \.self) {
                    Text($0)
                }
            }
            .pickerStyle(.wheel)
        }
        .frame(height: 120)
    }
}

```

```

.background(Color.orange.opacity(0.15))
        .cornerRadius(12)

        Button {
            ingredients.append(
                EditableIngredient(
                    name: pending,
                    amount: pendingAmount,
                    unit: pendingUnit,
                    isCustom: true
                )
            )
            pendingCustomIngredient = nil
        } label: {
            Text("Add Custom Ingredient")
                .fontWeight(.bold)
                .frame(maxWidth: .infinity)
                .padding()
                .background(Color.orange)
                .foregroundColor(.black)
                .cornerRadius(12)
        }
    }
}

stepsSection

Spacer()
}
.padding(.horizontal, 35)
.padding(.top, 20)
}
.navigationTitle("Edit Recipe")
.navigationBarTitleDisplayMode(.inline)
.toolbar {
    ToolbarItem(placement: .topBarLeading) {
        Button("Cancel") { dismiss() }
    }

    ToolbarItem(placement: .topBarTrailing) {
        Button("Save") { dismiss() }
        .opacity(0.4)
    }
}
}
}
// 🔥 네비게이션 바 배경도 투명 처리

```

```

        .toolbarBackground(.hidden, for: .navigationBar)
    }
}

private var ingredientsSection: some View {
    VStack(alignment: .leading, spacing: 8) {

        Text("Ingredients")
            .font(.title2)
            .fontWeight(.semibold)

        ForEach(ingredients.indices, id: \.self) { i in
            let item = ingredients[i]

            VStack(spacing: 6) {
                HStack(alignment: .top) {

                    // 🡀 LEFT: name + meta (멀티라인)
                    VStack(alignment: .leading, spacing: 4) {

                        // 1 메인 라인: 수량 + 이름
                        let titleText: String = {
                            if item.isCustom,
                                let amount = item.amount,
                                let unit = item.unit {
                                    return "\((amount) \((unit) \
(item.name)"

                                } else {
                                    return item.name
                                }
                            }()

                        TextField(
                            "Ingredient",
                            text: Binding(
                                get: { titleText },
                                set: { newValue in
                                    if item.isCustom,
                                        let amount = item.amount,
                                        let unit = item.unit {
                                            let prefix = "\((amount) \((unit) "
                                            if newValue.hasPrefix(prefix) {
                                                ingredients[i].name =

String(newValue.dropFirst(prefix.count))
                                            }

```

```

        } else {
            ingredients[i].name = newValue
        }
    }
)
)
.foregroundColor(item.isCustom ?
.orange : .white)
.textFieldStyle(.plain)
.focused($focusedIngredientIndex,
equals: i)
.lineLimit(nil) // ✅ 여러 줄 허용
.fixedSize(horizontal: false, vertical: true)

// 2 서브 라인: 칼로리 / 기준 단위
if let cal =
caloriesByIngredient[item.name] {
    Text("\((cal.kcal) kcal / \((cal.unit)")
        .font(.caption)

.foregroundColor(.yellow.opacity(0.9))
        .lineLimit(1)
    }
}

Spacer(minLength: 8)

// ➡ DELETE
Button {
    ingredients.remove(at: i)
    if focusedIngredientIndex == i {
        focusedIngredientIndex = nil
    }
} label: {
    Image(systemName: "minus.circle.fill")
        .foregroundColor(.red)
        .font(.system(size: 18, weight:
.semibold))
    }
    .buttonStyle(.plain)
}
}
.padding(.vertical, 10)
.padding(.horizontal, 12)
.background(
    RoundedRectangle(cornerRadius: 10)

```

```

        .fill(
            focusedIngredientIndex == i
            ? Color.blue.opacity(0.35)
            : Color.white.opacity(0.06)
        )
    )
)
.overlay(
    RoundedRectangle(cornerRadius: 10)
        .stroke(
            focusedIngredientIndex == i
            ? Color.blue.opacity(0.9)
            : Color.clear,
            lineWidth: 1.5
        )
    )
)
}

Button {
    showAddIngredient = true
} label: {
    HStack {
        Image(systemName: "plus.circle")
        Text("Add Ingredient")
    }
    .foregroundColor(.white.opacity(0.7))
}
// 🔥 여기 추가
.contentShape(Rectangle()) // 빈 공간도 터치 가능
하게
.onTapGesture {
    focusedIngredientIndex = nil // ✅ 선택 해제
}
.sheet(isPresented: $showAddIngredient) {
    AddIngredientBar(store: IngredientStore.shared) {
newIngredient in
        pendingCustomIngredient = newIngredient
        pendingAmount = 1
        pendingUnit = "g"
    }
}
}

private var stepsSection: some View {
    VStack(alignment: .leading, spacing: 8) {

        Text("Steps")
    }
}

```



```

        .font(.title2)
        .fontWeight(.semibold)

    if steps.isEmpty {
        Button {
            showAddStep = true
        } label: {
            HStack {
                Image(systemName: "plus.circle")
                Text("Add Steps")
            }
            .foregroundColor(.white.opacity(0.7))
        }
    } else {
        ForEach(steps.indices, id: \.self) { i in
            TextEditor(text: $steps[i])
                .frame(minHeight: 60)
                .foregroundColor(.white)
                .background(Color.white.opacity(0.08))
                .cornerRadius(8)
        }

        Button {
            showAddStep = true
        } label: {
            HStack {
                Image(systemName: "plus.circle")
                Text("Add Step")
            }
            .foregroundColor(.white.opacity(0.7))
        }
    }
}

.sheet(isPresented: $showAddStep) {
    VStack {
        Text("Add Step (Skeleton)")
        Button("Add Empty Step") {
            steps.append("")
            showAddStep = false
        }
    }
    .padding()
}
}
}

```

Show less ^

수종후 재료별 칼로리가 나오지 않는다. caloriesByIngredient는
기존 재료에대해서 그대로 쓰고 사용자가 새로 추가하는 사용자 재
료에 대해서만 DB에서 찾는다

DB에 fish에 관한 ingredient가 430개나 되는데 왜 my fish에 대해서 칼로라/단위가 하나도 안나오?

```
//  
// RecipeEditView.swift  
// MenuCal  
//  
// Created by Dae-Kyoo Kim on 12/30/25.  
//  
  
import Foundation  
import SwiftUI  
struct ExistingRecipeEditView: View {  
  
    let detail: RecipeDetail  
    let caloriesByIngredient: [String: (kcal: Int, unit: String)]  
    let totalCalories: Int  
  
    @Environment(\.dismiss) var dismiss  
  
    // 🖋 편집용 상태  
    @State private var ingredients: [EditableIngredient]  
    @State private var pendingCustomIngredient: String?  
    = nil  
    @State private var pendingAmount: Int = 1  
    @State private var pendingUnit: String = "g"  
    @State private var steps: [String]
```

```

// + Add bars
@State private var showAddIngredient = false
@State private var showAddStep = false

// 현재 편집 중인 ingredient index
@FocusState private var focusedIngredientIndex: Int?
private let calorieDB = MenuParser.shared

init(
    detail: RecipeDetail,
    caloriesByIngredient: [String: (kcal: Int, unit: String)],
    totalCalories: Int
) {
    self.detail = detail
    self.caloriesByIngredient = caloriesByIngredient
    self.totalCalories = totalCalories

    _ingredients = State(
        initialValue: detail.ingredients.map {
            EditableIngredient(
                name: $0,
                amount: nil,
                unit: nil,
                isCustom: false
            )
        }
    )
    _steps = State(initialValue: detail.steps)
}

var body: some View {
    ZStack {
        // 🔥 EDIT MODE BACKGROUND (확실히 구분됨)
        LinearGradient(
            colors: [
                Color.black,
                Color.blue.opacity(0.25)
            ],
            startPoint: .top,
            endPoint: .bottom
        )
        .ignoresSafeArea()

        NavigationStack {
            ScrollView {

```

```

VStack(alignment: .leading, spacing: 30) {

    // HEADER
    HStack(spacing: 12) {
        Text(autoEmoji(for: detail.name))
            .font(.system(size: 40))

        Text(detail.name)
            .font(.system(size: 26, weight: .bold,
design: .rounded))
            .foregroundColor(.white)

        Spacer()

        HStack(spacing: 4) {
            Text("\$(totalCalories)")
                .foregroundColor(.yellow)
                .font(.headline)
                .fontWeight(.bold)

            Text("kcal")

            .foregroundColor(.yellow.opacity(0.8))
                .font(.caption)

            Text("/ dish")

            .foregroundColor(.white.opacity(0.5))
                .font(.caption)
        }
    }

    CalorieProgressBar(calories: totalCalories)

    ingredientsSection

    if let pending = pendingCustomIngredient {
        VStack(alignment: .leading, spacing: 8) {
            Text("Amount & Unit · Custom
Ingredient")

                .foregroundColor(.orange)
                .font(.headline)

            HStack(spacing: 12) {
                Picker("Amount", selection:
$pendingAmount) {

```



```

        ForEach(1...30, id: \.self) { Text("\
($0)") }

    }

    .pickerStyle(.wheel)

    Picker("Unit", selection:
$pendingUnit) {

    ForEach(["g","ml","tbsp","tsp","cup","piece"], id: \.self) {
        Text($0)
    }
    }
    .pickerStyle(.wheel)
}
.frame(height: 120)

.background(Color.orange.opacity(0.15))
    .cornerRadius(12)

    Button {
        let callInfo = lookupCalories(for:
pending)

        ingredients.append(
            EditableIngredient(
                name: pending,
                amount: pendingAmount,
                unit: pendingUnit,
                isCustom: true,
                kcal: callInfo?.kcal,
                kcalUnit: callInfo?.unit
            )
        )

        pendingCustomIngredient = nil
    } label: {
        Text("Add Custom Ingredient")
        .fontWeight(.bold)
        .frame(maxWidth: .infinity)
        .padding()
        .background(Color.orange)
        .foregroundColor(.black)
        .cornerRadius(12)
    }
}
}
}

```

```

stepsSection

    Spacer()
}
.padding(.horizontal, 35)
.padding(.top, 20)
}
.navigationTitle("Edit Recipe")
.navigationBarTitleDisplayMode(.inline)
.toolbar {
    ToolbarItem(placement: .topBarLeading) {
        Button("Cancel") { dismiss() }
    }

    ToolbarItem(placement: .topBarTrailing) {
        Button("Save") { dismiss() }
        .opacity(0.4)
    }
}
}
// 🔥 네비게이션 바 배경도 투명 처리
.toolbarBackground(.hidden, for: .navigationBar)
}
}

```

```

private var ingredientsSection: some View {
    VStack(alignment: .leading, spacing: 8) {

        Text("Ingredients")
        .font(.title2)
        .fontWeight(.semibold)

        ForEach(ingredients.indices, id: \.self) { i in
            let item = ingredients[i]
            let isFocused = focusedIngredientIndex == i

            VStack(alignment: .leading, spacing: 6) {

                HStack(alignment: .top, spacing: 8) {

                    VStack(alignment: .leading, spacing: 4) {

                        // 1 메인 텍스트 (멀티라인 표시)
                        let displayText: String = {
                            if item.isCustom,

```

```

        let amount = item.amount,
        let unit = item.unit {
            return "\\(amount) \\(unit) \
(item.name)"
        } else {
            return item.name
        }
    }()

    if isFocused {
        // 🖋 편집 모드
        TextEditor(
            text: Binding(
                get: { item.name },
                set: { ingredients[i].name = $0 }
            )
        )
        .foregroundColor(item.isCustom ?
.orange : .white)
        .frame(minHeight: 40)
        .background(Color.clear)
    } else {
        // 👁 표시 모드 (여기서 줄바꿈 허용)
        Text(displayText)
        .foregroundColor(item.isCustom ?
.orange : .white)
        .font(.body)
        .multilineTextAlignment(.leading)
        .fixedSize(horizontal: false, vertical:
true)
        .onTapGesture {
            focusedIngredientIndex = i
        }
    }

    // 📋 2 칼로리 / 기준 단위 (항상 다음 줄)
    if let cal = calorieInfo(for: item) {
        Text("\\(cal.kcal) kcal / \\(cal.unit)")
        .font(.caption)

        .foregroundColor(.yellow.opacity(0.9))
    }
}

Spacer()

```

```

// ❌ 삭제 버튼
Button {
    ingredients.remove(at: i)
    focusedIngredientIndex = nil
} label: {
    Image(systemName: "minus.circle.fill")
    .foregroundColor(.red)
    .font(.system(size: 18, weight:
.semibold))
}
.buttonStyle(.plain)
}
}
.padding(.vertical, 10)
.padding(.horizontal, 12)
.background(
    RoundedRectangle(cornerRadius: 10)
    .fill(isFocused ? Color.blue.opacity(0.35)
        : Color.white.opacity(0.06))
)
.overlay(
    RoundedRectangle(cornerRadius: 10)
    .stroke(isFocused ? Color.blue.opacity(0.9)
        : Color.clear,
        lineWidth: 1.5)
)
}

Button {
    showAddIngredient = true
} label: {
    HStack {
        Image(systemName: "plus.circle")
        Text("Add Ingredient")
    }
    .foregroundColor(.white.opacity(0.7))
}
}
// 🔥 여기 추가
.contentShape(Rectangle()) // 빈 공간도 터치 가능
하게
.onTapGesture {
    focusedIngredientIndex = nil // ✅ 선택 해제
}
.sheet(isPresented: $showAddIngredient) {
    AddIngredientBar(store: IngredientStore.shared) {

```

```

newIngredient in
    pendingCustomIngredient = newIngredient
    pendingAmount = 1
    pendingUnit = "g"
}
}
}
private var stepsSection: some View {
    VStack(alignment: .leading, spacing: 8) {

        Text("Steps")
            .font(.title2)
            .fontWeight(.semibold)

        if steps.isEmpty {
            Button {
                showAddStep = true
            } label: {
                HStack {
                    Image(systemName: "plus.circle")
                    Text("Add Steps")
                }
                .foregroundColor(.white.opacity(0.7))
            }
        } else {
            ForEach(steps.indices, id: \.self) { i in
                TextEditor(text: $steps[i])
                    .frame(minHeight: 60)
                    .foregroundColor(.white)
                    .background(Color.white.opacity(0.08))
                    .cornerRadius(8)
            }

            Button {
                showAddStep = true
            } label: {
                HStack {
                    Image(systemName: "plus.circle")
                    Text("Add Step")
                }
                .foregroundColor(.white.opacity(0.7))
            }
        }
    }
}
.sheet(isPresented: $showAddStep) {
    VStack {

```

```

        Text("Add Step (Skeleton)")
        Button("Add Empty Step") {
            steps.append("")
            showAddStep = false
        }
    }
    .padding()
}
}

```

```

private fun lookupCalories(for name: String)
-> (kcal: Int, unit: String)? {

```

```

    guard let info =
MenuParser.shared.getIngredientInfo(for: name) else {
        return nil
    }

    return (
        kcal: Int(info.calories),
        unit: info.unit
    )
}

```

```

private fun calorieInfo(for item: EditableIngredient)
-> (kcal: Int, unit: String)? {

```

```

    // ✅ 1. 기존 레시피 재료 → 기존 map 사용
    if !item.isCustom {
        return caloriesByIngredient[item.name]
    }

```

```

    // ✅ 2. 사용자 추가 재료 → ingredients.json lookup
    if let info = MenuParser.shared.getIngredientInfo(for:
item.name) {
        return (
            kcal: Int(info.calories),
            unit: info.unit
        )
    }

    return nil
}
}

```


lookupcalorieforedit을 넣었음에도 my fish에 대한 칼로리/단위가 나오지 않는다 DB 430개나 있는데. 확인해라.

```
//  
// RecipeEditView.swift  
// MenuCal  
//  
// Created by Dae-Kyoo Kim on 12/30/25.  
//  
  
import Foundation  
import SwiftUI  
struct ExistingRecipeEditView: View {  
  
    let detail: RecipeDetail  
    let caloriesByIngredient: [String: (kcal: Int, unit: String)]  
    let totalCalories: Int
```



```
@Environment(\.dismiss) var dismiss
```

```
// 🖋 편집용 상태
```

```
@State private var ingredients: [EditableIngredient]
```

```
@State private var pendingCustomIngredient: String?
```

```
= nil
```

```
@State private var pendingAmount: Int = 1
```

```
@State private var pendingUnit: String = "g"
```

```
@State private var steps: [String]
```

```
// ➕ Add bars
```

```
@State private var showAddIngredient = false
```

```
@State private var showAddStep = false
```

```
// 현재 편집 중인 ingredient index
```

```
@FocusState private var focusedIngredientIndex: Int?
```

```
private let calorieDB = MenuParser.shared
```

```
init(
```

```
    detail: RecipeDetail,
```

```
    caloriesByIngredient: [String: (kcal: Int, unit: String)],
```

```
    totalCalories: Int
```

```
) {
```

```
    self.detail = detail
```

```
    self.caloriesByIngredient = caloriesByIngredient
```

```
    self.totalCalories = totalCalories
```

```
    _ingredients = State(
```

```
        initialValue: detail.ingredients.map {
```

```
            EditableIngredient(
```

```
                name: $0,
```

```
                amount: nil,
```

```
                unit: nil,
```

```
                isCustom: false
```

```
            )
```

```
        }
```

```
    )
```

```
    _steps = State(initialValue: detail.steps)
```

```
}
```

```
var body: some View {
```

```
    ZStack {
```

```
        // 🔥 EDIT MODE BACKGROUND (확실히 구분됨)
```

```
        LinearGradient(
```

```
            colors: [
```

```
                Color.black,
```

```

        Color.blue.opacity(0.25)
    ],
    startPoint: .top,
    endPoint: .bottom
)
.ignoresSafeArea()

NavigationStack {
    ScrollView {
        VStack(alignment: .leading, spacing: 30) {

            // HEADER
            HStack(spacing: 12) {
                Text(autoEmoji(for: detail.name))
                    .font(.system(size: 40))

                Text(detail.name)
                    .font(.system(size: 26, weight: .bold,
design: .rounded))
                    .foregroundColor(.white)

                Spacer()

                HStack(spacing: 4) {
                    Text("\(totalCalories)")
                        .foregroundColor(.yellow)
                        .font(.headline)
                        .fontWeight(.bold)

                    Text("kcal")

                    .foregroundColor(.yellow.opacity(0.8))
                        .font(.caption)

                    Text("/ dish")

                    .foregroundColor(.white.opacity(0.5))
                        .font(.caption)
                }
            }

            CalorieProgressBar(calories: totalCalories)

            ingredientsSection

            if let pending = pendingCustomIngredient {

```

```

VStack(alignment: .leading, spacing: 8) {
    Text("Amount & Unit · Custom
Ingredient")

    .foregroundColor(.orange)
    .font(.headline)

    HStack(spacing: 12) {
        Picker("Amount", selection:
$pendingAmount) {
            ForEach(1...30, id: \.self) { Text("\
($0)") }
        }
        .pickerStyle(.wheel)

        Picker("Unit", selection:
$pendingUnit) {
            ForEach(["g","ml","tbsp","tsp","cup","piece"], id: \.self) {
                Text($0)
            }
        }
        .pickerStyle(.wheel)
    }
    .frame(height: 120)

    .background(Color.orange.opacity(0.15))
    .cornerRadius(12)

    Button {
        let callInfo =
lookupIngredientForEdit(pending)

        ingredients.append(
            EditableIngredient(
                name: pending,
                amount: pendingAmount,
                unit: pendingUnit,
                isCustom: true,
                kcal: callInfo.map {
Int($0.calories) },
                kcalUnit: callInfo?.unit
            )
        )

        pendingCustomIngredient = nil
    } label: {

```

```

        Text("Add Custom Ingredient")
            .fontWeight(.bold)
            .frame(maxWidth: .infinity)
            .padding()
            .background(Color.orange)
            .foregroundColor(.black)
            .cornerRadius(12)
    }
}
}

stepsSection

Spacer()
}
.padding(.horizontal, 35)
.padding(.top, 20)
}
.navigationTitle("Edit Recipe")
.navigationBarTitleDisplayMode(.inline)
.toolbar {
    ToolbarItem(placement: .topBarLeading) {
        Button("Cancel") { dismiss() }
    }

    ToolbarItem(placement: .topBarTrailing) {
        Button("Save") { dismiss() }
        .opacity(0.4)
    }
}
}
// 🔥 네비게이션 바 배경도 투명 처리
.toolbarBackground(.hidden, for: .navigationBar)
}
}

```

```

private var ingredientsSection: some View {
    VStack(alignment: .leading, spacing: 8) {

        Text("Ingredients")
            .font(.title2)
            .fontWeight(.semibold)

        ForEach(ingredients.indices, id: \.self) { i in
            let item = ingredients[i]
            let isFocused = focusedIngredientIndex == i

```

```
VStack(alignment: .leading, spacing: 6) {
```

```
    HStack(alignment: .top, spacing: 8) {
```

```
        VStack(alignment: .leading, spacing: 4) {
```

```
            // 1 메인 텍스트 (멀티라인 표시)
```

```
            let displayText: String = {
```

```
                if item.isCustom,
```

```
                    let amount = item.amount,
```

```
                    let unit = item.unit {
```

```
                        return "\\(amount) \\(unit) \
```

```
(item.name)"
```

```
                    } else {
```

```
                        return item.name
```

```
                    }
```

```
            }()
```

```
            if isFocused {
```

```
                // ✎ 편집 모드
```

```
                TextEditor(
```

```
                    text: Binding(
```

```
                        get: { item.name },
```

```
                        set: { ingredients[i].name = $0 }
```

```
                    )
```

```
                )
```

```
                .foregroundColor(item.isCustom ?
```

```
.orange : .white)
```

```
                .frame(minHeight: 40)
```

```
                .background(Color.clear)
```

```
            } else {
```

```
                // 👁 표시 모드 (여기서 줄바꿈 허용)
```

```
                Text(displayText)
```

```
                    .foregroundColor(item.isCustom ?
```

```
.orange : .white)
```

```
                    .font(.body)
```

```
                    .multilineTextAlignment(.leading)
```

```
                    .fixedSize(horizontal: false, vertical:
```

```
true)
```

```
                    .onTapGesture {
```

```
                        focusedIngredientIndex = i
```

```
                    }
```

```
            }
```

```
            // 2 칼로리 / 기준 단위 (항상 다음 줄)
```

```

        if let cal = calorieInfo(for: item) {
            Text("\((cal.kcal) kcal / \((cal.unit)")
                .font(.caption)

.foregroundColor(.yellow.opacity(0.9))
        }
    }

    Spacer()

    // ❌ 삭제 버튼
    Button {
        ingredients.remove(at: i)
        focusedIngredientIndex = nil
    } label: {
        Image(systemName: "minus.circle.fill")
            .foregroundColor(.red)
            .font(.system(size: 18, weight:
.semibold))
    }
        .buttonStyle(.plain)
    }
    }
    .padding(.vertical, 10)
    .padding(.horizontal, 12)
    .background(
        RoundedRectangle(cornerRadius: 10)
            .fill(isFocused ? Color.blue.opacity(0.35)
                : Color.white.opacity(0.06))
    )
    .overlay(
        RoundedRectangle(cornerRadius: 10)
            .stroke(isFocused ? Color.blue.opacity(0.9)
                : Color.clear,
                lineWidth: 1.5)
    )
}

Button {
    showAddIngredient = true
} label: {
    HStack {
        Image(systemName: "plus.circle")
        Text("Add Ingredient")
    }
    .foregroundColor(.white.opacity(0.7))
}

```

```

    }
  }
  // 🔥 여기 추가
  .contentShape(Rectangle()) // 빈 공간도 터치 가능
  하게
  .onTapGesture {
    focusedIngredientIndex = nil // ✅ 선택 해제
  }
  .sheet(isPresented: $showAddIngredient) {
    AddIngredientBar(store: IngredientStore.shared) {
newIngredient in
      pendingCustomIngredient = newIngredient
      pendingAmount = 1
      pendingUnit = "g"
    }
  }
}

private var stepsSection: some View {
  VStack(alignment: .leading, spacing: 8) {

    Text("Steps")
      .font(.title2)
      .fontWeight(.semibold)

    if steps.isEmpty {
      Button {
        showAddStep = true
      } label: {
        HStack {
          Image(systemName: "plus.circle")
          Text("Add Steps")
        }
        .foregroundColor(.white.opacity(0.7))
      }
    } else {
      ForEach(steps.indices, id: \.self) { i in
        TextEditor(text: $steps[i])
          .frame(minHeight: 60)
          .foregroundColor(.white)
          .background(Color.white.opacity(0.08))
          .cornerRadius(8)
      }

      Button {
        showAddStep = true
      } label: {

```

```

        HStack {
            Image(systemName: "plus.circle")
            Text("Add Step")
        }
        .foregroundColor(.white.opacity(0.7))
    }
}
}
.sheet(isPresented: $showAddStep) {
    VStack {
        Text("Add Step (Skeleton)")
        Button("Add Empty Step") {
            steps.append("")
            showAddStep = false
        }
    }
    .padding()
}
}

```

```

private fun calorieInfo(for item: EditableIngredient)
-> (kcal: Int, unit: String)? {

```

```

    // ✅ 1. 기존 레시피 재료 → 기존 map 사용
    if !item.isCustom {
        return caloriesByIngredient[item.name]
    }

```

```

    // ✅ 2. 사용자 추가 재료 → ingredients.json lookup
    if let info = lookupIngredientForEdit(item.name) {
        return (Int(info.calories), info.unit)
    }

```

```

    return nil
}

```

```

private fun normalizeIngredientName(_ name: String)
-> [String] {

```

```

    fun normalize(_ s: String) -> String {
        s.lowercased()
            .replacingOccurrences(of: "[^a-z ]",
                                with: " ",
                                options: .regularExpression)
            .replacingOccurrences(of: "\\s+",
                                with: " ",

```



```

        options: .regularExpression)
        .trimmingCharacters(in: .whitespaces)
    }

    let cleaned = normalize(name)
    let words = cleaned.split(separator: "
").map(String.init)

    // ◆ noise 제거 (parseIngredientsOptimized와 동일 기준)
    let filtered = words.filter {
        !MenuParser.shared.isGenericNoise($0)
    }

    // ◆ 단수화 보정 (fish / fishes, egg / eggs)
    let singularized = filtered.map {
        $0.hasSuffix("s") ? String($0.dropLast()) : $0
    }

    return singularized
}

func lookupIngredientForEdit(_ input: String) -> IngredientInfo? {

    let tokens = normalizeIngredientName(input)
    guard !tokens.isEmpty else { return nil }

    let db = MenuParser.shared.calorieDB

    // ◆ 1. exact match (two-word 우선)
    for i in 0..

```

```
        return nil
    }

}
```

Show less ^

My recipe create에서도 추가 되는 재료에 대해서 같은 방식으로
칼로리/단위를 재료이름 밑에 추가 해라.

```
//  
// MyRecipeEditorView.swift  
// MenuCal  
//  
// Created by Dae-Kyoo Kim on 12/19/25.  
//
```

```

import SwiftUI
import Foundation //✅ 이 줄 추가

struct MyRecipeEditorView: View {

    enum Mode: Identifiable {
        case create
        case edit(UserRecipe)

        var id: String {
            switch self {
                case .create: return "create"
                case .edit(let r): return r.id.uuidString
            }
        }
    }

    let mode: Mode
    @Environment(\.dismiss) private var dismiss
    @State private var isTyping = false

    @StateObject private var store =
    UserRecipeStore.shared
    @State private var isShowingAddCustomIngredient =
    false

    // ---- name + suggestions
    @State private var recipeName: String = ""
    @State private var suggestions: [String] = []
    @State private var isSuggesting = false
    @FocusState private var isNameFocused: Bool

    private let units = ["g", "ml", "cup", "tbsp", "tsp", "piece"]

    @State private var selectedIngredient: String? = nil
    @State private var selectedUnit: String = "g"
    @State private var selectedAmount: Int = 1

    // ---- current recipe ingredients
    @State private var pickedIngredients: [UserIngredient]
= []
    @EnvironmentObject var ingredientStore:
    IngredientStore

    // 카테고리 expand / collapse 상태

```

```

@State private var expandedCategories:
Set<IngredientStore.IngredientCategory> = []

// ---- loading
@State private var isLoadingDB = true

@State private var isSelectingCustomIngredient = false

private var pickedExistingIngredients: [UserIngredient]
{
    pickedIngredients.filter {
        !ingredientStore.isCustomIngredient($0.name)
    }
}

private var pickedCustomIngredients: [UserIngredient]
{
    pickedIngredients.filter {
        ingredientStore.isCustomIngredient($0.name)
    }
}

var body: some View {
    ScrollView {
        VStack(alignment: .leading, spacing: 18) {

            if isLoadingDB {
                ProgressView()
                .padding(.top, 40)
                .frame(maxWidth: .infinity)
            } else {
                nameSection
                suggestionDropdown
                ingredientGridSection
                unitAmountSection
                addIngredientButton

                // =====
                // Existing Ingredients Added
                // =====
                if !pickedExistingIngredients.isEmpty {
                    VStack(alignment: .leading, spacing: 8) {

                        Text("Ingredients Added")
                            .foregroundColor(.white)
                            .font(.system(size: 16, weight: .bold))
                    }
                }
            }
        }
    }
}

```

```

ForEach(pickedExistingIngredients.indices, id: \.self) { idx
in
    let item =
pickedExistingIngredients[idx]

    HStack {
        Text("• \$(item.name)")
            .foregroundColor(.white)

        Spacer()

        Text("\$(item.amount) \$(item.unit)")

    }.foregroundColor(.white.opacity(0.8))

    Button {
        pickedIngredients.removeAll {
$0.id == item.id }
    } label: {
        Image(systemName: "trash")

    }.foregroundColor(.red.opacity(0.9))
    }
    .buttonStyle(.plain)
    }
    .padding(12)

    .background(Color.black.opacity(0.25))
    .cornerRadius(10)
    }
    }
}

Text("Can't find it?")
    .font(.footnote)
    .foregroundColor(.white.opacity(0.6))
    .frame(maxWidth: .infinity, alignment:
.center)

addCustomIngredientButton

// =====
// Custom Ingredients Added
// =====

```



```

        if !pickedCustomIngredients.isEmpty {
            VStack(alignment: .leading, spacing: 8) {

                Text("Custom Ingredients Added")
                    .foregroundColor(.white)
                    .font(.system(size: 16, weight: .bold))

                ForEach(pickedCustomIngredients.indices, id: \.self) { idx
                in
                    let item =
                    pickedCustomIngredients[idx]

                    HStack {
                        Image(systemName: "person.fill")
                            .foregroundColor(.orange)
                            .font(.caption)

                        Text(item.name)
                            .foregroundColor(.orange)

                        Spacer()

                        Text("\$(item.amount) \$(item.unit)")

                    }
                    .foregroundColor(.white.opacity(0.8))

                    Button {
                        pickedIngredients.removeAll {
                            $0.id == item.id }
                    } label: {
                        Image(systemName: "trash")

                    }
                    .foregroundColor(.red.opacity(0.9))
                    }
                    .buttonStyle(.plain)
                    }
                    .padding(12)

                .background(Color.orange.opacity(0.15))
                .cornerRadius(10)
            }
        }
    }
    saveButton
}

```

```

    }
    .padding()
}
// ✅🔥 핵심: ScrollView 바깥 배경
.navigationBarTitle(modeTitle)
.navigationBarTitleDisplayMode(.inline)
.toolbar {

    // Cancel (항상 표시)
    ToolbarItem(placement: .topBarLeading) {
        Button("Cancel") {
            dismiss()
        }
    }

    // 🔥 Delete (Edit 모드에서만)
    ToolbarItem(placement: .topBarTrailing) {
        if case .edit(let recipe) = mode {
            Button(role: .destructive) {
                store.delete(recipe)
                dismiss()
            } label: {
                Image(systemName: "trash")
            }
        }
    }
}

.sheet(isPresented:
$isShowingAddCustomIngredient) {
    AddIngredientBar(store: ingredientStore) {
newIngredient in
        selectedIngredient = newIngredient
        isSelectingCustomIngredient = true
    }
}

.task {
    await RecipeDetailStore.shared.loadIfNeeded()
    isLoadingDB = false

    switch mode {
    case .create:
        break
    case .edit(let r):
        recipeName = r.name
        pickedIngredients = r.ingredients
    }
}

```

```

    }
    .onChange(of: recipeName) { text in
        Task { await updateSuggestions(for: text) }
    }
    .onTapGesture {
        endTyping()
    }
}

private func endTyping() {
    isTyping = false
    suggestions = []
    isSuggesting = false
    isNameFocused = false // 🔥 키보드 내려감
}

private var modeTitle: String {
    switch mode {
    case .create: return "Create"
    case .edit: return "Edit"
    }
}

// MARK: - Sections

private var nameSection: some View {
    VStack(alignment: .leading, spacing: 8) {
        Text("Recipe Name")
            .foregroundColor(.white)
            .font(.system(size: 16, weight: .bold))

        TextField("Type recipe name...", text:
$recipeName)
            .textInputAutocapitalization(.words)
            .padding(12)
            .background(Color.black.opacity(0.7))
            .foregroundColor(.white)
            .cornerRadius(10)
            .focused($isNameFocused)
            .onChange(of: recipeName) { _ in
                isTyping = true
            }
        .onSubmit {
            // ✅ 타이핑 종료
            isTyping = false
            suggestions = []
        }
    }
}

```

```

        isSuggesting = false
    }
}

private var suggestionDropdown: some View {
    Group {
        if isTyping && !suggestions.isEmpty {
            VStack(alignment: .leading, spacing: 0) {
                ForEach(suggestions.indices, id: \.self) { i in
                    let s = suggestions[i]
                    Button {
                        recipeName = s
                        endTyping()
                        hideKeyboard()
                    } label: {
                        HStack {
                            Text(s)
                                .foregroundColor(.white)
                                .lineLimit(1)
                            Spacer()
                        }
                        .padding(.vertical, 10)
                        .padding(.horizontal, 12)
                        .background(Color.black.opacity(0.22))
                    }
                    .buttonStyle(.plain)

                    if i != suggestions.count - 1 {
                        Rectangle().fill(Color.white.opacity(0.12)).frame(height: 1)
                    }
                }
            }
            .cornerRadius(10)
        }
    }
}

private var ingredientGridSection: some View {
    VStack(alignment: .leading, spacing: 10) {

        Text("Select Ingredient (single)")
            .foregroundColor(.white)
            .font(.system(size: 16, weight: .bold))
    }
}

```

```

ScrollView {
    VStack(alignment: .leading, spacing: 20) {

        ForEach(ingredientStore.categorizedIngredients, id:
        \.category) { section in
            let isExpanded =
            expandedCategories.contains(section.category)

            VStack(alignment: .leading, spacing: 8) {

                // ✅ CATEGORY HEADER (탭 가능)
                HStack {
                    Text(section.category.rawValue)
                        .font(.system(size: 17, weight:
                        .semibold))

                    .foregroundColor(Color.mint.opacity(0.95))

                    Spacer()

                    Image(systemName: "chevron.down")

                    .foregroundColor(.white.opacity(0.6))
                        .rotationEffect(.degrees(isExpanded
                        ? 180 : 0))
                        .animation(.easeInOut(duration:
                        0.2), value: isExpanded)
                }
                .padding(.vertical, 8)
                .padding(.horizontal, 6)
                .contentShape(Rectangle()) // 🔥 전체 탭
영역

                .onTapGesture {
                    if isExpanded {

                        expandedCategories.remove(section.category)
                    } else {

                        expandedCategories.insert(section.category)
                    }
                }

                // divider
                Rectangle()
                    .fill(Color.white.opacity(0.15))

```

```

        .frame(height: 1)

        // ✅ EXPANDED CONTENT
        if isExpanded {
            LazyVGrid(
                columns: Array(
                    repeating: GridItem(.flexible()),
                    spacing: 10, alignment: .leading),
                count: 3
            ),
            spacing: 10
        ) {
            ForEach(section.items.indices, id:
\self) { index in

ingredientCell(section.items[index])
                }
            }
            .padding(.top, 6)
            .transition(.opacity.combined(with:
.move(edge: .top)))
        }
    }
}

.padding(.vertical, 4)
}
.frame(maxHeight: 320)
.background(Color.black.opacity(0.18))
.cornerRadius(12)
}
}

private var unitAmountSection: some View {
    Group {
        if let ing = selectedIngredient {
            VStack(alignment: .leading, spacing: 8) {

                // 🔥 custom ingredient일 때만 강조
                if isSelectingCustomIngredient {
                    Text("Amount & Unit · \(ing)")
                        .foregroundColor(.orange)
                        .font(.system(size: 16, weight: .bold))
                } else {
                    Text("Amount & Unit")
                        .foregroundColor(.white)
                }
            }
        }
    }
}

```

```

        .font(.system(size: 16, weight: .bold))
    }

    HStack(spacing: 12) {

        Picker("Amount", selection:
$selectedAmount) {
            ForEach(1...30, id: \.self) { Text("\($0)") }
        }
        .pickerStyle(.wheel)

        Picker("Unit", selection: $selectedUnit) {
            ForEach(units, id: \.self) { Text($0) }
        }
        .pickerStyle(.wheel)
    }
    .frame(minHeight: 120)
    .background(
        isSelectingCustomIngredient
        ? Color.orange.opacity(0.18)
        : Color.black.opacity(0.18)
    )
    .cornerRadius(12)
}
}
}
}
}

```

```

private var addCustomIngredientButton: some View {
    Button {
        // 기존 AddIngredientBar가 하던 동작으로 연결
        isShowingAddCustomIngredient = true
    } label: {
        HStack {
            Image(systemName: "plus.circle.fill")
            Text("Add Custom Ingredient")
                .fontWeight(.semibold)
        }
        .foregroundColor(.black)
        .frame(maxWidth: .infinity)
        .padding(.vertical, 14)
        .background(
            LinearGradient(
                colors: [Color.green.opacity(0.85),
Color.green],
                startPoint: .leading,

```

```

        endPoint: .trailing
    )
)
.cornerRadius(14)
}
}

private var addIngredientButton: some View {
    Button {
        guard let ing = selectedIngredient else { return }

        // 같은 재료를 여러 번 허용할지 정책 필요:
        // 지금은 "허용" (원하면 중복 방지로 바꿔줄게)
        pickedIngredients.append(
            UserIngredient(name: ing, unit: selectedUnit,
amount: selectedAmount)
        )

        // ✅ single selection reset
        selectedIngredient = nil
        selectedAmount = 1
        selectedUnit = "g"
    } label: {
        HStack {
            Image(systemName: "plus")
            Text("Add Ingredient")
                .fontWeight(.bold)
        }
        .foregroundColor(.black)
        .frame(maxWidth: .infinity)
        .padding(.vertical, 12)
        .background(
            (selectedIngredient == nil) ?
Color.gray.opacity(0.35) : Color.yellow.opacity(0.95)
        )
        .cornerRadius(14)
    }
    .disabled(selectedIngredient == nil)
}

private var pickedListSection: some View {
    VStack(alignment: .leading, spacing: 10) {
        Text("Ingredients Added")
            .foregroundColor(.white)
            .font(.system(size: 16, weight: .bold))
    }
}

```



```

        if pickedIngredients.isEmpty {
            Text("No ingredients yet.")
                .foregroundColor(.white.opacity(0.7))
                .font(.system(size: 14))
        } else {
            VStack(spacing: 8) {
                ForEach(pickedIngredients.indices, id: \.self) {
idx in
                    let item = pickedIngredients[idx]
                    HStack {
                        Text("• \${item.name}")
                            .foregroundColor(.white)
                        Spacer()
                        Text("\${item.amount} \${item.unit}")
                            .foregroundColor(.white.opacity(0.8))
                        Button {
                            pickedIngredients.remove(at: idx)
                        } label: {
                            Image(systemName: "trash")
                                .foregroundColor(.red.opacity(0.9))
                        }
                            .buttonStyle(.plain)
                    }
                        .padding(12)
                        .background(Color.black.opacity(0.25))
                        .cornerRadius(10)
                }
            }
        }
    }
}

```

```

private var saveButton: some View {
    Button {
        let finalName = resolvedRecipeName()
        guard !pickedIngredients.isEmpty else { return }

        switch mode {
        case .create:
            let new = UserRecipe(
                name: finalName,
                ingredients: pickedIngredients
            )
            store.add(new)

        case .edit(let old):

```

```

        var edited = old
        edited.name = finalName
        edited.ingredients = pickedIngredients
        edited.updatedAt = Date()
        store.update(edited)
    }

    dismiss()

} label: {
    HStack {
        Image(systemName: "checkmark.circle.fill")
        Text("Save")
            .fontWeight(.bold)
    }
    .foregroundColor(.black)
    .frame(maxWidth: .infinity)
    .padding(.vertical, 14)
    .background(Color.green.opacity(0.9))
    .cornerRadius(16)
}
.disabled(pickedIngredients.isEmpty)
.padding(.top, 6)
}

private func resolvedRecipeName() -> String {
    let trimmed = recipeName.trimmingCharacters(in:
.whitespacesAndNewlines)

    // 1 정상 입력된 이름이 있으면 그대로 사용
    if trimmed.count >= 2 {
        return trimmed
    }

    // 2 default 이름 생성 (중복 방지)
    let base = "My Recipe"
    let existingNames = store.recipes.map { $0.name }

    if !existingNames.contains(base) {
        return base
    }

    var index = 2
    while existingNames.contains("\(base) \(index)") {
        index += 1
    }
}

```

```

        return "\\(base) \\(index)"
    }

    // MARK: - Suggestions

    @MainActor
    private func updateSuggestions(for text: String) async
    {
        let q = text.trimmingCharacters(in:
        .whitespacesAndNewlines)

        // ✅ 1글자도 제안 나오게: 빈 문자열만 차단
        guard !q.isEmpty else {
            suggestions = []
            return
        }

        let qLower = q.lowercased()

        // ⚠️ allRecipeNames() 없으면, 네가 이미 쓰던 hits 기반
        (임베딩)으로 가져와도 됨.
        // 여기서는 "이름 풀"이 있다고 가정한 버전.
        let allNames =
        RecipeDetailStore.shared.allRecipeNames()

        // ✅ 핵심: 문자열 "처음부터" 정확 prefix 매칭
        let matches = allNames.filter { name in
            let candidate = name
                .lowercased()
                .trimmingCharacters(in:
                .whitespacesAndNewlines)

            return candidate.hasPrefix(qLower)
        }

        // ✅ 중복 제거 + 최대 5개
        let uniq = Array(
            Dictionary(grouping: matches, by: {
            $0.lowercased() })
                .values
                .compactMap { $0.first }
        )
        .prefix(5)

        suggestions = Array(uniq)
    }

```

```

    }

    private func ingredientCell(_ name: String) -> some
    View {
        let isSelected = selectedIngredient == name
        let isCustom =
ingredientStore.isCustomIngredient(name) // ✅ 핵심

        return Button {
            selectedIngredient = isSelected ? nil : name
        } label: {
            HStack(spacing: 6) {

                // ✅ 사용자 생성 재료 표시
                if isCustom {
                    Image(systemName: "person.fill")
                        .font(.caption2)
                        .foregroundColor(.orange)
                }

                Text(name)
                    .font(.system(size: 14, weight: .semibold))
                    .foregroundColor(
                        isSelected
                        ? .black
                        : isCustom
                        ? .orange // ✅ 사용자 재료 텍스트 색
                        : .white
                    )

                Spacer()
            }
            .frame(maxWidth: .infinity, alignment: .leading)
            .padding(.horizontal, 8)
            .padding(.vertical, 12)
            .background(
                isSelected
                ? Color.yellow.opacity(0.9)
                : isCustom
                ? Color.orange.opacity(0.18) // ✅ 사용자 재료
배경
                : Color.black.opacity(0.25)
            )
            .cornerRadius(10)
        }
        .buttonStyle(.plain)
    }

```

```
}  
}
```

```
// MARK: - Hide keyboard helper
```

```
private extension View {  
    func hideKeyboard() {  
        #if canImport(Uikit)
```

```
UIApplication.shared.sendAction(#selector(UIResponder.  
resignFirstResponder),
```

```
to: nil, from: nil, for: nil)
```

```
        #endif
```

```
    }
```

```
}
```

lookupIngredientForEdit을 재활용하면 될것 같다.

Show less ^

생성된 화일이 없다. 왜인가?

첨부된 화면에서 이미 만들어진 recipe를 선택하면 recipe detail view로 보여지게 해라. 이것은 이미 만들어진것이라 그쪽의 에디터로 편집되어야 한다. //

```
// MyRecipeEditorView.swift
```

```
// MenuCal
```

```
//
```

```
// Created by Dae-Kyoo Kim on 12/19/25.
```

```
//
```

```
import SwiftUI
```

```
import Foundation //✅ 이 줄 추가
```

```
struct MyRecipeEditorView: View {
```

```
    enum Mode: Identifiable {
```

```
        case create
```

```
        case edit(UserRecipe)
```

```

var id: String {
    switch self {
        case .create: return "create"
        case .edit(let r): return r.id.uuidString
    }
}
}
}

```

```

let mode: Mode
@Environment(\.dismiss) private var dismiss
@State private var isTyping = false

```

```

@StateObject private var store =
UserRecipeStore.shared
@State private var isShowingAddCustomIngredient =
false

```

```

// ---- name + suggestions
@State private var recipeName: String = ""
@State private var suggestions: [String] = []
@State private var isSuggesting = false
@FocusState private var isNameFocused: Bool

```

```

private let units = ["g","ml","cup","tbsp","tsp","piece"]

```

```

@State private var selectedIngredient: String? = nil
@State private var selectedUnit: String = "g"
@State private var selectedAmount: Int = 1

```

```

// ---- current recipe ingredients
@State private var pickedIngredients: [UserIngredient]
= []
@EnvironmentObject var ingredientStore:
IngredientStore

```

```

// 카테고리 expand / collapse 상태
@State private var expandedCategories:
Set<IngredientStore.IngredientCategory> = []

```

```

// ---- loading
@State private var isLoadingDB = true

```

```

@State private var isSelectingCustomIngredient = false

```

```

private var pickedExistingIngredients: [UserIngredient]

```

```

{
    pickedIngredients.filter {
        !ingredientStore.isCustomIngredient($0.name)
    }
}

private var pickedCustomIngredients: [UserIngredient]
{
    pickedIngredients.filter {
        ingredientStore.isCustomIngredient($0.name)
    }
}

var body: some View {
    ScrollView {
        VStack(alignment: .leading, spacing: 18) {

            if isLoadingDB {
                ProgressView()
                    .padding(.top, 40)
                    .frame(maxWidth: .infinity)
            } else {
                nameSection
                suggestionDropdown
                ingredientGridSection
                unitAmountSection
                addIngredientButton

                // =====
                // Existing Ingredients Added
                // =====
                if !pickedExistingIngredients.isEmpty {
                    VStack(alignment: .leading, spacing: 8) {

                        Text("Ingredients Added")
                            .foregroundColor(.white)
                            .font(.system(size: 16, weight: .bold))

                        ForEach(pickedExistingIngredients.indices, id: \.self) { idx
                        in
                            let item =
                                pickedExistingIngredients[idx]
                            let callInfo =
                                MenuParser.shared.lookupIngredientForEdit(item.name)

```



```

VStack(alignment: .leading, spacing:
4) {

    HStack {
        Text("• \${item.name}")
            .foregroundColor(.white)

        Spacer()

        Text("\${item.amount} \${item.unit}")

    .foregroundColor(.white.opacity(0.8))

    Button {
        pickedIngredients.removeAll {
$0.id == item.id }
        } label: {
            Image(systemName: "trash")

        .foregroundColor(.red.opacity(0.9))
        }
        .buttonStyle(.plain)
    }

    // 🔥 여기 추가
    if let info = callInfo {
        Text("\${Int(info.calories)} kcal / \
(info.unit)")

        .font(.caption)

    .foregroundColor(.yellow.opacity(0.9))
    }
    }
    .padding(12)

    .background(Color.black.opacity(0.25))
        .cornerRadius(10)
    }
    }
    }

    Text("Can't find it?")
        .font(.footnote)
        .foregroundColor(.white.opacity(0.6))
        .frame(maxWidth: .infinity, alignment:
.center)

```

addCustomIngredientButton

```
// =====  
// Custom Ingredients Added  
// =====  
if !pickedCustomIngredients.isEmpty {  
    VStack(alignment: .leading, spacing: 8) {  
  
        Text("Custom Ingredients Added")  
            .foregroundColor(.white)  
            .font(.system(size: 16, weight: .bold))  
  
        ForEach(pickedCustomIngredients.indices, id: \.self) { idx  
in  
            let item =  
pickedCustomIngredients[idx]  
            let callInfo =  
MenuParser.shared.lookupIngredientForEdit(item.name)  
  
            VStack(alignment: .leading, spacing:  
4) {  
  
                HStack {  
                    Image(systemName: "person.fill")  
                        .foregroundColor(.orange)  
                        .font(.caption)  
  
                    Text(item.name)  
                        .foregroundColor(.orange)  
  
                    Spacer()  
  
                    Text("\(item.amount) \(item.unit)")  
  
                }.foregroundColor(.white.opacity(0.8))  
  
                Button {  
                    pickedIngredients.removeAll {  
$0.id == item.id }  
                } label: {  
                    Image(systemName: "trash")  
  
                }.foregroundColor(.red.opacity(0.9))  
            }  
        }  
    }  
}
```

```

        .buttonStyle(.plain)
    }

    // 🔥 여기 추가
    if let info = callInfo {
        Text("\(Int(info.calories)) kcal /\
(info.unit)")
    }
    .font(.caption)

    .foregroundColor(.yellow.opacity(0.9))
    }
    }
    .padding(12)

    .background(Color.orange.opacity(0.15))
    .cornerRadius(10)
    }
    }
    }
    saveButton
    }
    }
    .padding()
}

// ✅🔥 핵심: ScrollView 바깥 배경
.navigationBarTitle(modeTitle)
.navigationBarTitleDisplayMode(.inline)
.toolbar {

    // Cancel (항상 표시)
    ToolbarItem(placement: .topBarLeading) {
        Button("Cancel") {
            dismiss()
        }
    }

    // 🔥 Delete (Edit 모드에서만)
    ToolbarItem(placement: .topBarTrailing) {
        if case .edit(let recipe) = mode {
            Button(role: .destructive) {
                store.delete(recipe)
                dismiss()
            } label: {
                Image(systemName: "trash")
            }
        }
    }
}

```

```

    }
}
.sheet(isPresented:
$isShowingAddCustomIngredient) {
    AddIngredientBar(store: ingredientStore) {
newIngredient in
        selectedIngredient = newIngredient
        isSelectingCustomIngredient = true
    }
}
.task {
    await RecipeDetailStore.shared.loadIfNeeded()
    isLoadingDB = false

    switch mode {
    case .create:
        break
    case .edit(let r):
        recipeName = r.name
        pickedIngredients = r.ingredients
    }
}
.onChange(of: recipeName) { text in
    Task { await updateSuggestions(for: text) }
}
.onTapGesture {
    endTyping()
}
}

private func endTyping() {
    isTyping = false
    suggestions = []
    isSuggesting = false
    isNameFocused = false // 🔥 키보드 내려감
}

private var modeTitle: String {
    switch mode {
    case .create: return "Create"
    case .edit: return "Edit"
    }
}
}

```

// MARK: - Sections

```

private var nameSection: some View {
    VStack(alignment: .leading, spacing: 8) {
        Text("Recipe Name")
            .foregroundColor(.white)
            .font(.system(size: 16, weight: .bold))

        TextField("Type recipe name...", text:
$recipeName)
            .textInputAutocapitalization(.words)
            .padding(12)
            .background(Color.black.opacity(0.7))
            .foregroundColor(.white)
            .cornerRadius(10)
            .focused($isNameFocused)
            .onChange(of: recipeName) { _ in
                isTyping = true
            }
            .onSubmit {
                // ✅ 타이핑 종료
                isTyping = false
                suggestions = []
                isSuggesting = false
            }
    }
}

private var suggestionDropdown: some View {
    Group {
        if isTyping && !suggestions.isEmpty {
            VStack(alignment: .leading, spacing: 0) {
                ForEach(suggestions.indices, id: \.self) { i in
                    let s = suggestions[i]
                    Button {
                        recipeName = s
                        endTyping()
                        hideKeyboard()
                    } label: {
                        HStack {
                            Text(s)
                                .foregroundColor(.white)
                                .lineLimit(1)
                            Spacer()
                        }
                    }
                }
                .padding(.vertical, 10)
                .padding(.horizontal, 12)
                .background(Color.black.opacity(0.22))
            }
        }
    }
}

```



```

? 180 : 0))
        .animation(.easeInOut(duration:
0.2), value: isExpanded)
    }
    .padding(.vertical, 8)
    .padding(.horizontal, 6)
    .contentShape(Rectangle()) // 🔥 전체 탭
영역
    .onTapGesture {
        if isExpanded {

expandedCategories.remove(section.category)
        } else {

expandedCategories.insert(section.category)
        }
    }

    // divider
    Rectangle()
        .fill(Color.white.opacity(0.15))
        .frame(height: 1)

    // ✅ EXPANDED CONTENT
    if isExpanded {
        LazyVGrid(
            columns: Array(
                repeating: GridItem(.flexible()),
spacing: 10, alignment: .leading),
            count: 3
        ),
        spacing: 10
    ) {
        ForEach(section.items.indices, id:
\self) { index in

ingredientCell(section.items[index])
            }
        }
        .padding(.top, 6)
        .transition(.opacity.combined(with:
.move(edge: .top)))
    }
}
}
}
}

```

```

        .padding(.vertical, 4)
    }
    .frame(maxHeight: 320)
    .background(Color.black.opacity(0.18))
    .cornerRadius(12)
}
}

private var unitAmountSection: some View {
    Group {
        if let ing = selectedIngredient {
            VStack(alignment: .leading, spacing: 8) {

                // 🔥 custom ingredient일 때만 강조
                if isSelectingCustomIngredient {
                    Text("Amount & Unit · \((ing)")
                        .foregroundColor(.orange)
                        .font(.system(size: 16, weight: .bold))
                } else {
                    Text("Amount & Unit")
                        .foregroundColor(.white)
                        .font(.system(size: 16, weight: .bold))
                }

                HStack(spacing: 12) {

                    Picker("Amount", selection:
$selectedAmount) {
                        ForEach(1...30, id: \.self) { Text("\( $0)") }
                    }
                    .pickerStyle(.wheel)

                    Picker("Unit", selection: $selectedUnit) {
                        ForEach(units, id: \.self) { Text($0) }
                    }
                    .pickerStyle(.wheel)
                }
                .frame(minHeight: 120)
                .background(
                    isSelectingCustomIngredient
                    ? Color.orange.opacity(0.18)
                    : Color.black.opacity(0.18)
                )
                .cornerRadius(12)
            }
        }
    }
}

```



```

    }
}

private var addCustomIngredientButton: some View {
    Button {
        // 기존 AddIngredientBar가 하던 동작으로 연결
        isShowingAddCustomIngredient = true
    } label: {
        HStack {
            Image(systemName: "plus.circle.fill")
            Text("Add Custom Ingredient")
                .fontWeight(.semibold)
        }
        .foregroundColor(.black)
        .frame(maxWidth: .infinity)
        .padding(.vertical, 14)
        .background(
            LinearGradient(
                colors: [Color.green.opacity(0.85),
Color.green],
                startPoint: .leading,
                endPoint: .trailing
            )
        )
        .cornerRadius(14)
    }
}

```

```

private var addIngredientButton: some View {
    Button {
        guard let ing = selectedIngredient else { return }

        // 같은 재료를 여러 번 허용할지 정책 필요:
        // 지금은 "허용" (원하면 중복 방지로 바꿔줄게)
        pickedIngredients.append(
            UserIngredient(name: ing, unit: selectedUnit,
amount: selectedAmount)
        )

        // ✅ single selection reset
        selectedIngredient = nil
        selectedAmount = 1
        selectedUnit = "g"
    } label: {
        HStack {
            Image(systemName: "plus")

```

```

        Text("Add Ingredient")
        .fontWeight(.bold)
    }
    .foregroundColor(.black)
    .frame(maxWidth: .infinity)
    .padding(.vertical, 12)
    .background(
        (selectedIngredient == nil) ?
Color.gray.opacity(0.35) : Color.yellow.opacity(0.95)
    )
    .cornerRadius(14)
}
.disabled(selectedIngredient == nil)
}

private var pickedListSection: some View {
    VStack(alignment: .leading, spacing: 10) {
        Text("Ingredients Added")
        .foregroundColor(.white)
        .font(.system(size: 16, weight: .bold))

        if pickedIngredients.isEmpty {
            Text("No ingredients yet.")
            .foregroundColor(.white.opacity(0.7))
            .font(.system(size: 14))
        } else {
            VStack(spacing: 8) {
                ForEach(pickedIngredients.indices, id: \.self) {
idx in
                    let item = pickedIngredients[idx]
                    HStack {
                        Text("\(item.name)")
                        .foregroundColor(.white)
                        Spacer()
                        Text("\(item.amount) \(item.unit)")
                        .foregroundColor(.white.opacity(0.8))
                        Button {
                            pickedIngredients.remove(at: idx)
                        } label: {
                            Image(systemName: "trash")
                            .foregroundColor(.red.opacity(0.9))
                        }
                        .buttonStyle(.plain)
                    }
                    .padding(12)
                    .background(Color.black.opacity(0.25))
                }
            }
        }
    }
}

```

```

        .cornerRadius(10)
    }
}
}
}
}

private var saveButton: some View {
    Button {
        let finalName = resolvedRecipeName()
        guard !pickedIngredients.isEmpty else { return }

        switch mode {
        case .create:
            let new = UserRecipe(
                name: finalName,
                ingredients: pickedIngredients
            )
            store.add(new)

        case .edit(let old):
            var edited = old
            edited.name = finalName
            edited.ingredients = pickedIngredients
            edited.updatedAt = Date()
            store.update(edited)
        }

        dismiss()
    } label: {
        HStack {
            Image(systemName: "checkmark.circle.fill")
            Text("Save")
                .fontWeight(.bold)
        }
        .foregroundColor(.black)
        .frame(maxWidth: .infinity)
        .padding(.vertical, 14)
        .background(Color.green.opacity(0.9))
        .cornerRadius(16)
    }
    .disabled(pickedIngredients.isEmpty)
    .padding(.top, 6)
}

```

```

private func resolvedRecipeName() -> String {
    let trimmed = recipeName.trimmingCharacters(in:
.whitespacesAndNewlines)

    // 1 정상 입력된 이름이 있으면 그대로 사용
    if trimmed.count >= 2 {
        return trimmed
    }

    // 2 default 이름 생성 (중복 방지)
    let base = "My Recipe"
    let existingNames = store.recipes.map { $0.name }

    if !existingNames.contains(base) {
        return base
    }

    var index = 2
    while existingNames.contains("\(base) \(index)") {
        index += 1
    }

    return "\(base) \(index)"
}

// MARK: - Suggestions

@MainActor
private func updateSuggestions(for text: String) async
{
    let q = text.trimmingCharacters(in:
.whitespacesAndNewlines)

    // 1글자도 제안 나오게: 빈 문자열만 차단
    guard !q.isEmpty else {
        suggestions = []
        return
    }

    let qLower = q.lowercased()

    // ! allRecipeNames() 없으면, 네가 이미 쓰던 hits 기반
    (임베딩)으로 가져와도 됨.
    // 여기서 "이름 풀"이 있다고 가정한 버전.
    let allNames =
RecipeDetailStore.shared.allRecipeNames()

```

```

// ✅ 핵심: 문자열 "처음부터" 정확 prefix 매칭
let matches = allNames.filter { name in
    let candidate = name
        .lowercased()
        .trimmingCharacters(in:
.whitespacesAndNewlines)

    return candidate.hasPrefix(qLower)
}

// ✅ 중복 제거 + 최대 5개
let uniq = Array(
    Dictionary(grouping: matches, by: {
$0.lowercased() })
        .values
        .compactMap { $0.first }
    )
    .prefix(5)

suggestions = Array(uniq)
}

private func ingredientCell(_ name: String) -> some
View {
    let isSelected = selectedIngredient == name
    let isCustom =
ingredientStore.isCustomIngredient(name) // ✅ 핵심

    return Button {
        selectedIngredient = isSelected ? nil : name
    } label: {
        HStack(spacing: 6) {

            // ✅ 사용자 생성 재료 표시
            if isCustom {
                Image(systemName: "person.fill")
                    .font(.caption2)
                    .foregroundColor(.orange)
            }

            Text(name)
                .font(.system(size: 14, weight: .semibold))
                .foregroundColor(
                    isSelected
                    ? .black

```

```

        : isCustom
        ? .orange // ✅ 사용자 재료 텍스트 색
        : .white
    )

    Spacer()
}
.frame(maxWidth: .infinity, alignment: .leading)
.padding(.horizontal, 8)
.padding(.vertical, 12)
.background(
    isSelected
    ? Color.yellow.opacity(0.9)
    : isCustom
    ? Color.orange.opacity(0.18) // ✅ 사용자 재료
배경
    : Color.black.opacity(0.25)
)
.cornerRadius(10)
}
.buttonStyle(.plain)
}
}

// MARK: - Hide keyboard helper
private extension View {
    func hideKeyboard() {
        #if canImport(UiKit)

        UIApplication.shared.sendAction(#selector(UIResponder.
resignFirstResponder),
                                        to: nil, from: nil, for: nil)

        #endif
    }
}

```

Show less ^

첨부화면에서 recipe name field의 배경색을 ingredient field의 배경색과 같게 해서 editable할수 있음을 보여주고 recipe name 이 full 로 보일수 있게 하라. 지금은 ...으로 잘린다.

```
//  
// RecipeEditView.swift  
// MenuCal  
//  
// Created by Dae-Kyoo Kim on 12/30/25.  
//  
  
import Foundation  
import SwiftUI  
struct ExistingRecipeEditView: View {  
  
    let detail: RecipeDetail  
    let caloriesByIngredient: [String: (kcal: Int, unit: String)]  
    let totalCalories: Int  
  
    @Environment(\.dismiss) var dismiss  
  
    // 🖋 편집용 상태  
    @State private var recipeName: String  
    @State private var ingredients: [EditableIngredient]  
    @State private var pendingCustomIngredient: String?  
    = nil  
    @State private var pendingAmount: Int = 1  
    @State private var pendingUnit: String = "g"  
    @State private var steps: [String]
```

```

// + Add bars
@State private var showAddIngredient = false
@State private var showAddStep = false

// 현재 편집 중인 ingredient index
@FocusState private var focusedIngredientIndex: Int?
private let calorieDB = MenuParser.shared

@StateObject private var userRecipeStore =
UserRecipeStore.shared

init(
    detail: RecipeDetail,
    caloriesByIngredient: [String: (kcal: Int, unit: String)],
    totalCalories: Int
) {
    self.detail = detail
    self.caloriesByIngredient = caloriesByIngredient
    self.totalCalories = totalCalories
    _recipeName = State(initialValue: detail.name)

    _ingredients = State(
        initialValue: detail.ingredients.map {
            EditableIngredient(
                name: $0,
                amount: nil,
                unit: nil,
                isCustom: false
            )
        }
    )
    _steps = State(initialValue: detail.steps)
}

var body: some View {
    ZStack {
        // 🔥 EDIT MODE BACKGROUND (확실히 구분됨)
        LinearGradient(
            colors: [
                Color.black,
                Color.blue.opacity(0.25)
            ],
            startPoint: .top,
            endPoint: .bottom
        )
    }
}

```



```

        .ignoresSafeArea()

        NavigationStack {
            ScrollView {
                VStack(alignment: .leading, spacing: 30) {

                    // HEADER
                    HStack(spacing: 12) {
                        Text(autoEmoji(for: detail.name))
                            .font(.system(size: 40))

                        TextField("Recipe Name", text:
$recipeName)
                            .font(.system(size: 26, weight: .bold,
design: .rounded))
                            .foregroundColor(.white)
                            .textInputAutocapitalization(.words)
                            .disableAutocorrection(true)

                    }

                    .background(Color.orange.opacity(0.15))

                    Spacer()

                    HStack(spacing: 4) {
                        Text("\(totalCalories)")
                            .foregroundColor(.yellow)
                            .font(.headline)
                            .fontWeight(.bold)

                        Text("kcal")

                    }

                    .foregroundColor(.yellow.opacity(0.8))
                    .font(.caption)

                    Text("/ dish")

                    .foregroundColor(.white.opacity(0.5))
                    .font(.caption)
                }
            }

            CalorieProgressBar(calories: totalCalories)

            ingredientsSection

            if let pending = pendingCustomIngredient {

```

```

VStack(alignment: .leading, spacing: 8) {
    Text("Amount & Unit · Custom
Ingredient")

    .foregroundColor(.orange)
    .font(.headline)

    HStack(spacing: 12) {
        Picker("Amount", selection:
$pendingAmount) {
            ForEach(1...30, id: \.self) { Text("\
($0)") }

        }
        .pickerStyle(.wheel)

        Picker("Unit", selection:
$pendingUnit) {
            ForEach(["g","ml","tbsp","tsp","cup","piece"], id: \.self) {
                Text($0)
            }
        }
        .pickerStyle(.wheel)
    }
    .frame(height: 120)

    .background(Color.orange.opacity(0.15))
    .cornerRadius(12)

    Button {
        let callInfo =
calorieDB.lookupIngredientForEdit(pending)

        ingredients.append(
            EditableIngredient(
                name: pending,
                amount: pendingAmount,
                unit: pendingUnit,
                isCustom: true,
                kcal: callInfo.map {
Int($0.calories) },

                kcalUnit: callInfo?.unit
            )
        )

        pendingCustomIngredient = nil
    } label: {

```

```

        Text("Add Custom Ingredient")
            .fontWeight(.bold)
            .frame(maxWidth: .infinity)
            .padding()
            .background(Color.orange)
            .foregroundColor(.black)
            .cornerRadius(12)
    }
}
}

stepsSection

Spacer()
}
.padding(.horizontal, 35)
.padding(.top, 20)
}
.navigationTitle("Edit Recipe")
.navigationBarTitleDisplayMode(.inline)
.toolbar {
    ToolbarItem(placement: .topBarLeading) {
        Button("Cancel") { dismiss() }
    }

    ToolbarItem(placement: .topBarTrailing) {
        Button("Save") {
            saveToUserRecipes()
            dismiss()
        }
    }
}
}
// 🔥 네비게이션 바 배경도 투명 처리
.toolbarBackground(.hidden, for: .navigationBar)
}
}

```

```

private var ingredientsSection: some View {
    VStack(alignment: .leading, spacing: 8) {

```

```

        Text("Ingredients")
            .font(.title2)
            .fontWeight(.semibold)

```

```

        ForEach(ingredients.indices, id: \.self) { i in

```

```

let item = ingredients[i]
let isFocused = focusedIngredientIndex == i

VStack(alignment: .leading, spacing: 6) {

    HStack(alignment: .top, spacing: 8) {

        VStack(alignment: .leading, spacing: 4) {

            // 1 메인 텍스트 (멀티라인 표시)
            let displayText: String = {
                if item.isCustom,
                    let amount = item.amount,
                    let unit = item.unit {
                        return "\\(amount) \\(unit) \
(item.name)"
                    } else {
                        return item.name
                    }
                }()

            if isFocused {
                // ✎ 편집 모드
                TextEditor(
                    text: Binding(
                        get: { item.name },
                        set: { ingredients[i].name = $0 }
                    )
                )
                .foregroundColor(item.isCustom ?
.orange : .white)
                .frame(minHeight: 40)
                .background(Color.clear)
            } else {
                // 👁 표시 모드 (여기서 줄바꿈 허용)
                Text(displayText)
                .foregroundColor(item.isCustom ?
.orange : .white)
                .font(.body)
                .multilineTextAlignment(.leading)
                .fixedSize(horizontal: false, vertical:
true)

                .onTapGesture {
                    focusedIngredientIndex = i
                }
            }
        }
    }
}

```

```

        // 2 칼로리 / 기준 단위 (항상 다음 줄)
        if let cal = calorieInfo(for: item) {
            Text("\(cal.kcal) kcal / \(cal.unit)")
                .font(.caption)

        .foregroundColor(.yellow.opacity(0.9))
        }
    }

    Spacer()

    // ✖ 삭제 버튼
    Button {
        ingredients.remove(at: i)
        focusedIngredientIndex = nil
    } label: {
        Image(systemName: "minus.circle.fill")
            .foregroundColor(.red)
            .font(.system(size: 18, weight:
.semibold))
    }
        .buttonStyle(.plain)
    }
    }
    .padding(.vertical, 10)
    .padding(.horizontal, 12)
    .background(
        RoundedRectangle(cornerRadius: 10)
            .fill(isFocused ? Color.blue.opacity(0.35)
                : Color.white.opacity(0.06))
    )
    .overlay(
        RoundedRectangle(cornerRadius: 10)
            .stroke(isFocused ? Color.blue.opacity(0.9)
                : Color.clear,
                lineWidth: 1.5)
    )
    }

    Button {
        showAddIngredient = true
    } label: {
        HStack {
            Image(systemName: "plus.circle")
            Text("Add Ingredient")
        }
    }

```

```

    }
    .foregroundColor(.white.opacity(0.7))
  }
}
// 🔥 여기 추가
.contentShape(Rectangle()) // 빈 공간도 터치 가능
하게
.onTapGesture {
  focusedIngredientIndex = nil // ✅ 선택 해제
}
.sheet(isPresented: $showAddIngredient) {
  AddIngredientBar(store: IngredientStore.shared) {
newIngredient in
    pendingCustomIngredient = newIngredient
    pendingAmount = 1
    pendingUnit = "g"
  }
}
}
private var stepsSection: some View {
  VStack(alignment: .leading, spacing: 8) {

    Text("Steps")
      .font(.title2)
      .fontWeight(.semibold)

    if steps.isEmpty {
      Button {
        showAddStep = true
      } label: {
        HStack {
          Image(systemName: "plus.circle")
          Text("Add Steps")
        }
        .foregroundColor(.white.opacity(0.7))
      }
    } else {
      ForEach(steps.indices, id: \.self) { i in
        TextEditor(text: $steps[i])
          .frame(minHeight: 60)
          .foregroundColor(.white)
          .background(Color.white.opacity(0.08))
          .cornerRadius(8)
      }

      Button {

```

```

        showAddStep = true
    } label: {
        HStack {
            Image(systemName: "plus.circle")
            Text("Add Step")
        }
        .foregroundColor(.white.opacity(0.7))
    }
}
}
.sheet(isPresented: $showAddStep) {
    VStack {
        Text("Add Step (Skeleton)")
        Button("Add Empty Step") {
            steps.append("")
            showAddStep = false
        }
    }
    .padding()
}
}

```

```

private fun calorieInfo(for item: EditableIngredient)
-> (kcal: Int, unit: String)? {

```

```

    // ✅ 1. 기존 레시피 재료
    if !item.isCustom {
        return caloriesByIngredient[item.name]
    }

```

```

    // ✅ 2. 사용자 추가 재료 (공용 lookup)
    if let info =
MenuParser.shared.lookupIngredientForEdit(item.name) {
        return (Int(info.calories), info.unit)
    }

    return nil
}

```

```

private fun saveToUserRecipes() {

```

```

    // EditableIngredient → UserIngredient 변환
    let userIngredients: [UserIngredient] =
ingredients.map { item in
    UserIngredient(
        name: item.name,

```

```
        unit: item.unit ?? "g",
        amount: item.amount ?? 1
    )
}

let finalName = recipeName.trimmingCharacters(in:
.whitespacesAndNewlines).isEmpty
    ? detail.name
    : recipeName

let newUserRecipe = UserRecipe(
    name: finalName,
    ingredients: userIngredients
)

userRecipeStore.add(newUserRecipe)
}
}
```

Show less ^

수정되지 않았다. 확인해라.

```
//
// RecipeEditView.swift
// MenuCal
//
// Created by Dae-Kyoo Kim on 12/30/25.
//

import Foundation
import SwiftUI
struct ExistingRecipeEditView: View {

    let detail: RecipeDetail
    let caloriesByIngredient: [String: (kcal: Int, unit: String)]
    let totalCalories: Int

    @Environment(\.dismiss) var dismiss

    // ✎ 편집용 상태
    @State private var recipeName: String
    @State private var ingredients: [EditableIngredient]
    @State private var pendingCustomIngredient: String?
    = nil
    @State private var pendingAmount: Int = 1
    @State private var pendingUnit: String = "g"
    @State private var steps: [String]

    // ➕ Add bars
    @State private var showAddIngredient = false
    @State private var showAddStep = false

    // 현재 편집 중인 ingredient index
    @FocusState private var focusedIngredientIndex: Int?
    private let calorieDB = MenuParser.shared

    @StateObject private var userRecipeStore =
    UserRecipeStore.shared

    init(
        detail: RecipeDetail,
        caloriesByIngredient: [String: (kcal: Int, unit: String)],
        totalCalories: Int
```



```

$recipeName)
        .font(.system(size: 26, weight: .bold,
design: .rounded))
        .foregroundColor(.white)
        .textInputAutocapitalization(.words)
        .disableAutocorrection(true)
        .textFieldStyle(.plain) // 기본 테두리 제거
        .padding(.vertical, 8)
        .padding(.horizontal, 12)
        .background(
            RoundedRectangle(cornerRadius:
12)
            // 🔥 재료 편집 칸
            (Color.black.opacity(0.35))과 동일한 배경색 적용
            .fill(Color.black.opacity(0.35))
        )
        .overlay(
            RoundedRectangle(cornerRadius:
12)
            // 미세한 테두리를 추가하여 입력 필드
            임을 강조
            .stroke(Color.white.opacity(0.15),
            lineWidth: 1)
        )
        // 입력 시 필드가 너무 작아 보이지 않도록 최
        소 가로 폭 보장
        .frame(minWidth: 150)

        Spacer()

        HStack(spacing: 4) {
            Text("\(totalCalories)")
                .foregroundColor(.yellow)
                .font(.headline)
                .fontWeight(.bold)

            Text("kcal")

            .foregroundColor(.yellow.opacity(0.8))
                .font(.caption)

            Text("/ dish")

            .foregroundColor(.white.opacity(0.5))
                .font(.caption)
        }

```

```

    }

    CalorieProgressBar(calories: totalCalories)

    ingredientsSection

    if let pending = pendingCustomIngredient {
        VStack(alignment: .leading, spacing: 8) {
            Text("Amount & Unit · Custom
Ingredient")
                .foregroundColor(.orange)
                .font(.headline)

            HStack(spacing: 12) {
                Picker("Amount", selection:
$pendingAmount) {
                    ForEach(1...30, id: \.self) { Text("\
($0)") }
                }
                .pickerStyle(.wheel)

                Picker("Unit", selection:
$pendingUnit) {
                    ForEach(["g","ml","tbsp","tsp","cup","piece"], id: \.self) {
                        Text($0)
                    }
                }
                .pickerStyle(.wheel)
            }
            .frame(height: 120)

            .background(Color.orange.opacity(0.15))
            .cornerRadius(12)

            Button {
                let callInfo =
calorieDB.lookupIngredientForEdit(pending)

                ingredients.append(
                    EditableIngredient(
                        name: pending,
                        amount: pendingAmount,
                        unit: pendingUnit,
                        isCustom: true,
                        kcal: callInfo.map {

```

```

Int($0.calories) },

                                kcalUnit: callInfo?.unit
                                )
                                )

                                pendingCustomIngredient = nil
                                } label: {
                                    Text("Add Custom Ingredient")
                                    .fontWeight(.bold)
                                    .frame(maxWidth: .infinity)
                                    .padding()
                                    .background(Color.orange)
                                    .foregroundColor(.black)
                                    .cornerRadius(12)
                                }
                                }
                                }

                                stepsSection

                                Spacer()
                                }
                                .padding(.horizontal, 35)
                                .padding(.top, 20)
                                }
                                .navigationTitle("Edit Recipe")
                                .navigationBarTitleDisplayMode(.inline)
                                .toolbar {
                                    ToolbarItem(placement: .topBarLeading) {
                                        Button("Cancel") { dismiss() }
                                    }

                                    ToolbarItem(placement: .topBarTrailing) {
                                        Button("Save") {
                                            saveToUserRecipes()
                                            dismiss()
                                        }
                                    }
                                }
                                }
                                }
                                // 🔥 네비게이션 바 배경도 투명 처리
                                .toolbarBackground(.hidden, for: .navigationBar)
                                }
                                }

```

```

private var ingredientsSection: some View {

```

```

VStack(alignment: .leading, spacing: 8) {

    Text("Ingredients")
        .font(.title2)
        .fontWeight(.semibold)

    ForEach(ingredients.indices, id: \.self) { i in
        let item = ingredients[i]
        let isFocused = focusedIngredientIndex == i

        VStack(alignment: .leading, spacing: 6) {

            HStack(alignment: .top, spacing: 8) {

                VStack(alignment: .leading, spacing: 4) {

                    // 1 메인 텍스트 (멀티라인 표시)
                    let displayText: String = {
                        if item.isCustom,
                            let amount = item.amount,
                            let unit = item.unit {
                                return "\((amount) \((unit) \
(item.name)"

                        } else {
                            return item.name
                        }
                    }()

                    if isFocused {
                        // ✎ 편집 모드
                        TextEditor(
                            text: Binding(
                                get: { item.name },
                                set: { ingredients[i].name = $0 }
                            )
                        )
                            .foregroundColor(item.isCustom ?
.orange : .white)
                            .frame(minHeight: 40)
                            .background(Color.clear)
                    } else {
                        // 👁 표시 모드 (여기서 줄바꿈 허용)
                        Text(displayText)
                            .foregroundColor(item.isCustom ?
.orange : .white)
                            .font(.body)

```

```

        .multilineTextAlignment(.leading)
        .fixedSize(horizontal: false, vertical:
true)

        .onTapGesture {
            focusedIngredientIndex = i
        }
    }

    // 2 칼로리 / 기준 단위 (항상 다음 줄)
    if let cal = calorieInfo(for: item) {
        Text("\(cal.kcal) kcal / \(cal.unit)")
        .font(.caption)

.foregroundColor(.yellow.opacity(0.9))
    }
}

Spacer()

// ✖ 삭제 버튼
Button {
    ingredients.remove(at: i)
    focusedIngredientIndex = nil
} label: {
    Image(systemName: "minus.circle.fill")
    .foregroundColor(.red)
    .font(.system(size: 18, weight:
.semibold))
}
.buttonStyle(.plain)
}
}
.padding(.vertical, 10)
.padding(.horizontal, 12)
.background(
    RoundedRectangle(cornerRadius: 10)
        .fill(isFocused ? Color.blue.opacity(0.35)
            : Color.white.opacity(0.06))
)
.overlay(
    RoundedRectangle(cornerRadius: 10)
        .stroke(isFocused ? Color.blue.opacity(0.9)
            : Color.clear,
            lineWidth: 1.5)
)
}

```

```

        Button {
            showAddIngredient = true
        } label: {
            HStack {
                Image(systemName: "plus.circle")
                Text("Add Ingredient")
            }
            .foregroundColor(.white.opacity(0.7))
        }
    }
    // 🔥 여기 추가
    .contentShape(Rectangle()) // 빈 공간도 터치 가능
하게
    .onTapGesture {
        focusedIngredientIndex = nil // ✅ 선택 해제
    }
    .sheet(isPresented: $showAddIngredient) {
        AddIngredientBar(store: IngredientStore.shared) {
            newIngredient in
                pendingCustomIngredient = newIngredient
                pendingAmount = 1
                pendingUnit = "g"
            }
        }
    }
private var stepsSection: some View {
    VStack(alignment: .leading, spacing: 8) {

        Text("Steps")
            .font(.title2)
            .fontWeight(.semibold)

        if steps.isEmpty {
            Button {
                showAddStep = true
            } label: {
                HStack {
                    Image(systemName: "plus.circle")
                    Text("Add Steps")
                }
                .foregroundColor(.white.opacity(0.7))
            }
        } else {
            ForEach(steps.indices, id: \.self) { i in
                TextEditor(text: $steps[i])
            }
        }
    }
}

```



```

        .frame(minHeight: 60)
        .foregroundColor(.white)
        .background(Color.white.opacity(0.08))
        .cornerRadius(8)
    }

    Button {
        showAddStep = true
    } label: {
        HStack {
            Image(systemName: "plus.circle")
            Text("Add Step")
        }
        .foregroundColor(.white.opacity(0.7))
    }
}

.sheet(isPresented: $showAddStep) {
    VStack {
        Text("Add Step (Skeleton)")
        Button("Add Empty Step") {
            steps.append("")
            showAddStep = false
        }
    }
    .padding()
}
}

```

```

private func calorieInfo(for item: EditableIngredient)
-> (kcal: Int, unit: String)? {

```

```

    // ✅ 1. 기존 레시피 재료
    if !item.isCustom {
        return caloriesByIngredient[item.name]
    }

```

```

    // ✅ 2. 사용자 추가 재료 (공용 lookup)
    if let info =
        MenuParser.shared.lookupIngredientForEdit(item.name) {
        return (Int(info.calories), info.unit)
    }

    return nil
}

```

```

private fun saveToUserRecipes() {

    // EditableIngredient → UserIngredient 변환
    let userIngredients: [UserIngredient] =
ingredients.map { item in
    UserIngredient(
        name: item.name,
        unit: item.unit ?? "g",
        amount: item.amount ?? 1
    )
}

    let finalName = recipeName.trimmingCharacters(in:
.whitespacesAndNewlines).isEmpty
        ? detail.name
        : recipeName

    let newUserRecipe = UserRecipe(
        name: finalName,
        ingredients: userIngredients
    )

    userRecipeStore.add(newUserRecipe)
}
}

```

Show less ^

favorite 항목들에 대해서 trash 를 chevron으로 replace해라.
chevron을 uncheck하면 delete하는것이다.

동일 이름의 식당이 두번 favorite으로 나와서 unfavorite을 해도 하나가 계속 남는다.

안된다. 이제 아예 unfavorite이 둘다 안된다.

같은 식당이 중복되 나오고 토글도 안된다.

json에 데이터가 갑자기 많아졌다.

--- HealthKit DEBUG (Status Check) ---

Glucose: 1

Systolic: 1

Diastolic: 1

✅ Custom ingredients loaded: 2

🔒 Auth changed: 4

▶ Authorization granted — restarting location updates

📶 Starting location updates (AUTO allowed) — source:
appLaunch

HKM Auth: Requesting authorization...

HKM Auth: SUCCESS

HKM Load: Starting data queries...

===== 📍 HealthKit REAL VALUES =====

📍 Got location: acc=5.0, age=0.3946950435638428

📶 Stable GPS fix acquired

📍 reverseGeocode CALLED with location:

CLLocationCoordinate2D(latitude: 42.6846714,
longitude: -83.1935685)

❤️ Systolic BP: 159.0 mmHg @ 2025-12-20 12:12:20
+0000

🔄 Refreshing HealthKit data...

✅ Health flags updated → diabetes=true,
hypertension=false

🩸 Glucose: 90.0 mg/dL @ 2025-12-22 18:25:00 +0000

❤️ Diastolic BP: 120.0 mmHg @ 2025-12-20 12:12:20
+0000

=====

==
✅ Health flags updated → diabetes=true,
hypertension=true

✅ Health flags refreshed → diabetes=false, hypertension=true

⚠️ B-set misalignment detected: names = 60001 vectors = 60000 → using minCount = 60000

✅ Loaded B index (aligned): 60000 recipes

✅ Got address: The Village of Rochester Hills, Rochester Hills, MI

📍 onAddressUpdated is nil? -> false

📍 Calling onAddressUpdated with: The Village of Rochester Hills, Rochester Hills, MI

📍 App launch location context:
LocationContext(address: "The Village of Rochester Hills, Rochester Hills, MI", coordinate: Optional(__C.CLLocationCoordinate2D(latitude: 42.6846714, longitude: -83.1935685)), source: MenuCal.LocationRequestSource.appLaunch)

✅ Loaded 511 chains, 9 cuisines

📍 AUTO detected restaurant: Mitchell's Fish Market

🍴 FavoriteRestaurant cleanup complete: 20

🍴 entity.menu.count = 7

🍴 first.title = Fresh Seafood Focus (Signature)

🍴 first.body.count = 122

🍴 first.body.preview = Typically refers to an establishment, often casual or fine dining, that emphasizes

🍴 Ingredient parsing START → 'Fresh Seafood Focus (Signature)'

✅ Loaded foodOnlyIngredients.json → 2689 entries (single+two)

🔄 FALLBACK(single) → try title-based single-word parsing

🗑️ NO single-word match → try two-word parsing

🔄 FALLBACK(two) → try title-based two-word parsing

🍴 Ingredient parsing END → found 0 items

🍴 Ingredient parsing START → 'Display Case/Market (Signature)'

🔄 FALLBACK(single) → try title-based single-word parsing

🗑️ NO single-word match → try two-word parsing

✅ TWO-WORD DB HIT → key='assorted fish' ui='fish' phrase='Often features a prominent display of the fresh fish available for the day'

🍴 Ingredient parsing END → found 1 items

🍴 Ingredient parsing START → 'Simple Preparation'

🔄 FALLBACK(single) → try title-based single-word parsing

🗑️ NO single-word match → try two-word parsing

✅ TWO-WORD DB HIT → key='assorted fish' ui='fish'
phrase='Focuses on preparation methods that highlight
the fish's natural quality (e.g.)'
🔍 Ingredient parsing END → found 1 items
🔍 Ingredient parsing START → 'Oyster Bar/Raw Bar'
🔍 Ingredient parsing END (single) → found 2 items
🔍 Ingredient parsing START → 'Seasonal Catch'
🔄 FALLBACK(single) → try title-based single-word
parsing
🔍 NO single-word match → try two-word parsing
🔄 FALLBACK(two) → try title-based two-word parsing
🔍 Ingredient parsing END → found 0 items
🔍 Ingredient parsing START → 'Casual to Upscale'
🔄 FALLBACK(single) → try title-based single-word
parsing
🔍 NO single-word match → try two-word parsing
🔄 FALLBACK(two) → try title-based two-word parsing
🔍 Ingredient parsing END → found 0 items
🔍 Ingredient parsing START → 'Clam Chowder
(Signature Soup)'
🔄 FALLBACK(single) → try title-based single-word
parsing
🔍 NO single-word match → try two-word parsing
🔄 FALLBACK(two) → try title-based two-word parsing
✅ TITLE TWO-WORD HIT → key='andean soup'
ui='soup'
🔍 Ingredient parsing END → found 1 items
🔍 parsed.count = 4
🔍 parsed.first.title = Display Case/Market (Signature)
🔍 parsed.first.restaurantName = Fish Market
🔍 parsed.first.ingredients.count = 1
✅ User recipes loaded (2)

cleanup했는데도 중복식당 뜯다. 토글도 안된다. 깨진 데이터 문제가 아니라는건가.

Show less ^

처음에 아무식당도 favorite되 있지않아 michell를 favorite했더니 하나만 나왔다. 그런데 unfavorit하려고 한번 더 눌렀더니 중복 식당이름이 떴다. 그이후론 unfavorit이 안된다.

```
//  
// FavoriteRestaurantStore.swift  
//  
  
import Foundation  
import Combine  
  
@MainActor  
final class FavoriteRestaurantStore: ObservableObject {  
  
    static let shared = FavoriteRestaurantStore()  
  
    @Published private(set) var favorites:  
    [FavoriteRestaurant] = []  
  
    // MARK: - Derived State (현재 유효한 즐겨찾기)  
  
    var currentFavorites: [FavoriteRestaurant] {  
        let grouped = Dictionary(  
            grouping: favorites,  
            by: { normalize($0.name) }  
        )  
  
        return grouped  
            .compactMap { _, events in
```

```

        events
            .sorted(by: { $0.savedAt > $1.savedAt })
            .first(where: { !$0.isHidden })
        }
        .sorted(by: { $0.savedAt > $1.savedAt }) // ★ UI 안
정성
    }

// ✅ JSON file name
private let fileName = "favorite_restaurants.json"

private init() {
    load()
    pruneCorruptedEvents()
}

// MARK: - Public API

func isFavorite(_ name: String) -> Bool {
    let normalized = normalize(name)
    return currentFavorites.contains {
        normalize($0.name) == normalized
    }
}

func toggle(
    _ name: String,
    origin: FavoriteOrigin = .manual,
    decision: FavoriteDecision = .manual
) {
    let normalized = normalize(name)

    // 🔑 해당 식당의 "마지막 이벤트" 찾기
    let lastEvent = favorites
        .filter { normalize($0.name) == normalized }
        .sorted(by: { $0.savedAt > $1.savedAt })
        .first

    let shouldHide = !(lastEvent?.isHidden ?? true)
    // 설명:
    // - 마지막 이벤트가 visible(false) → hide(true)
    // - 마지막 이벤트가 hidden(true) → show(false)
    // - 이벤트가 없으면 → show(false)

    favorites.append(
        FavoriteRestaurant(

```

```

        name: normalized,
        savedAt: Date(),
        origin: origin,
        decision: decision,
        isHidden: shouldHide
    )
}

save()
}

// MARK: - Persistence (JSON File)

private var fileURL: URL {
    let dir = FileManager.default.urls(
        for: .documentDirectory,
        in: .userDomainMask
    ).first!
    return dir.appendingPathComponent(fileName)
}

private func load() {

    // 1 JSON 파일 로드
    if let data = try? Data(contentsOf: fileURL),
        let decoded = try?
JSONDecoder().decode([FavoriteRestaurant].self, from:
data) {
        favorites = decoded
        return
    }

    // 2 v2 UserDefaults → JSON 마이그레이션
    if let data = UserDefaults.standard.data(forKey:
"favorite_restaurants_v2"),
        let decoded = try?
JSONDecoder().decode([FavoriteRestaurant].self, from:
data) {
        favorites = decoded
        save()
        UserDefaults.standard.removeObject(forKey:
"favorite_restaurants_v2")
        return
    }

    // 3 v1 UserDefaults (String array) → JSON 마이그레이션

```

이선

```
        if let legacy = UserDefaults.standard.array(forKey:
"favorite_restaurants") as? [String] {
            favorites = legacy.map {
                FavoriteRestaurant(
                    name: normalize($0),
                    savedAt: Date(),
                    origin: .manual,
                    decision: .manual
                )
            }
            save()
            UserDefaults.standard.removeObject(forKey:
"favorite_restaurants")
        }
    }
}
```

```
private func save() {
    do {
        let data = try JSONEncoder().encode(favorites)
        try data.write(to: fileURL, options: [.atomic])
    } catch {
        print("❌ Failed to save favorite restaurants:",
error)
    }
}
```

// MARK: - Helpers

```
private func normalize(_ name: String) -> String {
    name
        .lowercased()
        .trimmingCharacters(in:
.whitespacesAndNewlines)
}
```

```
private func pruneCorruptedEvents() {
    let grouped = Dictionary(
        grouping: favorites,
        by: { normalize($0.name) }
    )
}
```

```
var cleaned: [FavoriteRestaurant] = []
```

```
for (_, events) in grouped {
```

```

let sorted = events.sorted { $0.savedAt <
$1.savedAt }

var lastHidden: Bool? = nil

for e in sorted {
    if e.isHidden != lastHidden {
        cleaned.append(e)
        lastHidden = e.isHidden
    }
}

favorites = cleaned
save()

print("🧹 FavoriteRestaurant cleanup complete:",
cleaned.count)
}
}

--- HealthKit DEBUG (Status Check) ---
Glucose: 1
Systolic: 1
Diastolic: 1

-----
✅ Custom ingredients loaded: 2
🔒 Auth changed: 4
▶ Authorization granted — restarting location updates
📶 Starting location updates (AUTO allowed) — source:
appLaunch
HKM Auth: Requesting authorization...
HKM Auth: SUCCESS
HKM Load: Starting data queries...
===== 🌿 HealthKit REAL VALUES =====
📍 Got location: acc=5.0, age=1.3843399286270142
🎉 Stable GPS fix acquired
📍 reverseGeocode CALLED with location:
CLLocationCoordinate2D(latitude: 42.6846714,
longitude: -83.1935685)
🩸 Glucose: 90.0 mg/dL @ 2025-12-22 18:25:00 +0000
❤️ Systolic BP: 159.0 mmHg @ 2025-12-20 12:12:20
+0000
❤️ Diastolic BP: 120.0 mmHg @ 2025-12-20 12:12:20
+0000
=====
==

```

✓ Health flags updated → diabetes=false, hypertension=true

🔄 Refreshing HealthKit data...

✓ Health flags updated → diabetes=true, hypertension=true

⚠️ B-set misalignment detected: names = 60001 vectors = 60000 → using minCount = 60000

✓ Loaded B index (aligned): 60000 recipes

✓ Health flags refreshed → diabetes=false, hypertension=true

🔄 Refreshing HealthKit data...

✓ Health flags refreshed → diabetes=false, hypertension=true

✓ Got address: The Village of Rochester Hills, Rochester Hills, MI

📍 onAddressUpdated is nil? -> false

📍 Calling onAddressUpdated with: The Village of Rochester Hills, Rochester Hills, MI

📍 App launch location context:
LocationContext(address: "The Village of Rochester Hills, Rochester Hills, MI", coordinate: Optional(__C.CLLocationCoordinate2D(latitude: 42.6846714, longitude: -83.1935685)), source: MenuCal.LocationRequestSource.appLaunch)

👉 FavoriteRestaurant cleanup complete: 0

✓ User recipes loaded (2)

✓ Loaded 511 chains, 9 cuisines

📍 AUTO detected restaurant: Mitchell's Fish Market

🍽️ entity.menu.count = 7

🍽️ first.title = Fresh Seafood Focus (Signature)

🍽️ first.body.count = 122

🍽️ first.body.preview = Typically refers to an establishment, often casual or fine dining, that emphasizes

🍽️ Ingredient parsing START → 'Fresh Seafood Focus (Signature)'

✓ Loaded foodOnlyIngredients.json → 2689 entries (single+two)

🔄 FALLBACK(single) → try title-based single-word parsing

🗑️ NO single-word match → try two-word parsing

🔄 FALLBACK(two) → try title-based two-word parsing

🍽️ Ingredient parsing END → found 0 items

🍽️ Ingredient parsing START → 'Display Case/Market (Signature)'

🔄 FALLBACK(single) → try title-based single-word parsing

 NO single-word match → try two-word parsing

 TWO-WORD DB HIT → key='assorted fish' ui='fish'
phrase='Often features a prominent display of the fresh fish available for the day'

 Ingredient parsing END → found 1 items

 Ingredient parsing START → 'Simple Preparation'

 FALLBACK(single) → try title-based single-word parsing

 NO single-word match → try two-word parsing

 TWO-WORD DB HIT → key='assorted fish' ui='fish'
phrase='Focuses on preparation methods that highlight the fish's natural quality (e.g.'

 Ingredient parsing END → found 1 items

 Ingredient parsing START → 'Oyster Bar/Raw Bar'

 Ingredient parsing END (single) → found 2 items

 Ingredient parsing START → 'Seasonal Catch'

 FALLBACK(single) → try title-based single-word parsing

 NO single-word match → try two-word parsing

 FALLBACK(two) → try title-based two-word parsing

 Ingredient parsing END → found 0 items

 Ingredient parsing START → 'Casual to Upscale'

 FALLBACK(single) → try title-based single-word parsing

 NO single-word match → try two-word parsing

 FALLBACK(two) → try title-based two-word parsing

 Ingredient parsing END → found 0 items

 Ingredient parsing START → 'Clam Chowder (Signature Soup)'

 FALLBACK(single) → try title-based single-word parsing

 NO single-word match → try two-word parsing

 FALLBACK(two) → try title-based two-word parsing

 TITLE TWO-WORD HIT → key='andean soup'
ui='soup'

 Ingredient parsing END → found 1 items

 parsed.count = 4

 parsed.first.title = Display Case/Market (Signature)

 parsed.first.restaurantName = Fish Market

 parsed.first.ingredients.count = 1

 entity.menu.count = 7

 first.title = Fresh Seafood Focus (Signature)

 first.body.count = 122

 first.body.preview = Typically refers to an establishment, often casual or fine dining, that emphasizes

 Ingredient parsing START → 'Fresh Seafood Focus

(Signature)'

🔄 FALLBACK(single) → try title-based single-word parsing

🔍 NO single-word match → try two-word parsing

🔄 FALLBACK(two) → try title-based two-word parsing

🔍 Ingredient parsing END → found 0 items

🔍 Ingredient parsing START → 'Display Case/Market (Signature)'

🔄 FALLBACK(single) → try title-based single-word parsing

🔍 NO single-word match → try two-word parsing

✅ TWO-WORD DB HIT → key='assorted fish' ui='fish' phrase='Often features a prominent display of the fresh fish available for the day'

🔍 Ingredient parsing END → found 1 items

🔍 Ingredient parsing START → 'Simple Preparation'

🔄 FALLBACK(single) → try title-based single-word parsing

🔍 NO single-word match → try two-word parsing

✅ TWO-WORD DB HIT → key='assorted fish' ui='fish' phrase='Focuses on preparation methods that highlight the fish's natural quality (e.g.'

🔍 Ingredient parsing END → found 1 items

🔍 Ingredient parsing START → 'Oyster Bar/Raw Bar'

🔍 Ingredient parsing END (single) → found 2 items

🔍 Ingredient parsing START → 'Seasonal Catch'

🔄 FALLBACK(single) → try title-based single-word parsing

🔍 NO single-word match → try two-word parsing

🔄 FALLBACK(two) → try title-based two-word parsing

🔍 Ingredient parsing END → found 0 items

🔍 Ingredient parsing START → 'Casual to Upscale'

🔄 FALLBACK(single) → try title-based single-word parsing

🔍 NO single-word match → try two-word parsing

🔄 FALLBACK(two) → try title-based two-word parsing

🔍 Ingredient parsing END → found 0 items

🔍 Ingredient parsing START → 'Clam Chowder (Signature Soup)'

🔄 FALLBACK(single) → try title-based single-word parsing

🔍 NO single-word match → try two-word parsing

🔄 FALLBACK(two) → try title-based two-word parsing

✅ TITLE TWO-WORD HIT → key='andean soup' ui='soup'

🔍 Ingredient parsing END → found 1 items

 parsed.count = 4
 parsed.first.title = Display Case/Market (Signature)
 parsed.first.restaurantName = Fish Market
 parsed.first.ingredients.count = 1
 [ChainMenuService]
 placeName = Mitchell's Fish Market
 chainSnapshots.count = 4
 [UserScannedMenuStore]
 restaurantKey = mitchell's_fish_market_unknown
 userScannedMenus.count = 0
 MenuScanStore.save BEGIN
 restaurant: Mitchell's Fish Market
 restaurantKey: mitchell fish market
 items.count: 4
 MenuScanStore.save END
 Auto menu saved to history: Mitchell's Fish Market
Snapshot request 0x10fc94f60 complete with error:
<NSError: 0x137e008d0; domain: BSActionErrorDomain;
code: 6 ("anulled")>
 Refreshing HealthKit data...
 Health flags refreshed → diabetes=false,
hypertension=true
 entity.menu.count = 7
 first.title = Fresh Seafood Focus (Signature)
 first.body.count = 122
 first.body.preview = Typically refers to an
establishment, often casual or fine dining, that emphasiz
 Ingredient parsing START → 'Fresh Seafood Focus
(Signature)'
 FALLBACK(single) → try title-based single-word
parsing
 NO single-word match → try two-word parsing
 FALLBACK(two) → try title-based two-word parsing
 Ingredient parsing END → found 0 items
 Ingredient parsing START → 'Display Case/Market
(Signature)'
 FALLBACK(single) → try title-based single-word
parsing
 NO single-word match → try two-word parsing
  TWO-WORD DB HIT → key='assorted fish' ui='fish'
 phrase='Often features a prominent display of the fresh
 fish available for the day'
 Ingredient parsing END → found 1 items
 Ingredient parsing START → 'Simple Preparation'
 FALLBACK(single) → try title-based single-word
parsing

 NO single-word match → try two-word parsing

 TWO-WORD DB HIT → key='assorted fish' ui='fish'
phrase='Focuses on preparation methods that highlight the fish's natural quality (e.g.'

 Ingredient parsing END → found 1 items

 Ingredient parsing START → 'Oyster Bar/Raw Bar'

 Ingredient parsing END (single) → found 2 items

 Ingredient parsing START → 'Seasonal Catch'

 FALLBACK(single) → try title-based single-word parsing

 NO single-word match → try two-word parsing

 FALLBACK(two) → try title-based two-word parsing

 Ingredient parsing END → found 0 items

 Ingredient parsing START → 'Casual to Upscale'

 FALLBACK(single) → try title-based single-word parsing

 NO single-word match → try two-word parsing

 FALLBACK(two) → try title-based two-word parsing

 Ingredient parsing END → found 0 items

 Ingredient parsing START → 'Clam Chowder (Signature Soup)'

 FALLBACK(single) → try title-based single-word parsing

 NO single-word match → try two-word parsing

 FALLBACK(two) → try title-based two-word parsing

 TITLE TWO-WORD HIT → key='andean soup'
ui='soup'

 Ingredient parsing END → found 1 items

 parsed.count = 4

 parsed.first.title = Display Case/Market (Signature)

 parsed.first.restaurantName = Fish Market

 parsed.first.ingredients.count = 1

 [ChainMenuService]

placeName = Mitchell's Fish Market

chainSnapshots.count = 4

 [UserScannedMenuStore]

restaurantKey = mitchell's_fish_market_unknown

userScannedMenus.count = 0

 MenuScanStore.save BEGIN

restaurant: Mitchell's Fish Market

restaurantKey: mitchell fish market

items.count: 4

 MenuScanStore.save END

 Auto menu saved to history: Mitchell's Fish Market

 Refreshing HealthKit data...

 Health flags refreshed → diabetes=false,

hypertension=true

 Refreshing HealthKit data...

 Health flags refreshed → diabetes=false,
hypertension=true

 Refreshing HealthKit data...

unable to decode "ShellSceneKit.ProfileActivation"
because this class does not exist

-[BSSettings initWithXPCDictionary:]_block_invoke

Could not decode object for setting

17456962218540468756

No value decoded for key "profileActivation"
(17456962218540468756). Ignoring.

 Health flags refreshed → diabetes=false,
hypertension=true

Show less ^

여전히 중복 식당나이고 토들안됨. build cleanup하고 앱재실행해
도 마찬가지임.

--- HealthKit DEBUG (Status Check) ---

Glucose: 1

Systolic: 1

Diastolic: 1

✅ Custom ingredients loaded: 2

🔒 Auth changed: 4

▶ Authorization granted — restarting location updates

📶 Starting location updates (AUTO allowed) — source:
appLaunch

HKM Auth: Requesting authorization...

HKM Auth: SUCCESS

HKM Load: Starting data queries...

===== 🖋️ HealthKit REAL VALUES =====

📍 Got location: acc=5.0, age=0.38192903995513916

📶 Stable GPS fix acquired

📍 reverseGeocode CALLED with location:

CLLocationCoordinate2D(latitude: 42.6846714,
longitude: -83.1935685)

🔄 Refreshing HealthKit data...

❤️ Systolic BP: 159.0 mmHg @ 2025-12-20 12:12:20
+0000

❤️ Diastolic BP: 120.0 mmHg @ 2025-12-20 12:12:20
+0000

=====

==
🩸 Glucose: 90.0 mg/dL @ 2025-12-22 18:25:00 +0000

✅ Health flags updated → diabetes=false,
hypertension=true

✅ Health flags updated → diabetes=true,
hypertension=true

✅ Health flags refreshed → diabetes=false,
hypertension=true

⚠️ B-set misalignment detected: names = 60001 vectors
= 60000 → using minCount = 60000

✅ Loaded B index (aligned): 60000 recipes

✅ Got address: The Village of Rochester Hills, Rochester
Hills, MI

📍 onAddressUpdated is nil? -> false

📍 Calling onAddressUpdated with: The Village of
Rochester Hills, Rochester Hills, MI

📍 App launch location context:

LocationContext(address: "The Village of Rochester Hills,
Rochester Hills, MI", coordinate:

Optional(__C.CLLocationCoordinate2D(latitude:
42.6846714, longitude: -83.1935685)), source:

MenuCal.LocationRequestSource.appLaunch)

🔧 FavoriteRestaurant cleanup complete: 17

✅ User recipes loaded (2)

✅ Loaded 511 chains, 9 cuisines

📍 AUTO detected restaurant: Mitchell's Fish Market

🍽️ entity.menu.count = 7

🍽️ first.title = Fresh Seafood Focus (Signature)

🍽️ first.body.count = 122

🍽️ first.body.preview = Typically refers to an establishment, often casual or fine dining, that emphasizes

🍽️ Ingredient parsing START → 'Fresh Seafood Focus (Signature)'

✅ Loaded foodOnlyIngredients.json → 2689 entries (single+two)

🔄 FALLBACK(single) → try title-based single-word parsing

🔧 NO single-word match → try two-word parsing

🔄 FALLBACK(two) → try title-based two-word parsing

🍽️ Ingredient parsing END → found 0 items

🍽️ Ingredient parsing START → 'Display Case/Market (Signature)'

🔄 FALLBACK(single) → try title-based single-word parsing

🔧 NO single-word match → try two-word parsing

✅ TWO-WORD DB HIT → key='assorted fish' ui='fish' phrase='Often features a prominent display of the fresh fish available for the day'

🍽️ Ingredient parsing END → found 1 items

🍽️ Ingredient parsing START → 'Simple Preparation'

🔄 FALLBACK(single) → try title-based single-word parsing

🔧 NO single-word match → try two-word parsing

✅ TWO-WORD DB HIT → key='assorted fish' ui='fish' phrase='Focuses on preparation methods that highlight the fish's natural quality (e.g.'

🍽️ Ingredient parsing END → found 1 items

🍽️ Ingredient parsing START → 'Oyster Bar/Raw Bar'

🍽️ Ingredient parsing END (single) → found 2 items

🍽️ Ingredient parsing START → 'Seasonal Catch'

🔄 FALLBACK(single) → try title-based single-word parsing

🔧 NO single-word match → try two-word parsing

🔄 FALLBACK(two) → try title-based two-word parsing

🍽️ Ingredient parsing END → found 0 items

🍽️ Ingredient parsing START → 'Casual to Upscale'

🔄 FALLBACK(single) → try title-based single-word

parsing

 NO single-word match → try two-word parsing

 FALLBACK(two) → try title-based two-word parsing

 Ingredient parsing END → found 0 items

 Ingredient parsing START → 'Clam Chowder
(Signature Soup)'

 FALLBACK(single) → try title-based single-word
parsing

 NO single-word match → try two-word parsing

 FALLBACK(two) → try title-based two-word parsing

 TITLE TWO-WORD HIT → key='andean soup'

ui='soup'

 Ingredient parsing END → found 1 items

 parsed.count = 4

 parsed.first.title = Display Case/Market (Signature)

 parsed.first.restaurantName = Fish Market

 parsed.first.ingredients.count = 1

 entity.menu.count = 7

 first.title = Fresh Seafood Focus (Signature)

 first.body.count = 122

 first.body.preview = Typically refers to an
establishment, often casual or fine dining, that emphasizes

 Ingredient parsing START → 'Fresh Seafood Focus
(Signature)'

 FALLBACK(single) → try title-based single-word
parsing

 NO single-word match → try two-word parsing

 FALLBACK(two) → try title-based two-word parsing

 Ingredient parsing END → found 0 items

 Ingredient parsing START → 'Display Case/Market
(Signature)'

 FALLBACK(single) → try title-based single-word
parsing

 NO single-word match → try two-word parsing

 TWO-WORD DB HIT → key='assorted fish' ui='fish'
phrase='Often features a prominent display of the fresh
fish available for the day'

 Ingredient parsing END → found 1 items

 Ingredient parsing START → 'Simple Preparation'

 FALLBACK(single) → try title-based single-word
parsing

 NO single-word match → try two-word parsing

 TWO-WORD DB HIT → key='assorted fish' ui='fish'
phrase='Focuses on preparation methods that highlight
the fish's natural quality (e.g.)'

 Ingredient parsing END → found 1 items

 Ingredient parsing START → 'Oyster Bar/Raw Bar'
 Ingredient parsing END (single) → found 2 items
 Ingredient parsing START → 'Seasonal Catch'
 FALLBACK(single) → try title-based single-word parsing
 NO single-word match → try two-word parsing
 FALLBACK(two) → try title-based two-word parsing
 Ingredient parsing END → found 0 items
 Ingredient parsing START → 'Casual to Upscale'
 FALLBACK(single) → try title-based single-word parsing
 NO single-word match → try two-word parsing
 FALLBACK(two) → try title-based two-word parsing
 Ingredient parsing END → found 0 items
 Ingredient parsing START → 'Clam Chowder (Signature Soup)'
 FALLBACK(single) → try title-based single-word parsing
 NO single-word match → try two-word parsing
 FALLBACK(two) → try title-based two-word parsing
 TITLE TWO-WORD HIT → key='andean soup'
 ui='soup'
 Ingredient parsing END → found 1 items
 parsed.count = 4
 parsed.first.title = Display Case/Market (Signature)
 parsed.first.restaurantName = Fish Market
 parsed.first.ingredients.count = 1
 [ChainMenuService]
 placeName = Mitchell Fish Market
 chainSnapshots.count = 4
 [UserScannedMenuStore]
 restaurantKey = mitchell_fish_market_unknown
 userScannedMenus.count = 0
 MenuScanStore.save BEGIN
 restaurant: Mitchell Fish Market
 restaurantKey: mitchell fish market
 items.count: 4
 MenuScanStore.save END
 Auto menu saved to history: Mitchell Fish Market
 Short Tap: Go to detail for Display Case/Market (Signature)

 //
 // FavoriteRestaurantStore.swift
 //


```
import Foundation
```

```
import Combine
```

```
@MainActor
```

```
final class FavoriteRestaurantStore: ObservableObject {
```

```
    static let shared = FavoriteRestaurantStore()
```

```
    @Published private(set) var favorites:
    [FavoriteRestaurant] = []
```

```
    // MARK: - Derived State (현재 유효한 즐겨찾기)
```

```
    var currentFavorites: [FavoriteRestaurant] {
        let grouped = Dictionary(
            grouping: favorites,
            by: { normalize($0.name) }
        )
```

```
        return grouped
```

```
        .compactMap { _, events in
            events
```

```
                .sorted(by: { $0.savedAt > $1.savedAt })
```

```
                .first(where: { !$0.isHidden })
```

```
        }
```

```
        .sorted(by: { $0.savedAt > $1.savedAt }) // ★ UI 안
```

```
정성
```

```
    }
```

```
    // ✅ JSON file name
```

```
    private let fileName = "favorite_restaurants.json"
```

```
    private init() {
```

```
        load()
```

```
        pruneCorruptedEvents()
```

```
    }
```

```
    // MARK: - Public API
```

```
    func isFavorite(_ name: String) -> Bool {
```

```
        let normalized = normalize(name)
```

```
        return currentFavorites.contains {
```

```
            normalize($0.name) == normalized
```

```
        }
```

```
    }
```

```

func toggle(
    _ name: String,
    origin: FavoriteOrigin = .manual,
    decision: FavoriteDecision = .manual
) {
    let normalized = normalize(name)

    // ★ 현재 UI 기준 상태
    let isCurrentlyFavorite =
        currentFavorites.contains {
            normalize($0.name) == normalized
        }

    // ★ toggle은 "현재 상태"의 반대
    favorites.append(
        FavoriteRestaurant(
            name: normalized,
            savedAt: Date(),
            origin: origin,
            decision: decision,
            isHidden: isCurrentlyFavorite // 핵심
        )
    )

    save()
}

// MARK: - Persistence (JSON File)

private var fileURL: URL {
    let dir = FileManager.default.urls(
        for: .documentDirectory,
        in: .userDomainMask
    ).first!
    return dir.appendingPathComponent(fileName)
}

private func load() {

    // 1 JSON 파일 로드
    if let data = try? Data(contentsOf: fileURL),
        let decoded = try?
        JSONDecoder().decode([FavoriteRestaurant].self, from:
        data) {
        favorites = decoded
        return
    }
}

```

```

    }

    // 2 v2 UserDefaults → JSON 마이그레이션
    if let data = UserDefaults.standard.data(forKey:
"favorite_restaurants_v2"),
        let decoded = try?
JSONDecoder().decode([FavoriteRestaurant].self, from:
data) {
        favorites = decoded
        save()
        UserDefaults.standard.removeObject(forKey:
"favorite_restaurants_v2")
        return
    }

    // 3 v1 UserDefaults (String array) → JSON 마이그레이션
    if let legacy = UserDefaults.standard.array(forKey:
"favorite_restaurants") as? [String] {
        favorites = legacy.map {
            FavoriteRestaurant(
                name: normalize($0),
                savedAt: Date(),
                origin: .manual,
                decision: .manual
            )
        }
        save()
        UserDefaults.standard.removeObject(forKey:
"favorite_restaurants")
    }
}

private func save() {
    do {
        let data = try JSONEncoder().encode(favorites)
        try data.write(to: fileURL, options: [.atomic])
    } catch {
        print("❌ Failed to save favorite restaurants:",
error)
    }
}

// MARK: - Helpers

private func normalize(_ name: String) -> String {

```

```

        name
        .lowercased()
        .trimmingCharacters(in:
.whitespacesAndNewlines)
    }

private func pruneCorruptedEvents() {
    let grouped = Dictionary(
        grouping: favorites,
        by: { normalize($0.name) }
    )

    var cleaned: [FavoriteRestaurant] = []

    for (_, events) in grouped {

        let sorted = events.sorted { $0.savedAt <
$1.savedAt }

        var lastHidden: Bool? = nil

        for e in sorted {
            if e.isHidden != lastHidden {
                cleaned.append(e)
                lastHidden = e.isHidden
            }
        }
    }

    favorites = cleaned
    save()

    print("🧹 FavoriteRestaurant cleanup complete:",
cleaned.count)
}

import Foundation
import SwiftUI

struct PreferencesView: View {

    @Environment(\.dismiss) private var dismiss

    @EnvironmentObject var healthKitManager:
HealthKitManager
    @StateObject private var favoriteRestaurants =

```

```

FavoriteRestaurantStore.shared
    @StateObject private var favoriteRecipes =
FavoriteRecipeStore.shared
    @StateObject private var favoriteMenus =
FavoriteMenuStore.shared
    @State private var showAppSettings = false
    @StateObject private var settings = SettingsStore()
    @StateObject private var userScannedStore =
UserScannedMenuStore.shared
    @State private var expandedSections:
Set<PreferenceSection> = []
    @State private var showAnalytics = false
    @StateObject private var userRecipeStore =
UserRecipeStore.shared
    @StateObject private var userIngredientStore =
IngredientStore.shared


enum PreferenceSection: Hashable {
    case scanned
    case favoriteRestaurants
    case favoriteMenus
    case favoriteRecipes
    case myRecipes
    case myIngredients
}


// MARK: - Glass Background (Health Insight Card)

private var glassBackground: some View {
    RoundedRectangle(cornerRadius: 16)
        .fill(Color.white.opacity(0.08)) // 유리 느낌 베이
스
        .overlay(
            RoundedRectangle(cornerRadius: 16)
                .stroke(Color.white.opacity(0.18), lineWidth: 1)
        )
        .background(
            Color.black.opacity(0.25)
                .blur(radius: 12) // ✨
glassmorphism 핵심
        )
    }


    private var favoriteMenuSnapshots:
[MenuItemSnapshot] {

```

```

let favoriteIDs =
    favoriteMenus.favorites
        .filter { !$0.isHidden }
        .map { $0.id }

return MenuScanStore.shared
    .validSnapshots(for: Set(favoriteIDs))
}

var body: some View {
    NavigationStack {
        ZStack {
            Color.black.ignoresSafeArea()
            ScrollView {

                VStack(alignment: .leading, spacing: 24) {

                    // =====
                    // 🧭 Custom Header (다른 뷰들과 동일)
                    // =====
                    HStack {
                        Button {
                            dismiss()
                        } label: {
                            HStack(spacing: 4) {
                                Image(systemName: "chevron.left")
                                Text("Back")
                            }
                        }.font(.system(size: 17, weight:
.semibold))

                            .foregroundColor(.white)
                        }

                        Spacer()
                    }
                    .padding(.top, 24)
                    .padding(.horizontal)

                    // =====
                    // 🍽️ Title
                    // =====
                    Text("My MenuCal")
                        .font(.system(size: 30, weight: .bold))
                        .foregroundColor(.white)
                        .frame(maxWidth: .infinity, alignment:
.center)
                }
            }
        }
    }
}

```

```

        .padding(.horizontal)

VStack(alignment: .leading, spacing: 12) {

    sectionHeader(
        title: "Health Insights (from HealthKit)",
        systemImage: "heart.fill",
        color: healthInsightHeaderColor
    )

    healthInsightRow(
        healthKitManager: healthKitManager
    )
}

// =====
// OCR Scanned Restaurants & Menus
// =====
let scannedRestaurants =
userScannedStore.all()

expandableSection(
    section: .scanned,
    title: "Scanned Restaurants & Menus",
    systemImage:
"doc.text.magnifyingglass",
    color: .cyan,
    itemCount: scannedRestaurants.count
) {
    ForEach(scannedRestaurants.prefix(10))
{ restaurant in
        scannedRestaurantRow(restaurant)
    }
}

// =====
// Favorite Restaurants
// =====
expandableSection(
    section: .favoriteRestaurants,
    title: "Favorite Restaurants",
    systemImage: "fork.knife",
    itemCount:
favoriteRestaurants.currentFavorites.count
) {

```

```

        ForEach(
            favoriteRestaurants.currentFavorites.prefix(10)
                ) { restaurant in
                favoriteRestaurantRow(restaurant)
            }
        }

        // =====
        // Favorite Menus
        // =====
        expandableSection(
            section: .favoriteMenus,
            title: "Favorite Menus",
            systemImage: "menucard.fill",
            itemCount:
favoriteMenuSnapshots.count
        ) {

        ForEach(favoriteMenuSnapshots.prefix(10)) { snapshot in
            ExpandableFavoriteMenuRow(
                menu: snapshot,
                savedAt: favoriteMenus.favorites
                    .first(where: { $0.id ==
snapshot.id })?
                    .savedAt
            )
        }
    }

    // =====
    // Favorite Recipes
    // =====
    expandableSection(
        section: .favoriteRecipes,
        title: "Favorite Recipes",
        systemImage: "book.fill",
        itemCount:
favoriteRecipes.favorites.count
    ) {
        ForEach(
            favoriteRecipes.favorites
                .filter { !$0.isHidden }
                .prefix(10)
        ) { recipe in
            favoriteRecipeRow(recipe)
        }
    }
}

```



```

    }
  }

  // =====
  // My Recipes
  // =====
  expandableSection(
    section: .myRecipes,
    title: "My Recipes",
    systemImage: "book.closed.fill",
    color: .mint,
    itemCount:
userRecipeStore.recipes.count
  ) {

    ForEach(userRecipeStore.recipes.prefix(10)) { recipe in
      myRecipeRow(recipe)
    }
  }

  // =====
  // My Ingredients
  // =====
  expandableSection(
    section: .myIngredients,
    title: "My Ingredients",
    systemImage: "leaf.fill",
    color: .green,
    itemCount:
userIngredientStore.customIngredients.count
  ) {

    ForEach(userIngredientStore.customIngredients.prefix(1
0), id: \.name) { ingredient in
      myIngredientRow(ingredient)
    }
  }

  // =====
  // 🇮🇹 Analytics (NEW)
  // =====
  analyticsSection
}
.frame(maxWidth: .infinity, alignment:
.leading)
.padding()

```

```

    }
    if showAnalytics {
        AnalyticsView(
            onClose: {
                withAnimation(.easeInOut(duration:
0.25)) {
                    showAnalytics = false
                }
            }
        )
        .transition(.move(edge: .trailing)) // 👉 오른쪽
→ 왼쪽
        .zIndex(100)
    }
}
}
.sheet(isPresented: $showAppSettings) {
    SettingsView()
        .environmentObject(SettingsStore()) // ⚠ App
에서 쓰는 동일 인스턴스면 주입
}
}

```

```

private var healthInsightHeaderColor: Color {
    // HealthKit OFF → 중립 색
    guard healthKitManager.isHealthKitEnabled else {
        return .white
    }
}

```

```

let warningCount =
    (healthKitManager.hasDiabetes ? 1 : 0) +
    (healthKitManager.hasHypertension ? 1 : 0)

```

```

switch warningCount {
case 0:
    return .green
case 1:
    return .orange
default:
    return .red
}
}

```

```

private var analyticsSection: some View {
    VStack(alignment: .leading, spacing: 12) {

```

```

        Button {
            withAnimation(.easeInOut(duration: 0.25)) {
                showAnalytics = true
            }
        } label: {
            HStack {
                HStack(spacing: 10) {
                    Image(systemName: "chart.bar.xaxis")
                        .font(.system(size: 16, weight:
.semibold))
                        .foregroundColor(.purple.opacity(0.9))

                    Text("ANALYTICS")
                        .font(.system(size: 13, weight: .bold))
                        .foregroundColor(.purple.opacity(0.85))
                        .tracking(1.0)
                }

                Spacer()

                Image(systemName: "chevron.right")
                    .foregroundColor(.white.opacity(0.6))
            }
            .contentShape(Rectangle()) // 🌟 어디 눌러도 반응
        }
        .buttonStyle(.plain)

        Rectangle()
            .fill(Color.purple.opacity(0.25))
            .frame(height: 1)
    }
    .padding(.top, 24)
}

```

// MARK: - Components

```

@ViewBuilder
private func expandableSection<Content: View>(
    section: PreferenceSection,
    title: String,
    systemImage: String,
    color: Color = .yellow,
    itemCount: Int,
    @ViewBuilder content: () -> Content
) -> some View {

```

```

VStack(alignment: .leading, spacing: 12) {

    // 💎 Header + Chevron
    Button {
        toggle(section)
    } label: {
        HStack {
            HStack(spacing: 10) {
                Image(systemName: systemImage)
                    .font(.system(size: 16, weight:
.semibold))
                    .foregroundColor(color.opacity(0.9))

                Text(title.capitalized)
                    .font(.system(size: 18, weight: .bold))
                    .foregroundColor(color.opacity(0.9))
            }

            Spacer()

            Image(systemName: "chevron.down")

                .rotationEffect(.degrees(isExpanded(section) ? 0 : -90))
                    .foregroundColor(.white.opacity(0.6))
                    .animation(.easeInOut, value:
isExpanded(section))
            }
            .contentShape(Rectangle()) // ★ 이 줄이 핵심
        }
        .buttonStyle(.plain)

        Rectangle()
            .fill(color.opacity(0.25))
            .frame(height: 1)

    // 💎 Content
    if isExpanded(section) {
        if itemCount == 0 {
            emptyHint(text: "No items yet.")
        } else if itemCount <= 10 {
            VStack(alignment: .leading, spacing: 10) {
                content()
            }
        } else {
            ScrollView {
                VStack(alignment: .leading, spacing: 10) {

```

```

        content()
    }
    .padding(.vertical, 4)
}
.frame(maxHeight: 420) // ✅ 10개 이상일 때 스
크롤
    }
    }
    }
}

private func isExpanded(_ section: PreferenceSection)
-> Bool {
    expandedSections.contains(section)
}

private func toggle(_ section: PreferenceSection) {
    withAnimation(.easeInOut) {
        if expandedSections.contains(section) {
            expandedSections.remove(section)
        } else {
            expandedSections.insert(section)
        }
    }
}

private func scannedRestaurantRow(
    _ restaurant: UserScannedRestaurant
) -> some View {

    VStack(alignment: .leading, spacing: 6) {

        // 🍽️ Restaurant name
        NavigationLink {
            UserScannedRestaurantDetailView(restaurant:
restaurant)
        } label: {
            HStack {
                Text(restaurant.name)
                    .foregroundColor(.white)
                    .font(.system(size: 15, weight: .semibold))

                Spacer()

                Image(systemName: "chevron.right")
                    .foregroundColor(.white.opacity(0.4))
            }
        }
    }
}

```

```

        .font(.system(size: 13, weight: .semibold))
    }
}

// 📄 Menu summary
Text("\((restaurant.menus.count) scanned menu
items")
    .font(.caption)
    .foregroundColor(.white.opacity(0.6))
}
.padding(12)
.background(
    RoundedRectangle(cornerRadius: 12)
        .fill(Color.black.opacity(0.35))
)
}

private fun sectionHeader(
    title: String,
    systemImage: String,
    color: Color = .yellow
) -> some View {
    VStack(alignment: .leading, spacing: 8) {

        HStack(spacing: 10) {
            Image(systemName: systemImage)
                .font(.system(size: 16, weight: .semibold))
                .foregroundColor(color.opacity(0.9))

            Text(title.capitalized)
                .font(.system(size: 18, weight: .bold))
                .foregroundColor(color.opacity(0.9))
        }

        Rectangle()
            .fill(color.opacity(0.25))
            .frame(height: 1)
    }
    .padding(.top, 12)
}

private fun emptyHint(text: String) -> some View {
    Text(text)
        .font(.caption)
        .foregroundColor(.white.opacity(0.5))
        .padding(.leading, 4)
}

```

```
}
```

```
@ViewBuilder
```

```
private func healthInsightRow(
```

```
    healthKitManager: HealthKitManager
```

```
) -> some View {
```

```
//  HealthKit OFF
```

```
if !healthKitManager.isHealthKitEnabled {
```

```
    healthAlertCard(
```

```
        content: {
```

```
            VStack(alignment: .leading, spacing: 10) {
```

```
                //  Title + Button Row
```

```
                HStack(alignment: .center) {
```

```
                    healthInsightItem(
```

```
                        icon: " ",
```

```
                        title: "Health Data Unavailable"
```

```
                    )
```

```
                    Spacer()
```

```
                    Button {
```

```
                        showAppSettings = true
```

```
                    } label: {
```

```
                        HStack(spacing: 6) {
```

```
                            Image(systemName:
```

```
"gearshape.fill")
```

```
                                .font(.system(size: 12, weight:
```

```
                                .semibold))
```

```
                            Text("Open")
```

```
                                .font(.system(size: 13, weight:
```

```
                                .semibold))
```

```
                        }
```

```
                        .foregroundColor(.white)
```

```
                        .padding(.vertical, 6)
```

```
                        .padding(.horizontal, 10)
```

```
                        .background(
```

```
                            RoundedRectangle(cornerRadius: 8)
```

```
                                .fill(Color.white.opacity(0.18))
```

```
                        )
```

```
                    }
```

```
                }.buttonStyle(.plain)
```

```

    }

    // 💡 Description
    Text(
      "HealthKit access is turned off. Enable
HealthKit to receive personalized health insights."
    )
    .font(.system(size: 14))
    .foregroundColor(.white.opacity(0.85))
  }
},
accent: Color.white.opacity(0.6),
background: Color.white.opacity(0.08)
)

// 2 HealthKit ON + 하나라도 경고 조건 존재
} else if healthKitManager.hasDiabetes ||
healthKitManager.hasHypertension {

  healthAlertCard(
    content: {
      VStack(alignment: .leading, spacing: 10) {

        if healthKitManager.hasDiabetes {
          Button {
            // TODO: Diabetic-Friendly 상세 화면으
로 이동
          } label: {
            healthInsightItem(
              icon: "🩸",
              title: "Glucose Pattern Detected",
              showsChevron: true
            )
          }
          .buttonStyle(.plain)
        }

        if healthKitManager.hasHypertension {
          Button {
            // TODO: Low Sodium 상세 화면으로 이동
          } label: {
            healthInsightItem(
              icon: "❤️",
              title: "Blood Pressure Pattern
Detected",
              showsChevron: true
            )
          }
          .buttonStyle(.plain)
        }
      }
    )
  )
}

```



```

        )
    }
    .buttonStyle(.plain)
}
}
},
accent: Color.red.opacity(0.9),
background: Color.red.opacity(0.18)
)

} else {

    healthAlertCard(
        content: {
            healthInsightItem(
                icon: "🌟",
                title: "No notable health patterns detected"
            )
        },
        accent: Color.green.opacity(0.85),
        background: Color.green.opacity(0.15)
    )
}
}

private func healthAlertCard<Content: View>(
    @ViewBuilder content: () -> Content,
    accent: Color,
    background: Color
) -> some View {

    VStack(alignment: .leading) {
        content()
    }
    .padding(16)
    .background(
        RoundedRectangle(cornerRadius: 16)
        .fill(background)
        .overlay(
            RoundedRectangle(cornerRadius: 16)
            .stroke(accent.opacity(0.35), lineWidth: 1)
        )
    )
}

private func healthInsightItem(

```

```
    icon: String,  
    title: String,  
    showsChevron: Bool = false  
  ) -> some View {
```

```
HStack(spacing: 12) {  
  Text(icon)  
  .font(.system(size: 18))
```

```
Text(title)
    .font(.system(size: 15, weight: .semibold))
    .foregroundColor(.white.opacity(0.9))
```

Spacer()

```
if showsChevron {
    Image(systemName: "chevron.right")
        .font(.system(size: 13, weight: .semibold))
        .foregroundColor(.white.opacity(0.6))
}
```

```
private func favoriteRecipeRow(_ recipe:
FavoriteRecipe) -> some View {
    let canonicalName =
        NameNormalizer.canonical(recipe.name, kind:
.recipe)
```

```

return HStack {
    NavigationLink {
        RecipeDetailLoadingView(name:
canonicalName)
    } label: {
        VStack(alignment: .leading, spacing: 4) {
            Text(NameNormalizer.display(recipe.name,
kind: .recipe))
                .foregroundColor(.white)
                .font(.system(size: 15, weight: .medium))

            Text(
                FavoriteDateFormatter.shared.string(from:
recipe.savedAt)
            )
                .font(.caption2)
                .foregroundColor(.white.opacity(0.5))
        }
    }
}

```

```

    }
    .foregroundColor(.white)
    .font(.system(size: 15, weight: .medium))
    .frame(maxWidth: .infinity, alignment: .leading)
}

Button {
    favoriteRecipes.toggle(
        canonicalName,
        origin: recipe.origin,
        decision: .manual
    )
} label: {
    Image(systemName: "star.fill")
        .foregroundColor(.yellow.opacity(0.85))
}
.buttonStyle(.plain)
}
.padding(12)
.background(
    RoundedRectangle(cornerRadius: 12)
        .fill(Color.black.opacity(0.35))
)
}

private func favoriteRestaurantRow(_ restaurant:
FavoriteRestaurant) -> some View {
    let canonicalName =
        NameNormalizer.canonical(restaurant.name, kind:
.restaurant)

    let displayName =
        NameNormalizer.display(restaurant.name, kind:
.restaurant)

    return HStack {
        NavigationLink {
            AutoMenuLoaderView(placeName:
displayName)
        } label: {
            VStack(alignment: .leading, spacing: 4) {
                Text(displayName)
                    .foregroundColor(.white)
                    .font(.system(size: 15, weight: .medium))

                Text(

```

```

FavoriteDateFormatter.shared.string(from:
restaurant.savedAt)
    )
    .font(.caption2)
    .foregroundColor(.white.opacity(0.5))
}
.frame(maxWidth: .infinity, alignment: .leading)
}

Button {
    favoriteRestaurants.toggle(
        canonicalName,
        origin: restaurant.origin,
        decision: .manual
    )
} label: {
    Image(systemName: "star.fill")
        .foregroundColor(.yellow.opacity(0.85))
}
.buttonStyle(.plain)
}
.padding(12)
.background(
    RoundedRectangle(cornerRadius: 12)
        .fill(Color.black.opacity(0.35))
)
}

private func openAppSettings() {
    if let url = URL(string:
UIApplication.openSettingsURLString) {
        UIApplication.shared.open(url)
    }
}

private func myRecipeRow(_ recipe: UserRecipe) ->
some View {
    NavigationLink {
        RecipeDetailLoadingView(name: recipe.name)
    } label: {
        HStack {
            Text(recipe.name)
                .foregroundColor(.white)
                .font(.system(size: 15, weight: .medium))

            Spacer()

```

```

        Image(systemName: "chevron.right")
            .foregroundColor(.white.opacity(0.4))
            .font(.system(size: 13, weight: .semibold))
    }
    .padding(12)
    .background(
        RoundedRectangle(cornerRadius: 12)
            .fill(Color.black.opacity(0.35))
    )
}
}

```

```

private func myIngredientRow(
    _ ingredient: IngredientStore.CustomIngredient
) -> some View {
    HStack {
        Text(ingredient.name)
            .foregroundColor(.white)
            .font(.system(size: 15, weight: .medium))

        Spacer()

        Text(ingredient.category.rawValue)
            .foregroundColor(.white.opacity(0.6))
            .font(.caption)
    }
    .padding(12)
    .background(
        RoundedRectangle(cornerRadius: 12)
            .fill(Color.black.opacity(0.35))
    )
}

```

```

private static let favoriteDateFormatter: DateFormatter
= {
    let f = DateFormatter()
    f.dateStyle = .medium // 예: Jan 12, 2025
    f.timeStyle = .none
    return f
}()
}

```

Show less ^

normalize가 어떻게 잘못됐는지 확실히 보인다. 저렇게 나오면 안
돼. 수정해라.

왜 여전히 `_s_`이 들어가나?

됐다. 이제 하나만 보인다. 근데 토글 버튼이 안된다. 아래 토글함수로직 분석해라.

```
func toggle(  
    _ name: String,  
    origin: FavoriteOrigin = .manual,  
    decision: FavoriteDecision = .manual  
) {
```

```
let normalized = normalize(name)
```

```
// 🗝️ 해당 식당의 "마지막 이벤트" 찾기
```

```
let lastEvent = favorites  
  .filter { normalize($0.name) == normalized }  
  .sorted(by: { $0.savedAt > $1.savedAt })  
  .first
```

```
let shouldHide = !(lastEvent?.isHidden ?? true)
```

```
// 설명:
```

```
// - 마지막 이벤트가 visible(false) → hide(true)
```

```
// - 마지막 이벤트가 hidden(true) → show(false)
```

```
// - 이벤트가 없으면 → show(false)
```

```
favorites.append(  
  FavoriteRestaurant(  
    name: normalized,  
    savedAt: Date(),  
    origin: origin,  
    decision: decision,  
    isHidden: shouldHide  
  )  
)  
  
save()  
}
```

Show less ^

리스트에 보여줄때도 비 정규화 이름을 쓴다. 그리고 recipe detail view로 넘겨줄때도 비정규화 이름을쓴다. 어딘지 짚어라

모든곳을 ChainNameNormalizer.normalize으로 틀어막았는데
어떻게 두번째것이 들어갔나.

첨부된 뷰에서 favorite된 식당은 노란색으로 표시할수 있나?

my menucal UI를 home screen하고 recipes UI처럼 9개의 버튼으로 할거야. 버튼 모양은 삼각형. 지금 8의 item들이 있는데 여기서 menu to photo mapping을 넣을거야. 총 9개의 버튼이름을 제안해.

My MenuCal should be triangle grid.

triangles do not look good. suggest another shape.

make the hexagon more round. width and height should be the same.

```
struct SoftHexagonShape: Shape {

    var cornerRadius: CGFloat = 8

    func path(in rect: CGRect) -> Path {
        let w = rect.width
        let h = rect.height
        let r = cornerRadius

        let points = [
            CGPoint(x: w * 0.25, y: 0),
            CGPoint(x: w * 0.75, y: 0),
            CGPoint(x: w, y: h * 0.5),
            CGPoint(x: w * 0.75, y: h),
            CGPoint(x: w * 0.25, y: h),
            CGPoint(x: 0, y: h * 0.5)
        ]

        var path = Path()
        path.move(to: points[0])

        for i in 0..
```

}

}

Show less ^

it got completely messed up. fix.

make angles round.

좋아, 지금 상태는 정육각형은 안정적인데 각이 너무 날카로운 상태야.

해결은 명확해:

👉 꼭짓점마다 원호(arc)를 넣어서 각을 둥글게 만들면 된다.

(지금처럼 quadCurve 트릭 ❌, 수학적으로 정확하게 ○)

아래는 깨지지 않고, 프리미엄 느낌 나는 "rounded hexagon" 정답 코드야.

✅ 최종 권장: Rounded Hexagon (각 둥글게)

✓ 특징

- width == height 전제
- 각 꼭짓점에 **균일한 radius**
- LazyVGrid / animation / drag 전부 안전
- 지금 스크린샷에서 바로 "각만 둥글어짐"

🔧 SoftRoundedHexagonShape (교체)

Swift



```
struct SoftHexagonShape: Shape {  
  
    /// 0.08 ~ 0.18 권장  
    var cornerRadiusRatio: CGFloat = 0.14  
  
    func path(in rect: CGRect) -> Path {  
  
        let size = min(rect.width, rect.height)  
        let r = size / 2
```

```

let center = CGPoint(x: rect.midX, y: rect.midY)

let cornerRadius = r * cornerRadiusRatio
let angleStep = CGFloat.pi / 3 // 60°

// 꼭짓점 방향 (flat-top hexagon)
let angles: [CGFloat] = [
    0,
    angleStep,
    angleStep * 2,
    angleStep * 3,
    angleStep * 4,
    angleStep * 5
]

// 꼭짓점 위치
let points = angles.map { angle in
    CGPoint(
        x: center.x + cos(angle) * (r - cornerRadius),
        y: center.y + sin(angle) * (r - cornerRadius)
    )
}

var path = Path()

// 시작점
path.move(to: points[0])

for i in 0..

```

🎯 사용 예 (지금 Hex 버튼에 그대로)



```
SoftHexagonShape(cornerRadiusRatio: 0.14)
    .fill(
        FeatureButtonGradients.gradient(for: feature.gradientID)
    )

SoftHexagonShape(cornerRadiusRatio: 0.14)
    .stroke(Color.white.opacity(0.18), lineWidth: 1)
```

- 👉 0.12 → 살짝 둥글
- 👉 0.16 → 더 부드러움
- 👉 0.18 이상 → pill 느낌 (비추천)

🧠 왜 이 방식은 안 깨지나?

- ❌ control point가 rect 밖으로 튀지 않음
- ❌ path bounding box 확장 없음
- ✅ 모든 곡선이 hex 내부에서만 형성
- ✅ SwiftUI hit-test / shadow 정상

✨ 시각적 결과

지금 스샷 기준으로:

- ❌ 날카로운 "게임 버튼 느낌"
- ✅ iOS 시스템 아이콘에 가까운 부드러운 각
- ✅ My MenuCal 톤에 훨씬 잘 맞음

다음으로 추천할 미세 조정 (선택)

1. 중앙(My Kitchen)만

⌘ Swift

```
.scaleEffect(1.06)
```



2. Analytics만 살짝 강조

⌘ Swift

```
.shadow(color: .white.opacity(0.25), radius: 12)
```



3. TODO 버튼 채도 ↓

⌘ Swift

```
.saturation(0.75)
```



원하면

- 가장 예쁜 **cornerRadiusRatio** 값 같이 잡아줄 수도 있고
- ****iOS 18 스타일 "squircle-hex hybrid"****도 만들어줄 수 있어.

다음은 어디까지 갈까?



make the length of all sides same.

좋은 포인트야. 지금 요구는 ****각을 둥글게 + 모든 변의 길이가 동일한 정규 육각형****이야.

이건 감으로 quadCurve를 섞으면 절대 안 되고, **정규육각형의 기하를 그대로 유지한 상태에서 코너만 라운딩**해야 한다.

아래는 수학적으로 정규 + 안정적으로 둥근 **hexagon**이다.

지금 깨졌던 원인도 같이 해결된다.

✅ 핵심 정리 (중요)

- ✓ 모든 변 길이 동일 (regular hexagon)
- ✓ width == height 전제
- ✓ 각 꼭짓점에 동일한 radius
- ✓ path가 rect 밖으로 절대 안 나감
- ✓ SwiftUI shadow / hit test / animation 안전

🔧 정답: Regular Rounded Hexagon

✓ 방식

1. 정규 육각형 꼭짓점 6개를 정확한 각도로 계산
2. 각 변의 시작/끝을 **cornerRadius**만큼 잘라냄
3. 꼭짓점마다 **addArc** 로 둥글게 연결

★ 최종 코드 (이걸로 교체)

⌘ Swift



```
struct SoftHexagonShape: Shape {
```

```
    /// 권장: 6 ~ 12 (size 100 기준)
```

```

var cornerRadius: CGFloat = 10

func path(in rect: CGRect) -> Path {

    let size = min(rect.width, rect.height)
    let center = CGPoint(x: rect.midX, y: rect.midY)

    // 정규 육각형 반지름 (flat-top)
    let radius = size / 2

    // 각도 (flat-top hexagon)
    let angles: [CGFloat] = [
        0,
        .pi / 3,
        2 * .pi / 3,
        .pi,
        4 * .pi / 3,
        5 * .pi / 3
    ]

    // 꼭짓점 좌표
    let vertices = angles.map { angle in
        CGPoint(
            x: center.x + cos(angle) * radius,
            y: center.y + sin(angle) * radius
        )
    }

    var path = Path()

    for i in 0..

```

```

        if i == 0 {
            path.move(to: p1)
        } else {
            path.addLine(to: p1)
        }

        // 둥근 꼭짓점
        path.addQuadCurve(
            to: p2,
            control: current
        )
    }

    path.closeSubpath()
    return path
}
}

```

권장 파라미터 (지금 UI 기준)

Swift



```

SoftHexagonShape(cornerRadius: 10)
    .frame(width: 104, height: 104)

```

cornerRadius	느낌
6	거의 각진
8	자연스러움
10	✅ 가장 안정적
12	살짝 pill 느낌

왜 이걸 “진짜 정규”인가?

- 모든 꼭짓점이 같은 radius
- 모든 변이 동일한 길이
- 코너 처리 전/후에도 기하 구조 유지
- 시각적으로도 “찌그러짐 / 늘어짐 없음”

이전에 깨졌던 이유 요약

- ❌ mid-point 기반 quadCurve → 변 길이 불균형

- ❌ control point가 중심 쪽으로 끌려가 shape 왜곡
- ❌ bounding box 초과 → 스샷처럼 전체가 퍼짐

이 코드에서는 그 문제 전부 없음.

이제 이 상태면:

- My MenuCal 홈에 어울리고
- box / circle / hex 계열 중 가장 성숙한 버튼 형태

다음 단계로는

- 👉 중앙 버튼만 살짝 키우기,
- 👉 Analytics만 edge glow,
- 👉 Health Insights만 pulse animation

같은 UX 디테일 들어갈 수 있어.

어디까지 다듬을까?



i'm going to open a new chat. remember the conversation.

알겠어 👍

다만 중요한 점 하나만 정확히 짚고 갈게.

⚠️ 내가 “새 채팅”을 넘어 기억을 유지하는 것은 자동이 아니야

새 채팅을 열면 이 대화 내용은 기본적으로 이어지지 않는다.

지금 같은 UI/Shape/FeatureButton 설계 맥락은 새 세션에서 사라져.

✅ 가장 안전한 방법 (강력 추천)

새 채팅 첫 메시지에 아래 요약を 그대로 붙여서 시작해줘.

그러면 내가 즉시 이어서 작업할 수 있어.

📌 새 채팅 시작용 컨텍스트 요약 (복붙용)

MenuCal iOS app (SwiftUI).



We redesigned the My MenuCal Preferences screen.
– Replaced box and triangle buttons.

- Settled on a regular hexagon button shape.
- Requirements:
 - All 6 sides must be equal (true regular hexagon)
 - Width == height
 - Rounded corners (soft hexagon)
 - Uses Shape with correct geometry (no mid-point quad hacks)
- Current solution uses vertex-based regular hexagon with rounded corners using
- FeatureButtonHexView uses SoftHexagonShape(cornerRadius: ~10, frame ~104x104)
- FeatureButtonGridView already supports .box / .circle / .triangle styles.
- We are now polishing visual quality and interaction details.

Continue from here.

🧠 왜 이게 중요한지 (짧게)

- 너는 지금 디자인 + 기하 + SwiftUI Shape를 동시에 다루고 있음
- 이건 "단발 질문"이 아니라 연속 설계 작업
- 컨텍스트 없으면 내가 다시 삼각형부터 얘기하게 됨 🤖

선택지 정리

-  추천: 위 요약 붙이고 새 채팅 시작
-  그냥 새 채팅 → 다시 설명 필요
-  "기억해줘" → 현재는 세션 간 자동 기억 불가

새 채팅 열고 요약 붙이면

👉 바로 **hexagon polish / shadow / hover / animation / spacing** 이어서 가자.

기다릴게.

